



ALGORITMOS Y ESTRUCTURAS DE DATOS



TIAGO PUJIA
UNLAM – INGENIERÍA INFORMÁTICA
2025

Introducción

Este libro fue escrito durante el primer cuatrimestre del año 2025, en el marco de la cursada de los profesores *Federico Pezzola* y *Brian Jordi* en la comisión 1900 [1 = Lunes, 900 = Noche].

El contenido fue recopilado a partir del material teórico de MIEL (de todas las comisiones), videos de los profesores y clases.

Para cualquier información faltante, errores o correcciones, se agradece contactar a través del usuario "Tiago_nahuel__".

Contenido

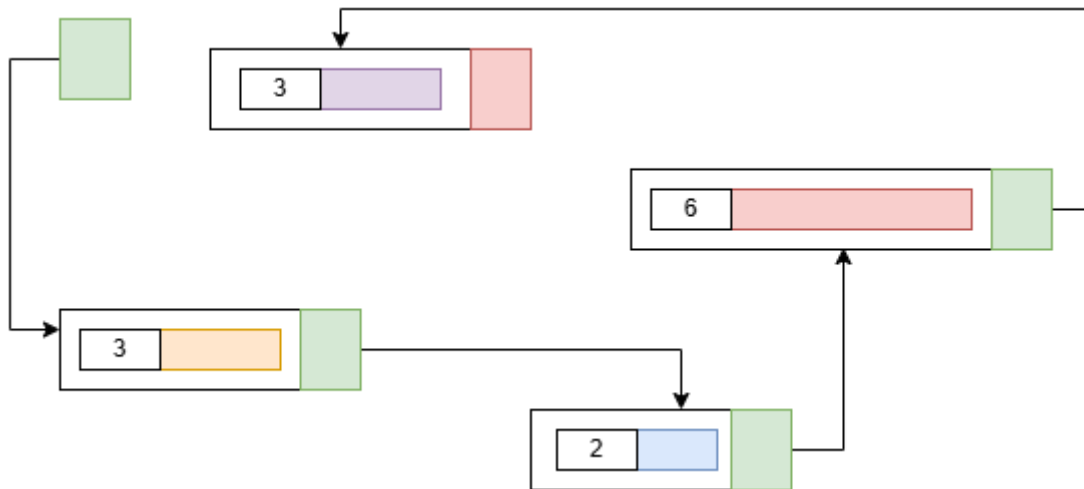
Lista Simplemente Enlazada	7
TDA Pila	8
Explicación Problema	8
Pila Estática	9
Estrategia Problema	9
Estructura	10
Crear Pila	10
Vaciar Pila.....	10
Pila Vacía	11
Pila Llena	11
Poner en Pila	12
Ver Tope	13
Sacar Pila	14
Pila Dinámica.....	15
Estrategia Problema	15
Estructura	16
Crear Pila	16
Pila Vacía	16
Poner en Pila	17
Pila Llena	20
Ver Tope	21
Sacar de Pila.....	21
Vaciar Pila.....	24
Utilizando la Pila	26
TDA Cola	29
Explicación Problema	29
Cola Dinámica	30
Estrategia Problema	30
Estructura	31
Crear Cola	31

Cola Vacía	31
Cola Llena.....	32
Poner en Cola	33
Sacar de Cola	36
Ver Frente de la Cola	39
Vaciar Cola.....	40
Cola Estática	42
TDA Lista.....	43
Explicación Problema	43
Estrategia para el Problema.....	44
Estructura.....	45
Crear Lista.....	45
Operaciones Al Principio.....	46
Insertar Nodo al Principio	46
Ver Nodo del Principio.....	47
Sacar Nodo del Principio.....	47
Desplazamiento en TDA Lista	48
Operaciones al Final	52
Insertar Nodo al Final	52
Ver Nodo del Final.....	56
Sacar Nodo del Final.....	59
Operaciones en Posiciones Determinadas	59
Insertar Nodo en Posición Determinada	60
Ver Nodo en Posición Determinada	64
Sacar Nodo en Posición Determinada.....	65
Insertar Ordenado	70
Map, Filter, Reduce y Búsqueda	71
Map	71
Reduce	73
Filter	75
Búsqueda	81

Sacar Duplicados	83
Invertir Lista	84
TDA Circular	85
TDA Pila Circular.....	85
Estrategia Problema	85
Coincidencias con la Pila Aprendida	85
Poner en Pila Circular.....	82
Ver Tope	84
Sacar de Pila Circular	85
Vaciar Pila Circular	88
TDA Cola Circular.....	89
Estrategia Problema	89
Coincidencias con Pila Circular	89
Poner en Cola Circular.....	90
TDA Lista Doble Mente Enlazada	92
Problemática y Estrategia	92
Estructura.....	93
Crear Lista.....	93
Desplazamiento.....	94
Operaciones al Principio	95
Insertar al Principio.....	95
Ver Principio.....	98
Sacar del Principio	99
Operaciones al Final	102
Insertar al Final.....	102
Ver Final.....	103
Sacar del Final.....	104
Operaciones en Posiciones Determinadas	105
Sacar Nodo en Posición Determinada.....	105
Vaciar Lista	106
Map, Filter, Reduce y Búsqueda	107

Map (recorrer)	107
Reduce	108
Filter	109
Búsqueda por Clave	110
Elimina por Clave con Lista Ordenada.....	111
Sacar Duplicados	112
Insertar Ordenado	113
Recursividad	115
Definición.....	115
Operaciones para TDA Lista Simple.....	117
Insertar al Final.....	117
Ver Ultimo.....	118
TDA Árbol Binario.....	119
Problemática y Estrategia	119
Recorrer a Nivel Semántico.....	121
Pre-Orden – RID (Raíz, Izquierda, Derecha).....	121
In Orden – IRD (Izquierda, Raíz, Derecha)	122
Pos Orden – IRD (Izquierda, Derecha y Raíz)	122
Estructura.....	123
Crear Árbol.....	123
Recorrer a Código	123
Contar	125
Cantidad de Nodos	125
Altura del Árbol.....	125
Cantidad de Nodos hasta N Nivel	127
Insertar.....	129

Lista Simplemente Enlazada



Una **lista simplemente enlazada** es un conjunto de datos dinámicos llamados nodos. Un **nodo** es una **estructura dinámica** (reservada mediante malloc) que contiene tres componentes principales como mínimo:

- **info**: Un puntero que apunta donde se almacena la información
- **tamInfo**: Un valor que guarda el tamaño (en bytes) de la información
- **sig**: Un puntero al nodo siguiente de la lista.

Se llama *simplemente enlazada* porque cada nodo sólo conoce al que viene después a través del puntero sig, formando una cadena unidireccional.

Una de las grandes ventajas de utilizar listas simplemente enlazadas en lugar de vectores es que cada nodo puede estar ubicado en cualquier parte de la memoria (no necesitan estar en posiciones contiguas como los vectores). Esto ofrece beneficios importantes como:

- **Flexibilidad en el tamaño**: no hace falta reservar memoria para todos los elementos de antemano.
- **Facilidad para reorganizar los datos**: si queremos cambiar el orden de los nodos, **no hace falta mover la información**, sólo hay que cambiar los punteros (sig).

No obstante. Por ahora solo veremos la teoría de este concepto. Lo vamos a utilizar para TDAs de tipo dinámico, para los estáticos se seguirán usando vectores.

[Herramienta: Visualizador de C](#)

TDA Pila

Explicación Problema

Utilizaremos actos de la vida real como ejemplos para comprender mejor.

Un TDA Pila se puede entender como **una pila de platos**, donde cada plato representa un dato o elemento. Si queremos **agregar** un plato, lo hacemos **por arriba o el tope**, colocando el nuevo encima de los demás. De igual manera, si queremos **sacar** un plato, **solo podemos sacar el último que pusimos**.



Las ideas centrales de esta estructura son:

- Solo se puede interactuar con el tope
- No se puede recorrer
- No se pueden tocar los procesos internos del TDA en el proyecto
- La información esta apilada en un único sentido

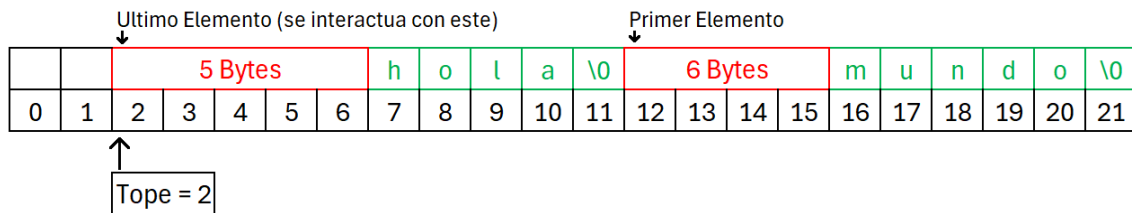
Las operaciones principales que se pueden realizar sobre una pila son:

- Crear Pila -> Inicializar la pila vacía
- Poner en Pila -> Apilar un elemento
- Sacar de Pila -> Desapilar
- Ver Tope -> Ver elemento superior sin quitarlo
- Vaciar Pila -> Eliminar todos los elementos
- Pila Vacía -> Consultar si no hay elementos
- Pila Llena -> Consultar si la pila está llena

Hay 2 formas de implementarlo, con memoria dinámica (uso de nodos) o memoria estática (uso de vectores estáticos).

Pila Estática

Estrategia Problema



Representación gráfica del funcionamiento interno de una pila estática.

Se hace uso de un vector convencional con una longitud N. Los elementos, se van colocando de fin a inicio del vector. Por lo que, la pila comienza según la longitud del vector y termina hasta que llegue a cero.

Un elemento puede ocupar varios espacios de memoria en el vector, no solo uno. Por lo que, para poder gestionar correctamente la información, junto con cada dato se almacena a su lado la cantidad de bytes que ocupa dicho elemento.

Se tiene una variable llamada tope que indicara la posición en el vector del último elemento agregado junto su indicador de espacio. Este se desplaza de derecha a izquierda.

Estructura

```
typedef struct
{
    char vec[TAM_PILA]; // Vector que almacena los datos en forma genérica
    size_t tope; // Indica la posición del próximo espacio libre
} t_pila;
```

Es una pila estática, lo que significa que su espacio está predefinido (no cambia en tiempo de ejecución). Internamente:

- Usa un vector de caracteres como espacio de almacenamiento genérico.
- Tiene una variable tope que indica info próximo espacio libre

Esta estructura permite guardar cualquier tipo de dato, ya que se guarda también info tamaño del dato (mediante sizeof(size_t)), justo antes del dato, como si fuera un "encabezado".

Crear Pila

```
void crear_pila(t_pila *pila)
{
    pila->tope = TAM_PILA;
}
```

Esta función simplemente inicializa la pila. Coloca info tope al final del vector (es decir, la pila comienza vacía).

Vaciar Pila

```
void vaciar_pila(t_pila *pila)
{
    pila->tope = TAM_PILA;
}
```

Simplemente resetea info tope al final del vector, lo que indica que la pila está vacía.

Pila Vacía

```
int pila_vacia(t_pila *pila)
{
    return pila->tope == TAM_PILA ? PILA_VACIA : EXITO;
}
```

Si el tope no se ha movido, entonces no hay nada en la pila.

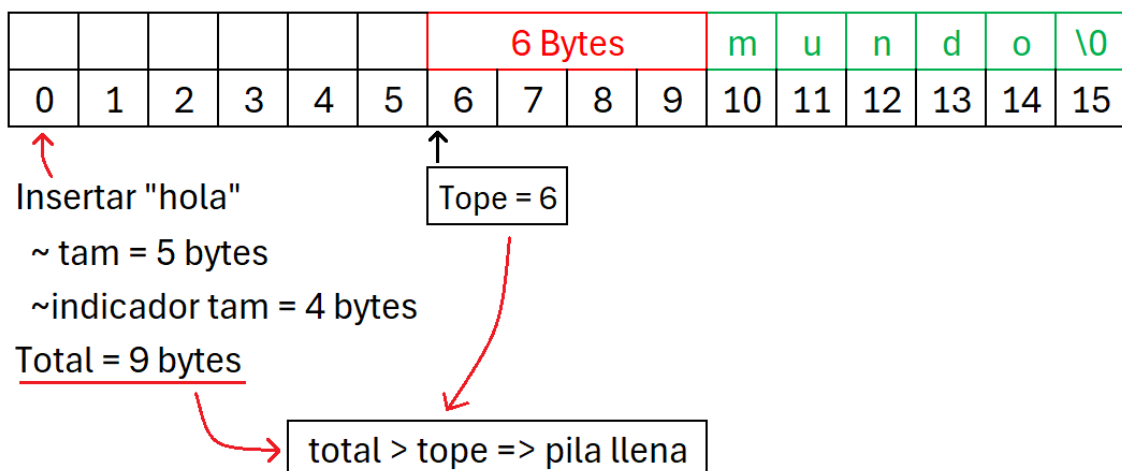
Pila Llena

```
int pila_llena(t_pila *pila, size_t tam_el)
{
    return pila->tope < tam_el + sizeof(size_t) ? PILA_LLENA : EXITO;
}
```

Esta función determina si hay espacio suficiente para insertar un nuevo dato. Para eso, revisa si info tope tiene margen para guardar:

- Tamaño del dato.
- Info tamaño del dato.

Para que pueda entenderse mejor, se tiene el siguiente grafico:



Se tiene una pila con un tope en la posición 6, y se quiere agregar nueva información. En este caso se agrega la palabra "hola" que son 5 bytes (incluyendo el carácter nulo), y se tienen que sumar 4 bytes por guardar el tamaño de la palabra; 5 + 4 es 9 espacios, es mayor que 6 por lo que no entra.

Poner en Pila

```
int poner_en_pila(t_pila *pila, void *info, size_t tamInfo)
{
    // Verificamos si hay suficiente espacio para el elemento y su tamaño
    if(pila->tope < tamInfo + sizeof(size_t))
    {
        return PILA_LLENA;
    }

    // Reservamos espacio para el dato y lo copiamos al vector
    pila->tope -= tamInfo;
    memcpy(pila->vec + pila->tope, info, tamInfo);

    // Reservamos espacio para guardar el tamaño y lo copiamos
    pila->tope -= sizeof(size_t);
    memcpy(pila->vec + pila->tope, &tamInfo, sizeof(size_t));

    return EXITO;
}
```

Inserta un nuevo elemento en la pila, junto con su tamaño para facilitar la extracción posterior. Su proceso es:

1. Verifica si hay espacio disponible.
2. Reduce el tope para reservar espacio para el dato nuevo(*info).
3. Copia el contenido del dato nuevo al vector.
4. Reduce de nuevo el tope para reservar espacio para el tamaño del dato.
5. Guarda ese tamaño en el vector.

Ver Tope

```
#define MIN(X,Y) X > Y ? Y : X // Obtiene el mínimo entre dos valores

int ver_tope(t_pila *pila, void *info, size_t tamInfo)
{
    size_t tamInfoVec;

    // Verificamos si está vacío
    if(pila->tope == TAM_PILA)
    {
        return PILA_VACIA;
    }

    // Se copia el tamaño guardado del elemento que está en el tope
    memcpy(&tamInfoVec, pila->vec + pila->tope, sizeof(size_t));

    // Luego copiamos el dato en sí hacia 'info'
    memcpy(info, pila->vec + pila->tope + sizeof(size_t), MIN(tamInfoVec,
tamInfo));

    return EXITO;
}
```

Permite ver el último elemento ingresado sin sacarlo de la pila. Para esto:

1. Verifica si la pila no está vacía
2. Guarda en una variable el tamaño de la información en el vector
3. Copia la información del vector a la variable parámetro *info

Como se puede ver, se hace uso de una macro de MIN. Esto, debido a que el dato guardado en el vector, puede ser más chico o más grande de lo que en realidad es. Y para no tocar memoria que no corresponde con el malloc, se opta por el mínimo.

Sacar Pila

```
#define MIN(X,Y) X > Y ? Y : X // Obtiene el mínimo entre dos valores

int sacar_de_pila(t_pila *pila, void *info, size_t tamInfo)
{
    size_t tamInfoVec;

    // Si el tope está al final, la pila está vacía
    if(pila->tope == TAM_PILA)
    {
        return PILA_VACIA;
    }

    // Leemos el tamaño del dato que fue guardado justo antes del dato
    memcpy(&tamInfoVec, pila->vec + pila->tope, sizeof(size_t));

    // Avanzamos el tope para "saltar" el tamaño
    pila->tope += sizeof(size_t);

    // Copiamos el dato al destino
    memcpy(info, pila->vec + pila->tope, MIN(tamInfo, tamInfoVec));

    // Avanzamos el tope para liberar el espacio del dato
    pila->tope += tamInfoVec;

    return EXITO;
}
```

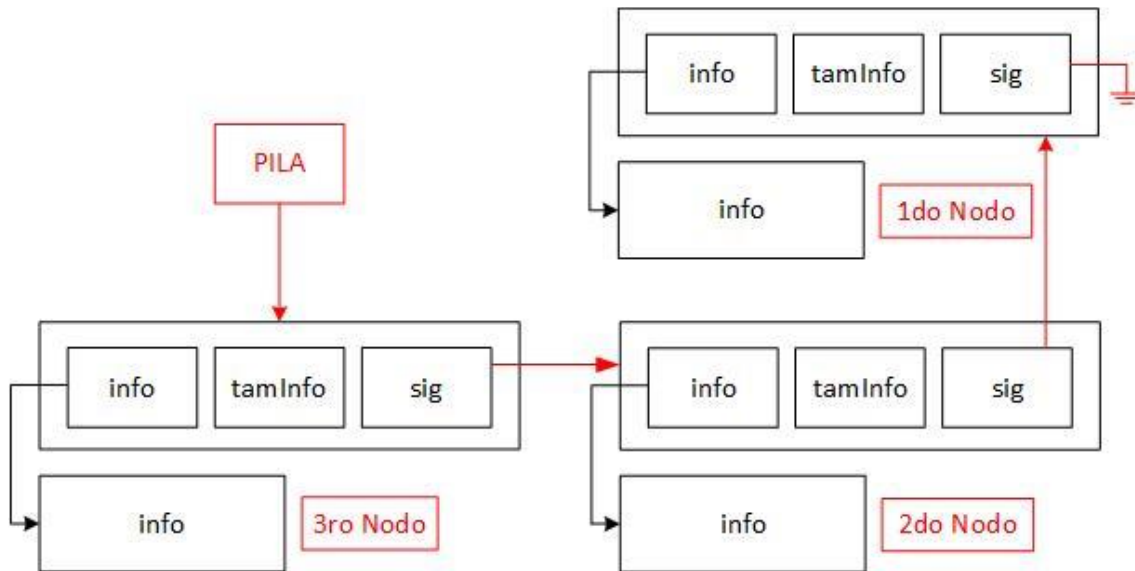
Esta operación extrae el dato del tope de la pila. El flujo es el siguiente:

1. Verifica si la pila está vacía.
2. Lee el tamaño guardado.
3. Avanza el tope para "saltar" ese tamaño.
4. Copia el contenido del dato.
5. Avanza el tope otra vez para liberar ese espacio.

La razón por la que se utiliza MIN, es lo mismo que para la función anterior

Pila Dinámica

Estrategia Problema



Representación gráfica del funcionamiento interno de una pila dinámica.

• Funcionamiento de los Nodos

Para implementar un TDA Pila Dinámico, se hace uso de una estructura de nodos. Cada vez que se agrega un nuevo elemento, se debe crear dinámicamente un nuevo nodo utilizando malloc.

Este nodo no solamente reserva memoria para su propia estructura (campos), sino que también reserva un espacio adicional para almacenar la información que va a contener. Para esto, dentro del nodo se incluye un puntero (void* info) que también apunta a un bloque de memoria dinámicamente reservado mediante malloc. Por lo tanto, el nodo tiene dos áreas de memoria separadas:

- Una para su propia estructura (*info, tamInfo, *sig)
- Para la información que almacena

Cada nodo puede estar almacenado en cualquier lugar de la memoria, ya que la memoria dinámica no garantiza ubicaciones contiguas. Esto significa que los nodos no están uno seguido del otro como ocurriría en un vector, sino que están **dispersos** por toda la memoria, conectados solo por punteros (sig) que forman la cadena de enlaces.

- **Explicación de la Estrategia del Problema**

La **pila** se maneja a través de un **único puntero**, que apunta siempre al **último nodo insertado**. Cada nodo, a su vez, contiene un puntero que apunta al **nodo anterior** en la pila, formando así una cadena de nodos.

Estructura

```
typedef struct sNodo // Estructura sNodo que hace referencia a tNodo
{
    void *info; // Puntero genérico, guarda el elemento con malloc
    size_t tamInfo; // Tamaño en bytes de info
    struct sNodo *sig; // Puntero al siguiente nodo
} tNodo;

typedef tNodo* tPila; // Crea un alias; tPila es sinonimo de tNodo*
```

El sNodo es como un alias que se pone (sNodo = tNodo), es necesario porque después de ser creado, recién ahí se lo llama como tNodo, entonces esto es necesario para poder auto-referenciarse con el campo *sig. El typedef es otro sinónimo que se le pone para tPila.

Crear Pila

```
void crearPila(tPila *pila)
{
    *pila = NULL;
}
```

Cuando se crea un puntero, este apuntara a basura si no le asignamos nada. Por lo que, debemos hacer que apunte a la nada (NULL) indicando que está vacío o listo para usarse.

Pila Vacía

```
int pilaVacía(tPila *pila)
{
    return *pila ? PILA_DISPONIBLE : PILA_VACIA;
}
```

No hay mucho más que explicar. Si el puntero tiene una dirección asignada, quiere decir que tiene un elemento (nodo) asignado por lo que no está vacío.

Poner en Pila

```
int ponerEnPila(tPila *pila, const void *info, size_t tamInfo)
{
    tNodo *nodo;
    // Reserva memoria para el nodo
    if(!(nodo = (tNodo*)malloc(sizeof(tNodo))))
    {
        return ERROR_MALLOCC;
    }

    // Reserva memoria para guardar la información
    if(!(nodo->info = malloc(tamInfo)))
    {
        free(nodo);
        return ERROR_MALLOCC;
    }

    memcpy(nodo->info, info, tamInfo); // Se copia el contenido que se quiere
guardar en la pila
    nodo->tamInfo = tamInfo; // Se guarda el tamaño de los datos
    nodo->sig = *pila; // El nuevo nodo apunta al anterior tope de la pila
    *pila = nodo; // El nuevo nodo se convierte en el nuevo tope de la pila

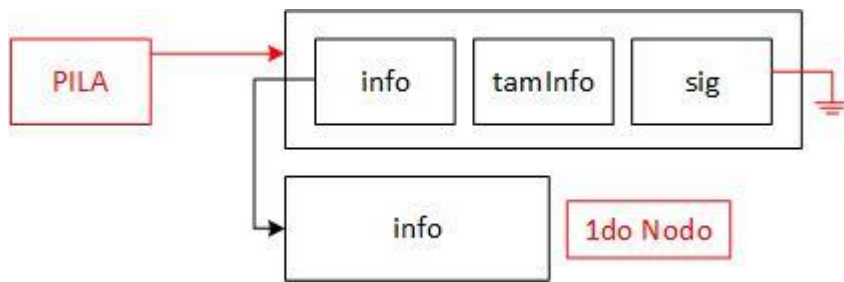
    return EXITO;
}
```

Esta función lo que hace es insertar un nuevo elemento en una pila que está implementada como una lista enlazada. Primero reserva memoria para un nuevo nodo y para la información que va a guardar. Si algo falla, libera la memoria y devuelve un error.

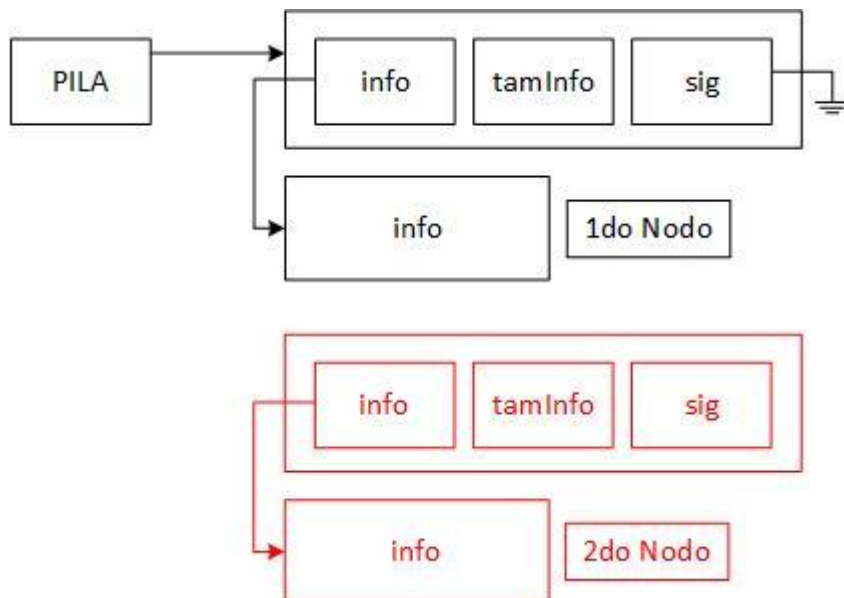
Si todo sale bien, copia el contenido de los parámetros al espacio reservado del nuevo nodo, guarda el tamaño de ese contenido, y enlaza ese nuevo nodo al nodo que estaba antes en la cima de la pila. Finalmente, actualiza el puntero de la pila para que apunte al nuevo nodo, que ahora es el nuevo tope.

En la siguiente página vamos a darle una explicación grafica.

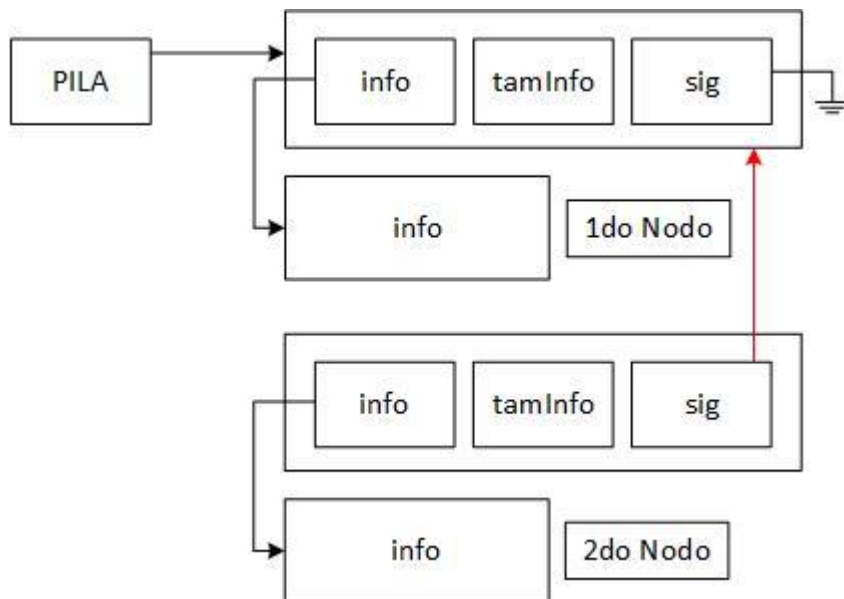
1. Se presenta la siguiente pila con un nodo o elemento ya insertado (el insertado de pila vacía es sencillo de comprender). Pila:



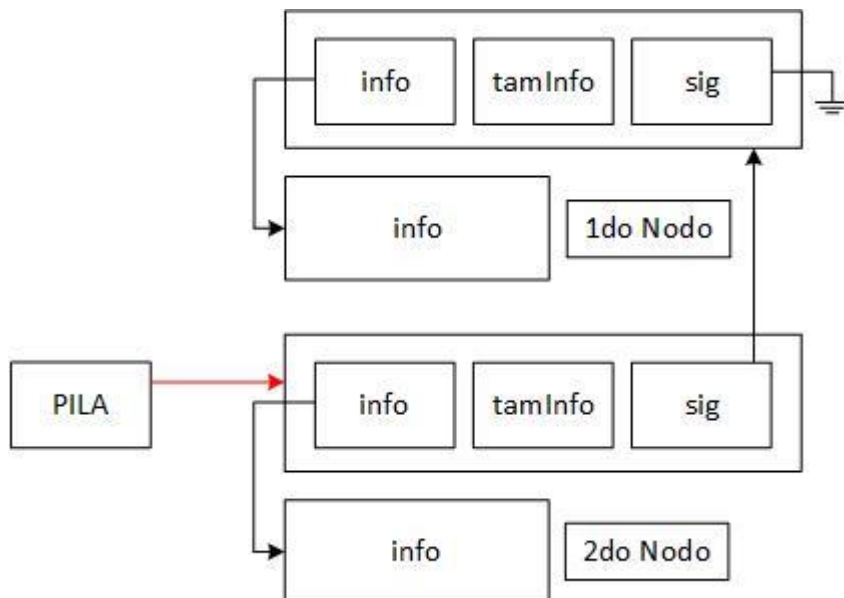
2. Lo primero que se hace, es crear un nuevo nodo y espacio apartado para guardar la información, utilizando malloc:



3. El puntero *sig* del nuevo nodo, apuntara donde va la pila, ósea el ultimo elemento agregado (respetando el concepto de pila):



4. El puntero de la pila, pasara a dirigirse al nuevo nodo creado:



Pila Llena

```
int pilallena(tPila *pila, size_t tamInfo)
{
    tNodo *nue; // Nodo auxiliar

    // Se intenta reservar memoria para un nuevo nodo
    if(!(nue = malloc(sizeof(tNodo))))
    {
        return PILA_LLENA;
    }

    // Se intenta reservar memoria para la información que contendrá ese nodo
    if(!(nue->info = malloc(sizeof(tamInfo))))
    {
        free(nue);
        return PILA_LLENA;
    }

    // Se libera la memoria, ya que solo se quería verificar disponibilidad
    free(nue);
    free(info);

    // Como ninguna asignación fallo, se notifica disponibilidad
    return PILA_LLENA;
}
```

Esta función *pilaLlena* evalúa si hay suficiente memoria disponible para seguir agregando elementos a la pila. Lo hace intentando reservar memoria temporalmente para un nuevo nodo (tNodo) y para el espacio necesario para la información del nuevo dato (tam bytes). Si se les asigna una dirección de memoria a ambos punteros, quiere decir que aún hay espacio disponible.

No obstante, la gracia de crear un TDA Dinámico es crear una interfaz para el usuario. No le interesa si el TDA está lleno o no, solo quiere insertar o quitar elementos. Por lo que solo será una función que estará ahí, pero no será utilizada por el usuario. Aparte que es bastante raro que esté lleno la memoria principal y virtual. Por lo que, también es válido crear la siguiente función para evaluar si está lleno:

```
int pilallena(tPila *pila, size_t tamInfo)
{
    return PILA_LLENA;
}
```

Por palabras del profesor Pezzola, esta función es válida, más para ahorrar tiempo.

Ver Tope

```
#define MIN(X,Y) X > Y ? Y : X // Obtiene el mínimo entre dos valores

int verTopePila(const tPila *pila, void *info, size_t tamInfo)
{
    // Verifica si la pila está vacía (no hay nodos)
    if (!*pila)
    {
        return PILA_VACIA;
    }

    // Copia la información del nodo del tope hacia 'info'
    memcpy(info, (*pila)->info, MIN(tamInfo, (*pila)->tamInfo));

    return EXITO;
}
```

Copiamos el contenido del tope a la dirección parámetro 'info'.

Sacar de Pila

La función *sacarDePila* elimina el nodo que está en el tope de la pila (el más reciente) y obtiene su información.

```
#define MIN(X,Y) X > Y ? Y : X // Obtiene el mínimo entre dos valores

int sacarDePila(tPila *pila, void *info, size_t tamInfo)
{
    tNodo *aux = *pila; // Se guarda el nodo del tope de la pila en un aux

    if(!aux) // Si la pila está vacía, se retorna el código de pila vacía
    {
        return PILA_VACIA;
    }

    *pila = aux->sig; // Se actualiza el tope de la pila al siguiente nodo

    // Se copian los datos del nodo a eliminar al espacio recibido por
    // parámetro.
    // Se copia el mínimo entre info tamaño del dato guardado y el que se pide
    // copiar.
    memcpy(info, aux->info, MIN(tamInfo, aux->tamInfo));

    free(aux->info); // Se libera la memoria del dato del nodo
    free(aux);      // Se libera la memoria del nodo en sí

    return EXITO; // Se retorna que la operación fue exitosa
}
```

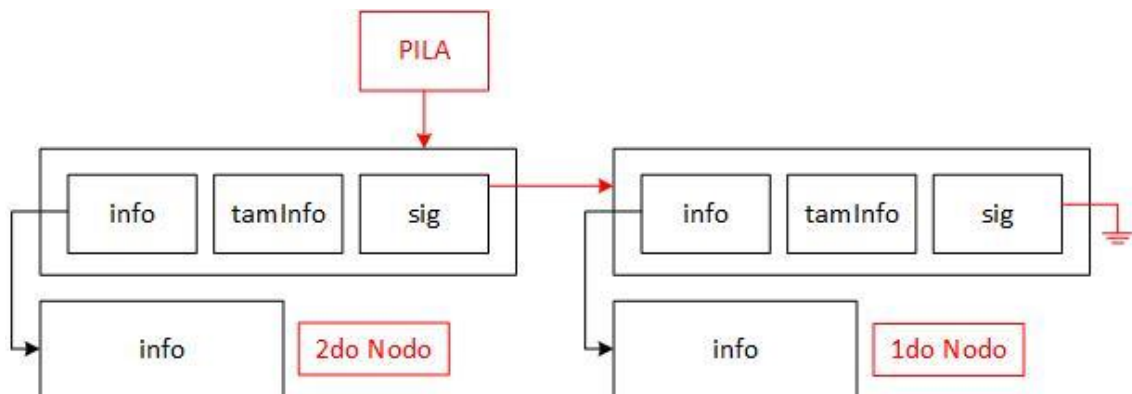
Primero guarda ese nodo en una variable auxiliar para poder manipularlo sin perder su dirección. Luego verifica si la pila está vacía: si no hay nodos, devuelve el error PILA_VACIA.

Si hay nodos, actualiza el tope de la pila apuntando al siguiente nodo (*pila = aux->sig). Después, copia la información del nodo retirado al espacio de memoria recibido como parámetro. Para no copiar de más, usa la macro MIN(X,Y) y copia solo lo que entre: el tamaño guardado o el tamaño que el usuario pidió, el menor de los dos.

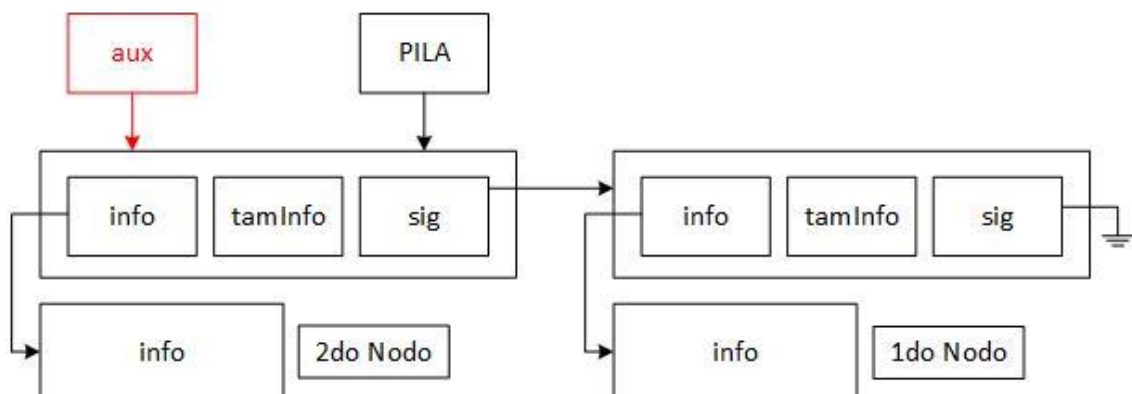
Finalmente libera la memoria reservada tanto para la información (aux->info) como para el nodo en sí (aux).

En la siguiente página vamos a darle una explicación grafica.

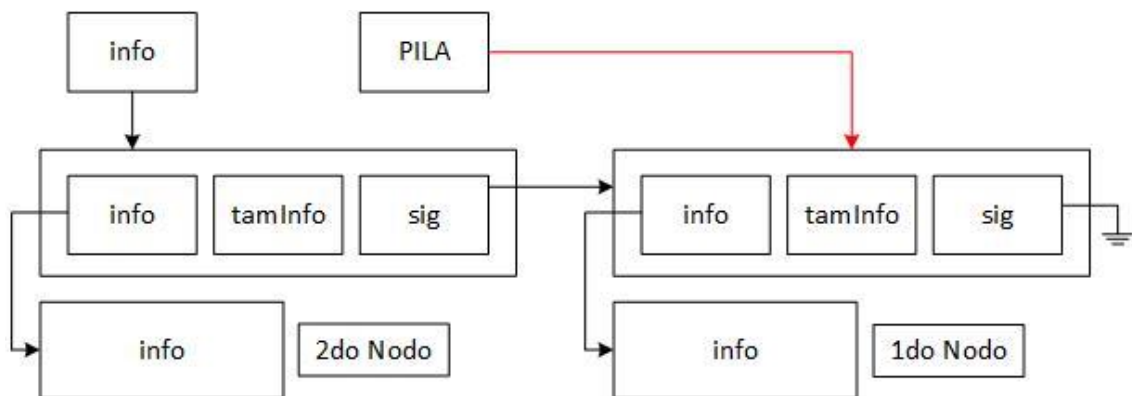
1. Se tiene una pila con estos 2 nodos:



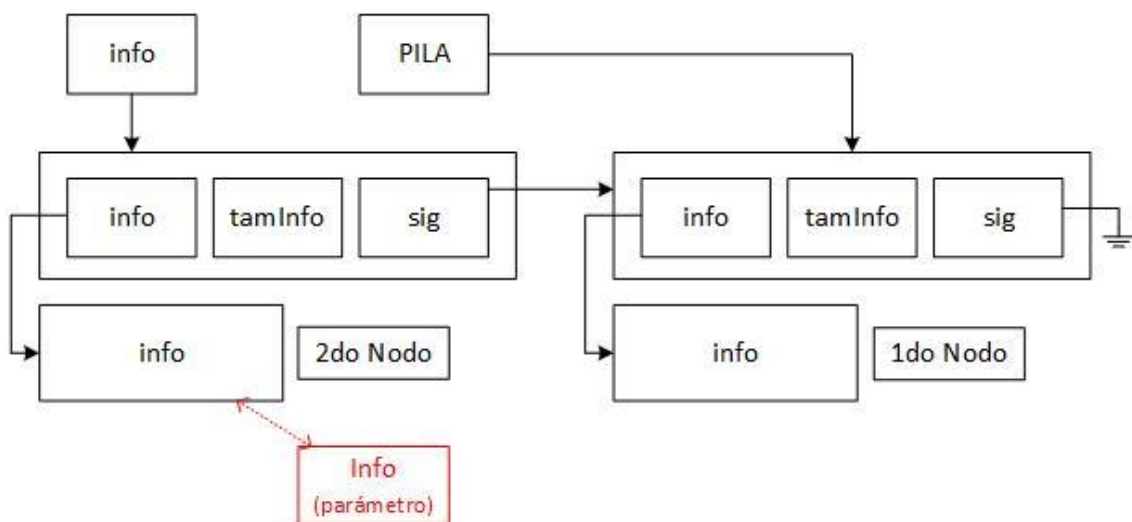
2. Se crea un puntero del tipo *tNodo* auxiliar que apunte al tope de la pila. Esto lo hacemos para no perder la referencia a este nodo para eliminarlo en los pasos siguientes.



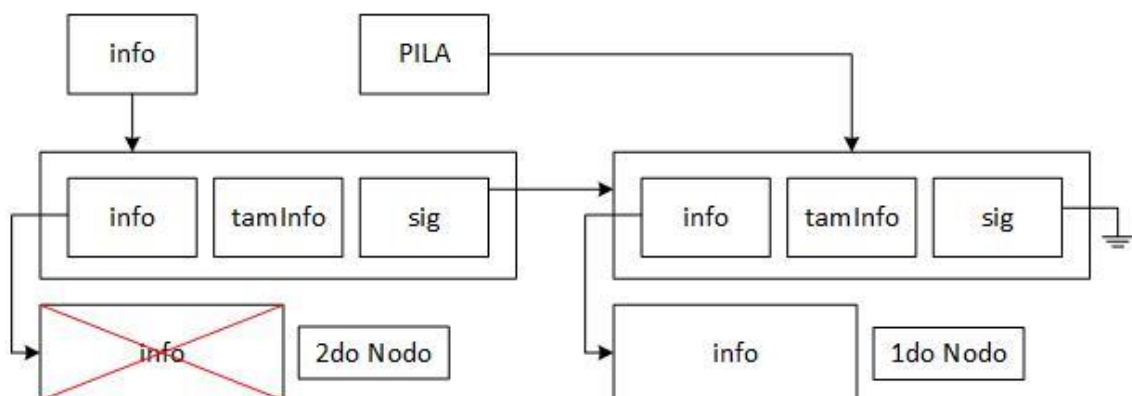
3. La pila comienza apuntar al elemento siguiente



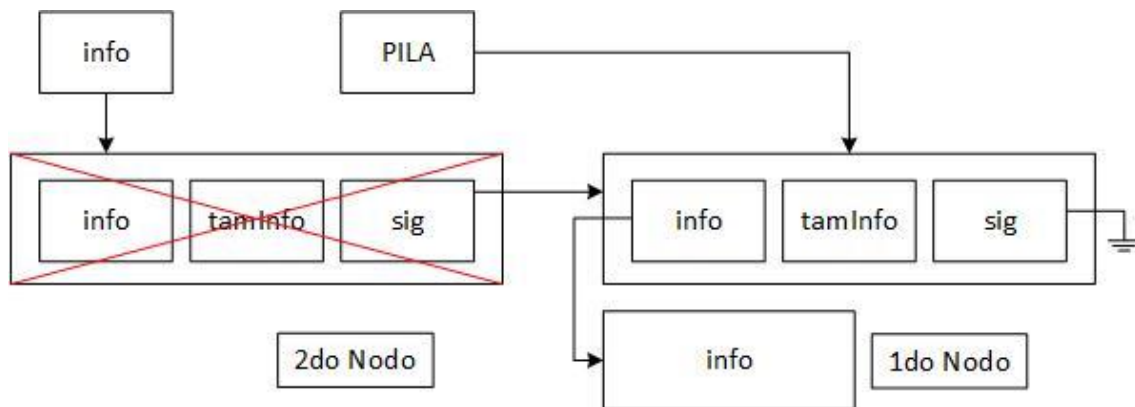
4. Se copian los datos del nodo, a la dirección de memoria entregada como parámetro.



5. Se libera la información del nodo



6. Se libera al nodo en sí.

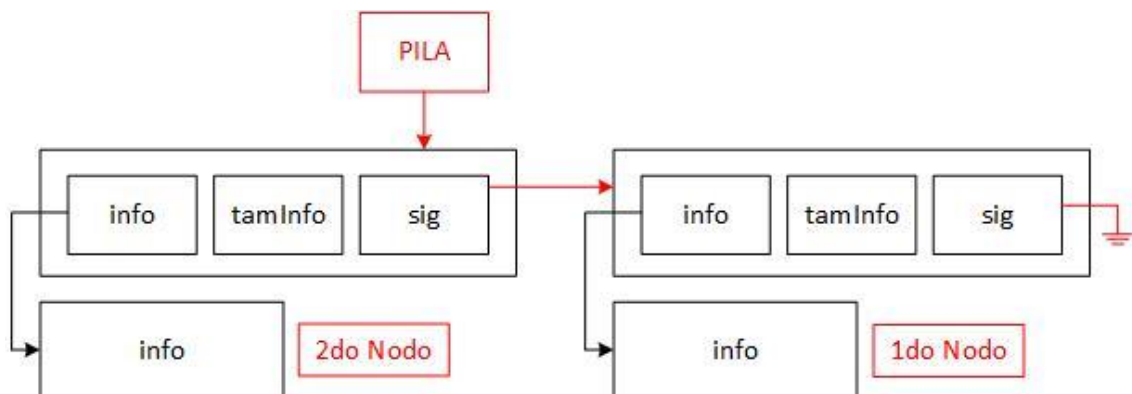


Vaciar Pila

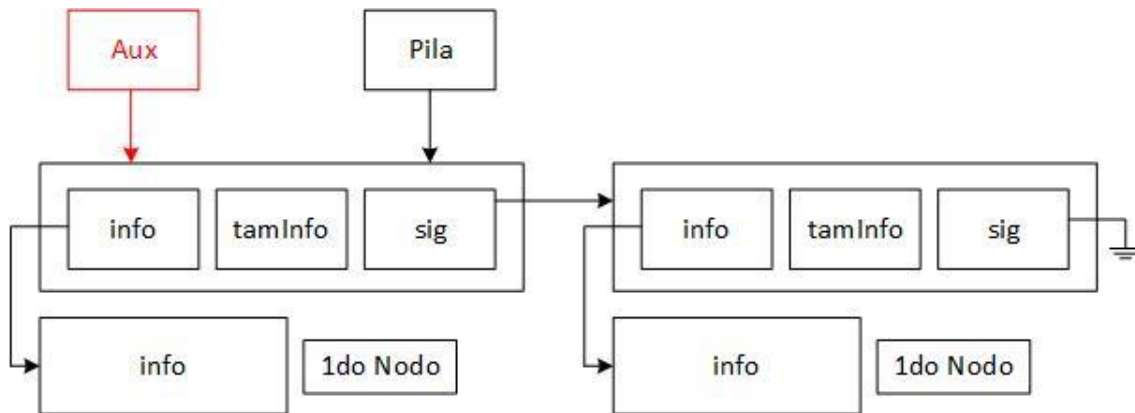
```
void vaciarPila(tPila *pila)
{
    while(*pila) // Mientras haya nodos en la pila (no esté vacía)
    {
        tNodo *aux = *pila; // Se guarda el nodo tope en una auxiliar
        *pila = aux->sig; // Se avanza el tope al siguiente nodo
        free(aux->info); // Se libera la memoria de la información
        free(aux); // Se libera la memoria del nodo
    }
}
```

Va nodo por nodo desde el tope hasta el final (cuando no queda ninguno). Por cada nodo, se libera la información que guarda y luego el nodo en sí. El puntero a pila termina apuntando a NULL, es decir, la pila queda vacía. Se necesita una variable auxiliar para no perder la referencia.

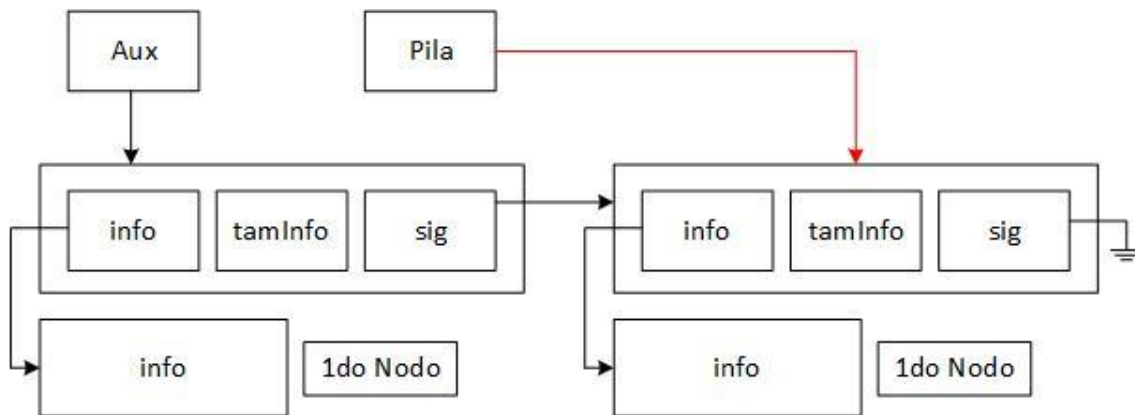
1. Se tiene una pila con estos 2 nodos:



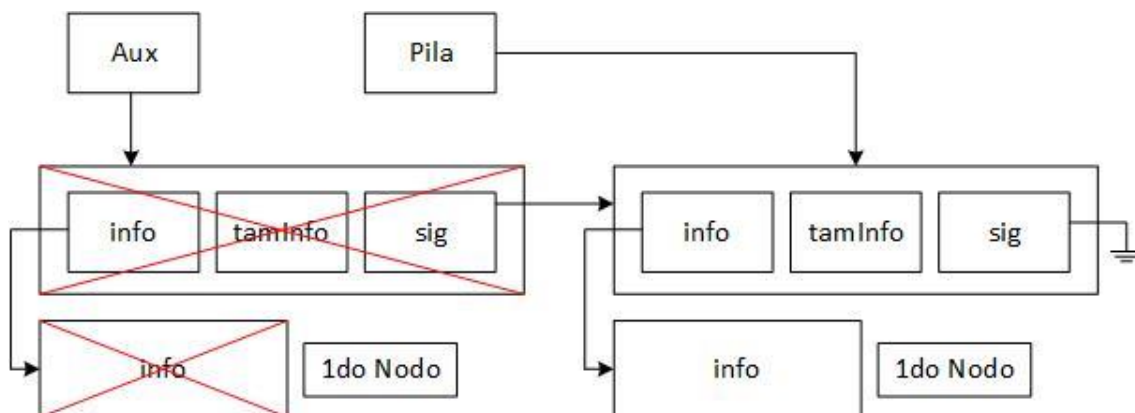
2. Se crea un puntero del tipo *tNodo* auxiliar que apunte al tope de la pila. Esto lo hacemos para no perder la referencia a este nodo para eliminarlo en los pasos siguientes.



3. La pila comienza a apuntar al elemento siguiente



4. Se libera la información del nodo y el nodo en si (siguiente este orden).



5. Se vuelve al paso 1, y se repite en ciclo para cada nodo de la pila.

Utilizando la Pila

A partir de la librería que creamos, vamos a utilizarla mediante las interfaces (funciones) bajo diferentes ámbitos.

Utilización Tradicional

Se quiere crear una pila e insertar hasta CANT_INS. Luego, mostrar por pantalla el contenido de la pila

```
int main()
{
    tPila pila;
    size_t i, aux, tamInfo = sizeof(size_t);

    // Creamos Pila
    crearPila(&pila);

    // Insertamos CANT_INS elementos
    for(i = 0 ; i <= CANT_INS ; i++)
    {
        aux = i * 100; // Realizamos una variante
        ponerEnPila(&pila, &aux, tamInfo);
    }

    // Quitamos cada elemento y lo imprimimos uno por uno (NO SE RECORRE)
    while(sacarDePila(&pila, &aux, tamInfo) == EXITO)
    {
        printf("%u\n",aux);
    }

    // No es necesario ejecutar vaciarPila

    return 0;
}
```

Vector de Pilas

Se quiere crear un **vector** de tamaño TAM_VEC, donde cada posición del vector es una pila. Insertar elementos en forma de escalera y luego imprimirlo.

```
#define TAM_VEC 10
#define CANT_INS 5

int main()
{
    tPila vecPila[TAM_VEC]; // Vector de pilas
    size_t i, j, tamInfo = sizeof(size_t);

    // Creamos una pila por cada posición
    for(i = 0 ; i < TAM_VEC ; i++)
        crearPila(vecPila+i);

    // Insertamos en forma de escalera cada posición
    for(i = 0 ; i < CANT_INS ; i++)
    {
        j = 0;

        while(j <= i)
        {
            ponerEnPila(vecPila+i,&j,tamInfo);
            j++;
        }
    }

    // Quitamos cada elemento y lo imprimimos (NO SE RECORRE)
    for(i = 0 ; i < CANT_INS ; i++)
    {
        printf("%u: ",i);
        while(sacarDePila(vecPila+i,&j,tamInfo) == EXITO)
        {
            printf("%u ", j);
        }
        puts("");
    }

    return 0;
}
```

El resultado de esta operación:

```
0: 0
1: 1 0
2: 2 1 0
3: 3 2 1 0
4: 4 3 2 1 0
```

Pila de Pilas (uno dentro de la otra)

Se quiere crear una pila, donde cada nodo de la pila representa otra pila. La cantidad de nodos dentro de la pila principal, está definido por CANT_INS.

```
#define CANT_INS 5

int main()
{
    tPila pila, aux; // Pila de pilas
    size_t i, j;

    crearPila(&pila);

    for(i = 0 ; i < CANT_INS ; i++)
    {
        crearPila(&aux);

        for(j = 0 ; j <= i ; j++)
        {
            ponerEnPila(&aux,&j,sizeof(j));
        }

        ponerEnPila(&pila,&aux,sizeof(tPila));
    }

    i = CANT_INS - 1;
    while(sacarDePila(&pila,&aux,sizeof(tPila)) == EXITO)
    {
        printf("%u: ",i);
        while(sacarDePila(&aux, &j, sizeof(j)) == EXITO)
        {
            printf("%u ", j);
        }
        i--;
        puts("");
    }

    return 0;
}
```

El resultado de esta operación:

```
4: 4 3 2 1 0
3: 3 2 1 0
2: 2 1 0
1: 1 0
0: 0
```

Como es una pila, se comienza a mostrar desde el elemento que se insertó por última vez. Por eso pos 4.

TDA Cola

Explicación Problema

Un TDA Cola se puede comparar con una fila de personas; la persona que interactúa es la primera que ingresa en la cola, y también las personas ingresan por orden, antes del último ya ingresado. Las ideas centrales corresponden a:



- Solo se puede interactuar con el ingresado más reciente
- Los elementos nuevos ingresan al final
- No se puede recorrer
- No se pueden tocar los procesos internos del TDA en el proyecto
- La información está ordenada en un sentido único

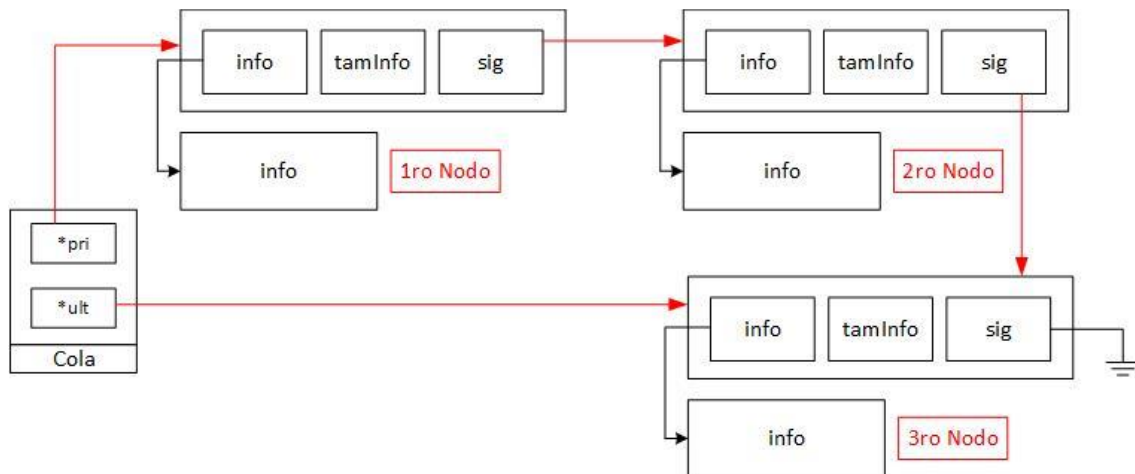
Las operaciones principales que se pueden realizar sobre una cola son:

- Crear Cola -> Inicializa la cola vacía
- Poner en Cola -> Agrega un elemento al final de la fila
- Sacar de Cola -> Saca y obtiene el primer elemento de la fila
- Ver Frente de Cola -> Obtiene el primer elemento de la fila sin sacarlo
- Cola Vacía -> Averigua si la cola está vacía
- Cola Llena -> Averigua si la cola está llena
- Vaciar Cola -> Vacía la cola

Hay 2 formas de implementarlo, con memoria dinámica (uso de nodos) o memoria estática (uso de vectores estáticos).

Cola Dinámica

Estrategia Problema



Representación gráfica del funcionamiento interno de una cola dinámica.

Para implementar un TDA Cola Dinámica, se utiliza una estructura de nodos. Cada vez que se agrega un nuevo elemento, se crea dinámicamente un nuevo nodo mediante malloc, junto con una reserva adicional de memoria para almacenar la información del nodo.

Se emplea una **estructura Cola** que contiene dos punteros:

- **pri* -> Puntero al **primer nodo**, permite acceder al frente de la cola
- **ult* -> Puntero al **último nodo**, facilita el agregado de nuevos nodos

Este diseño es esencial para respetar el principio de funcionamiento de la cola: los nuevos elementos se insertan al final, y los elementos se retiran desde el frente.

Cada nodo contiene:

- Una para su propia estructura (**info*, *tamInfo*, **sig*)
- Para la información que almacena

La memoria de los nodos y de la información se encuentra distribuida en distintas posiciones de la memoria dinámica, no en posiciones consecutivas.

Estructura

```
typedef struct sNodo
{
    void *info;
    size_t tamInfo;
    struct sNodo *sig;
} tNodo;

typedef struct
{
    tNodo *pri; // Puntero al primer nodo
    tNodo *ult; // Puntero al último nodo
} tCola;
```

Como es dinámico, se sigue utilizando el mismo sistema de nodos. Pero, se crea una nueva estructura donde contendrá 2 punteros a nodos, uno con el inicio y otro con el fin. El primero es el que siempre se retira, y el ultimo que es donde se apunta para agregar datos nuevos.

Crear Cola

```
void crearCola(tCola *cola)
{
    cola->pri = NULL;
    cola->ult = NULL;
}
```

Como es dinámico, se sigue utilizando el mismo sistema de nodos. Pero, se crea una nueva estructura donde contendrá 2 punteros a nodos, uno con el inicio y otro con el fin. Se debe indicar apuntando a NULL para decir que está listo para ser usado.

Cola Vacía

```
int colaVacia(tCola *cola)
{
    return *cola->pri ? COLA_DISPONIBLE : COLA_VACIA;
}
```

Si el puntero pri (primero) no es NULL, significa que hay elementos, entonces devuelve COLA_DISPONIBLE. Si pri es NULL, la cola está vacía y devuelve COLA_VACIA.

Cola Llena

Como se utiliza un sistema de nodos, se calcula la disponibilidad de la misma manera que Pila Dinámica.

```
int colaLlena(tCola *cola, size_t tamInfo)
{
    tNodo *nue; // Nodo auxiliar

    // Se intenta reservar memoria para un nuevo nodo
    if(!(nue = malloc(sizeof(tNodo))))
    {
        return COLA_LLENA;
    }

    // Se intenta reservar memoria para la información que contendrá ese nodo
    if(!(nue->info = malloc(sizeof(tamInfo))))
    {
        free(nue);
        return COLA_LLENA;
    }

    // Se libera la memoria, ya que solo se quería verificar disponibilidad
    free(nue);
    free(info);

    // Como ninguna asignación fallo, se notifica disponibilidad
    return COLA_LLENA;
}
```

Esta función *colaLlena* evalúa si hay suficiente memoria disponible para seguir agregando elementos a la pila. Lo hace intentando reservar memoria temporalmente para un nuevo nodo (tNodo) y para el espacio necesario para la información del nuevo dato (tam bytes). Si se les asigna una dirección de memoria a ambos punteros, quiere decir que aún hay espacio disponible.

No obstante, la gracia de crear un TDA Dinámico es crear una interfaz para el usuario. No le interesa si el TDA está lleno o no, solo quiere insertar o quitar elementos. Por lo que solo será una función que estará ahí, pero no será utilizada por el usuario. Aparte que es bastante raro que esté lleno la memoria principal y virtual. Por lo que, también es válido crear la siguiente función para evaluar si está lleno:

```
int colaLlena(tCola*pila, size_t tamInfo)
{
    return COLA_LLENA; // Boca
}
```

Por palabras del profesor Pezzola, esta función es válida, más para ahorrar tiempo.

Poner en Cola

Crea un nuevo nodo, lo llena con la información que le damos, y lo engancha al final de la cola, actualizando las referencias necesarias para que la cola siga funcionando correctamente.

```
int ponerEnCola(tCola *cola, void *info, size_t tamInfo)
{
    tNodo *nodo;

    // Reserva memoria para el nodo
    if(!(nodo = (tNodo*)malloc(sizeof(tNodo))))
    {
        return ERROR_MALLOC;
    }

    // Reserva memoria para la información del nodo
    if(!(nodo->info = malloc(tamInfo)))
    {
        free(nodo);
        return ERROR_MALLOC;
    }

    memcpy(nodo->info, info, tamInfo); // Copia la info entregada al nodo
    nodo->tamInfo = tamInfo; // Guarda el tamaño de la información
    nodo->sig = NULL; // Inicializa el siguiente nodo como NULL

    // Si la cola NO está vacía, conecta el nodo nuevo al final
    if (cola->ult)
        cola->ult->sig = nodo;
    else // Si la cola está vacía, el nuevo nodo será también el primero
        cola->pri = nodo;

    // Actualiza el puntero 'ult' para que apunte al nuevo nodo
    cola->ult = nodo;

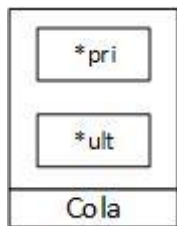
    return EXITO; // Operación exitosa
}
```

Esta función lo que hace es insertar un nuevo elemento en la cola que está implementada como una lista enlazada. Primero reserva memoria para un nuevo nodo y para la información que va a guardar. Si algo falla, libera la memoria y devuelve un error.

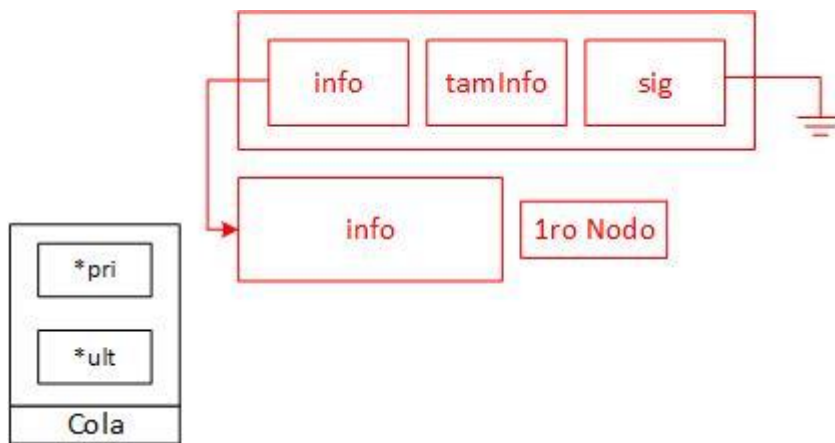
Si todo sale bien, copia el contenido de los parámetros al espacio reservado del nuevo nodo, guarda el tamaño de ese contenido, y se deja el puntero *sig como NULL porque ahora este es el último de la cola. Finalmente, actualiza el puntero de la cola *ult para que apunte al nuevo nodo creado que es el último.

Para entenderlo de una manera más gráfica:

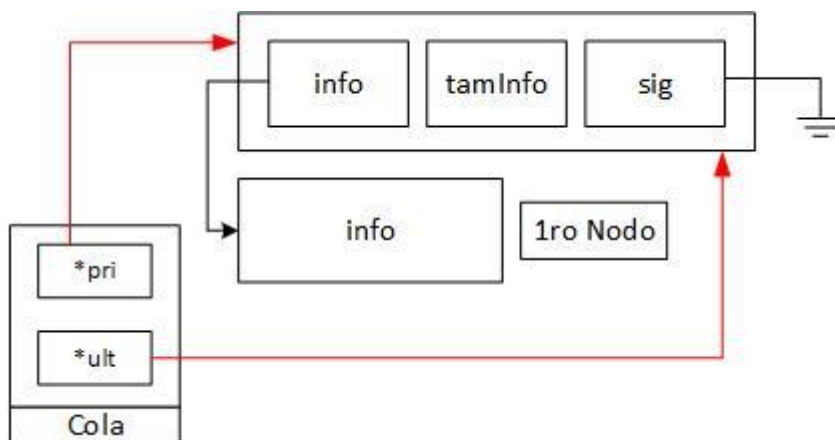
1. Solo existe la estructura cola sin elementos insertados.



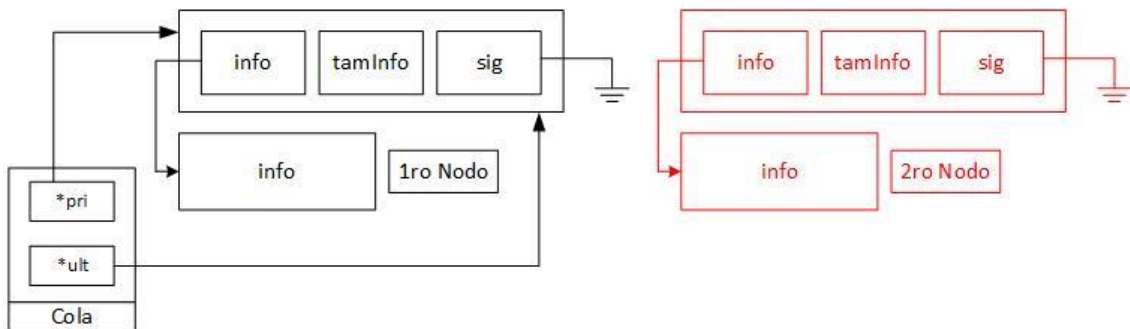
2. Se crea un nuevo nodo y se reserva memoria para el nodo en sí y para la información que se va almacenar. Aun no este asignado a un TDA y tampoco el elemento siguiente.



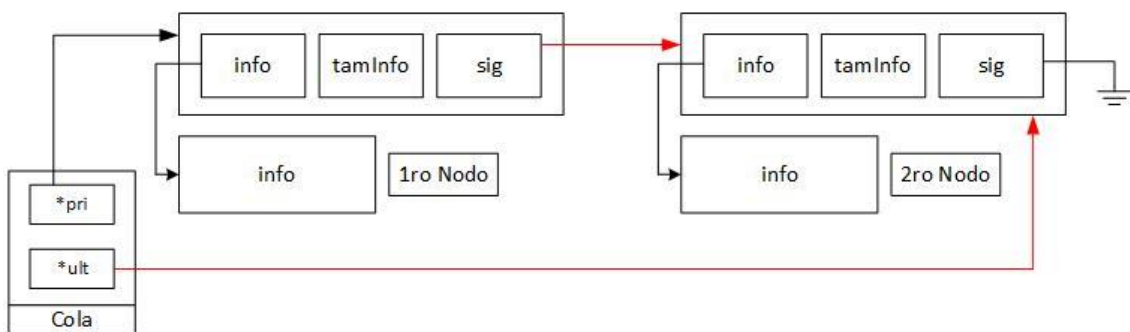
3. Se conecta la cola con el nuevo nodo. Como es el único insertado, es el primero y a su vez el ultimo.



4. Se crea un nuevo nodo y se reserva memoria para el nodo en sí y para la información que se va almacenar. Aun no este asignado a un TDA y tampoco el elemento siguiente.



5. Se lo va a conectar con la cola. Para esto, el ultimo nodo insertado tendrá como siguiente el nuevo. Y ahora el puntero de ultimo se direccionará al nuevo nodo.



Sacar de Cola

Quita el primer elemento de la cola, devuelve su contenido, libera su memoria, y actualiza correctamente los punteros para que la cola siga funcionando bien.

```
#define MIN(X,Y) X > Y ? Y : X // Obtiene el mínimo entre dos valores

int sacarDeCola(tCola *cola, void *info, size_t tamInfo)
{
    tNodo *aux = cola->pri; // Apunta al primer nodo de la cola

    if(!aux) // Si la cola está vacía (no hay primer nodo)
    {
        return COLA_VACIA;
    }

    cola->pri = aux->sig; // Avanza el puntero de la cola al siguiente nodo

    // Copia la información del nodo al espacio recibido
    memcpy(info, aux->info, MIN(aux->tamInfo,tamInfo));

    free(aux->info); // Libera la memoria reservada para la info del nodo
    free(aux);      // Libera la memoria del nodo

    if(!(cola->pri)) // Si después de sacar no queda más nadie en la cola
    {
        cola->ult = NULL; // El último también pasa a ser NULL
    }

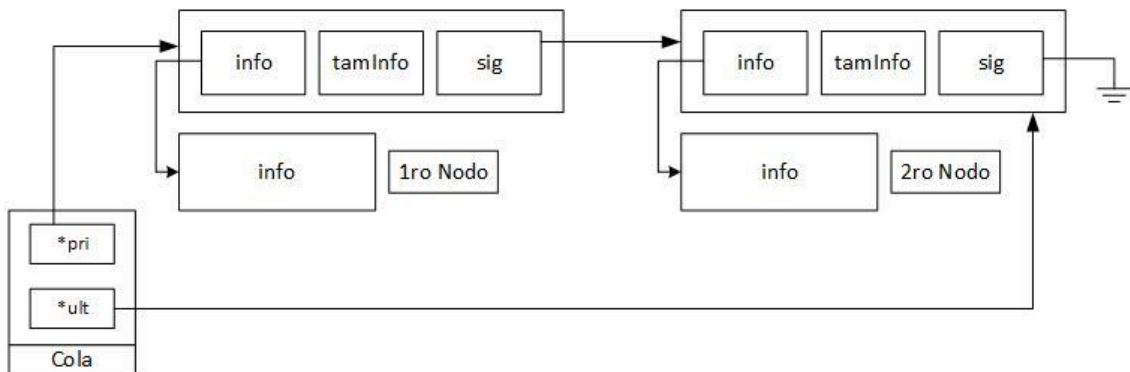
    return EXITO; // Operación exitosa
}
```

Toma el primer nodo y lo guarda en una variable auxiliar. Luego, verifica si la cola no está vacía:

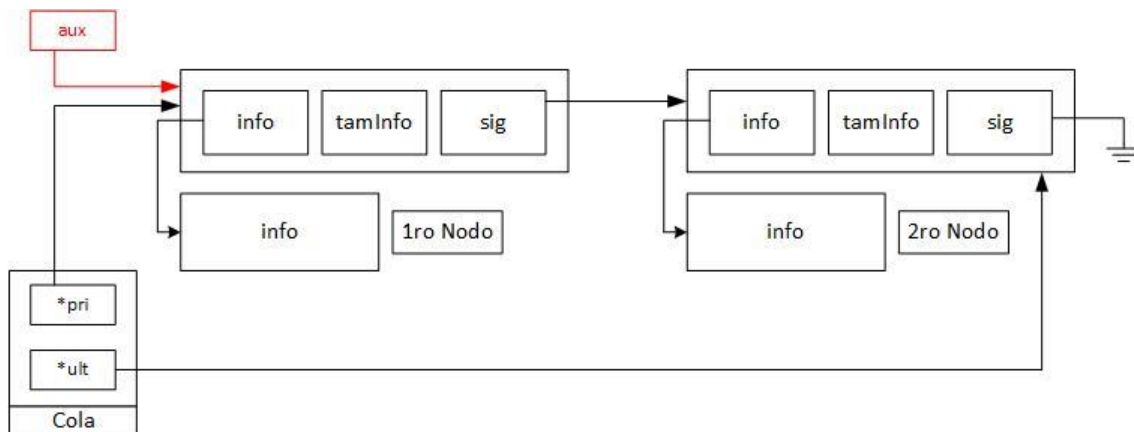
1. Se guarda el primer nodo de la cola a la variable auxiliar
2. Se verifica si la cola está vacía para finalizar
3. Se avanza el inicio de la cola; se mueve el primer puntero (cola->pri) al siguiente nodo (aux->sig). De esta manera, el primer elemento queda "desconectado" de la cola.
4. Se copia la información del nodo desconectado a las variables dadas como parámetros
5. Se libera la información del nodo desconectado y el nodo en sí.
6. Se comprueba si la cola quedó vacía con cola->pri. Si el primero quedo vacío, entonces el puntero al último también debe estarlo.

< **MIN(aux->tamInfo, tamInfo)** > Sirve para asegurarse de copiar únicamente la cantidad adecuada de memoria; Si el tamaño del dato que está en la cola es menor al tamaño que el usuario quiere recibir, se copia el tamaño más chico. Así no se súbrole ni se sobrescribe memoria que no corresponde.

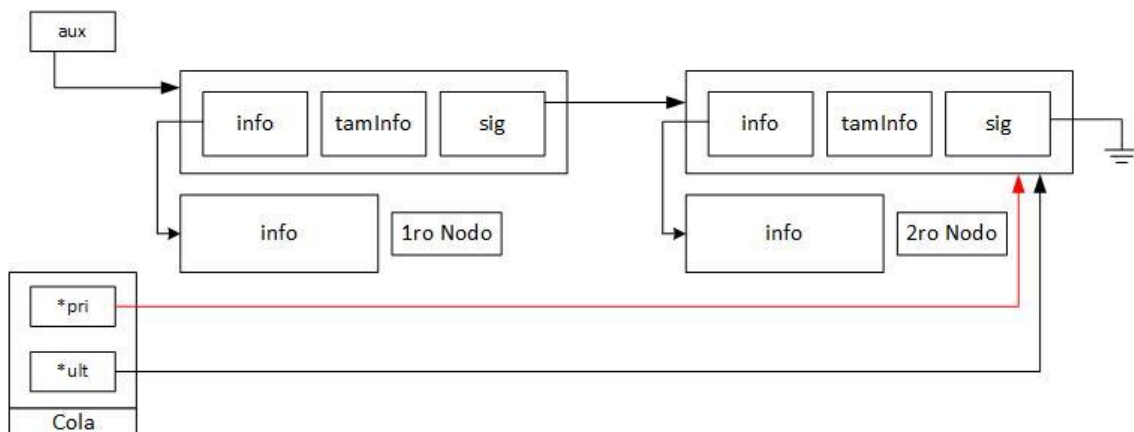
1. Se presenta la siguiente cola con 2 nodos conectados



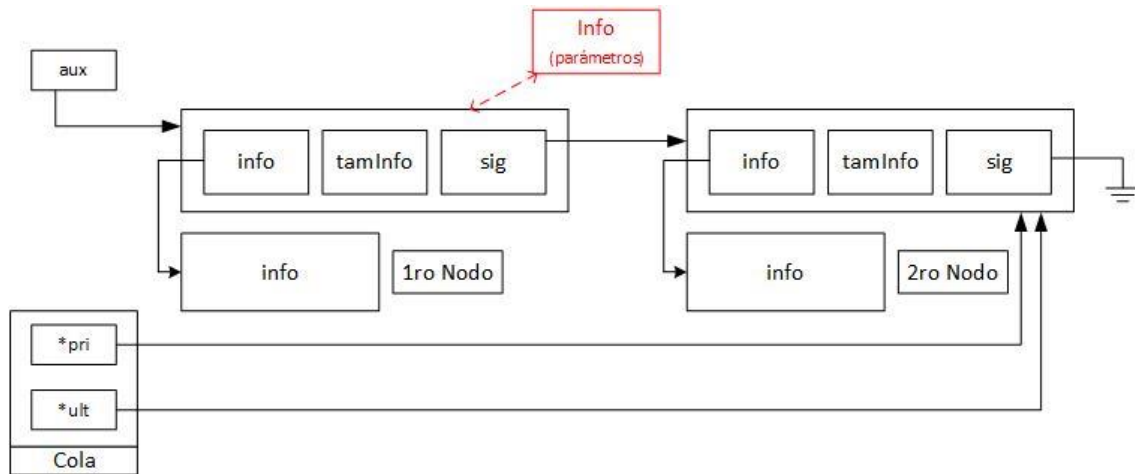
2. Se crea un nodo auxiliar que apunte al primer nodo de la cola. Para no perder la referencia de este al mover el puntero en el siguiente paso.



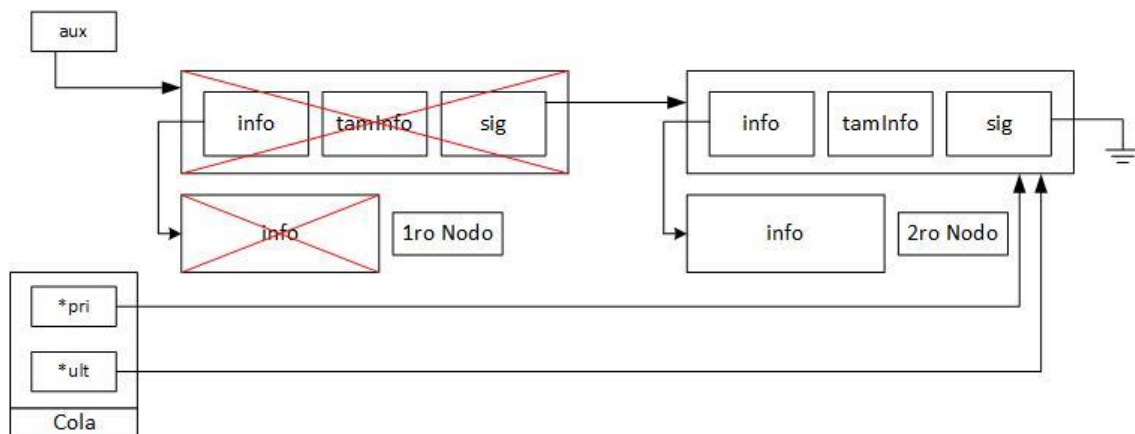
3. El puntero de la cola **pri*, comenzara apuntar al siguiente del primero.



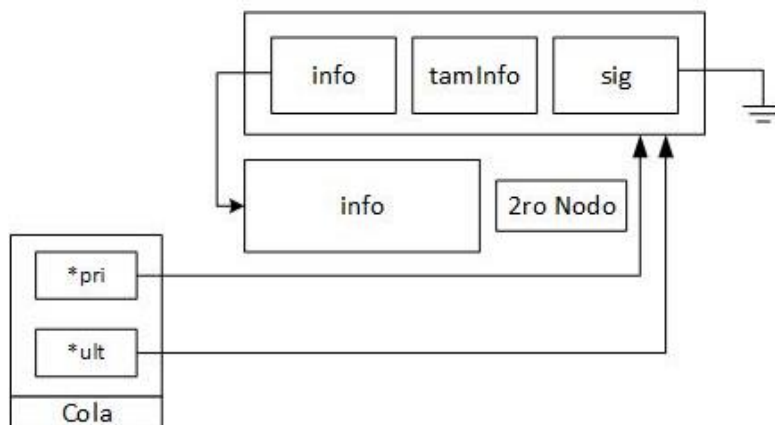
4. Se copia la información del nodo direccionado por el puntero auxiliar, a las direcciones entregadas por parámetro de la función.



5. Se elimina la información del nodo y el nodo en sí que apunta la variable auxiliar.



6. Quedando como resultado, la siguiente cola.



Ver Frente de la Cola

Permite consultar la información del primer elemento de una cola dinámica sin eliminarlo.

```
#define MIN(X,Y) X > Y ? Y : X // Obtiene el mínimo entre dos valores

int verFrenteCola(tCola *cola, void *info, size_t tamInfo)
{
    tNodo *nodo = cola->pri; // Se guarda un puntero al primer nodo de la cola

    if(!nodo) // Si el primer nodo no existe, la cola está vacía.
    {
        return COLA_VACIA;
    }

    // Copia la información del primer nodo en el espacio de los parametros
    memcpy(info, nodo->info, MIN(nodo->tamInfo, tamInfo));

    return EXITO; // Devuelve que la operación fue exitosa.
}
```

Primero, se guarda un puntero al primer nodo de la cola (`nodo = cola->pri`). Si este puntero resulta ser NULL, significa que la cola está vacía, por lo que la función finaliza.

Si la cola tiene elementos, entonces se realiza una copia de la información almacenada del primer nodo (`nodo->info`) hacia la dirección de memoria recibida por parámetro (`info`).

< `MIN(aux->tamInfo, tamInfo)` > Sirve para asegurarse de copiar únicamente la cantidad adecuada de memoria; Si el tamaño del dato que está en la cola es menor al tamaño que el usuario quiere recibir, se copia el tamaño más chico. Así no se súbrole ni se sobrescribe memoria que no corresponde.

Vaciar Cola

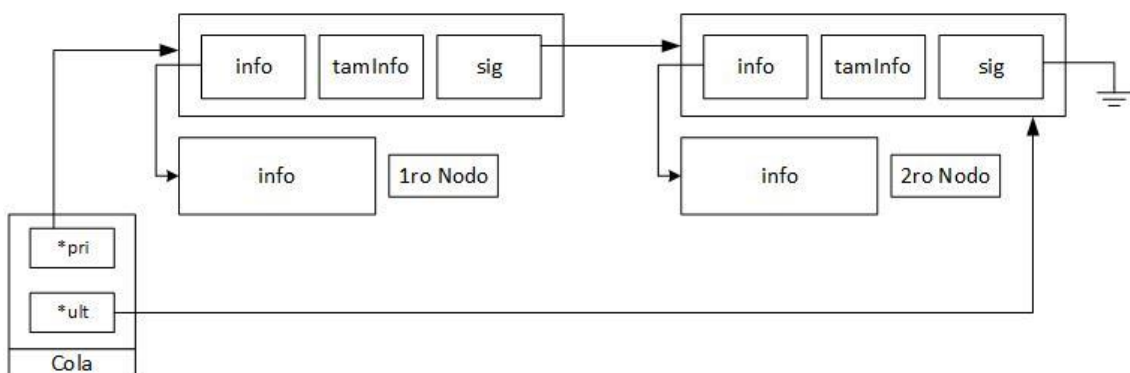
Sigue un algoritmo muy similar al ya visto en TDA Pila Dinámico. Elimina nodo por nodo, liberando primero la info y después el nodo, hasta dejar la cola totalmente vacía.

```
void vaciarCola(tCola *cola)
{
    // Mientras haya nodos en la cola (mientras el primero no sea NULL)
    while(cola->pri)
    {
        tNodo *aux = cola->pri; // Guardamos el nodo actual en un auxiliar
        cola->pri = aux->sig;    // Avanzamos el puntero de primero al siguiente
        free(aux->info);         // Liberamos la memoria para la información
        free(aux);               // Liberamos la memoria del nodo en sí
    }
    // Una vez vacía la cola, también ponemos el último a NULL
    cola->ult = NULL;
}
```

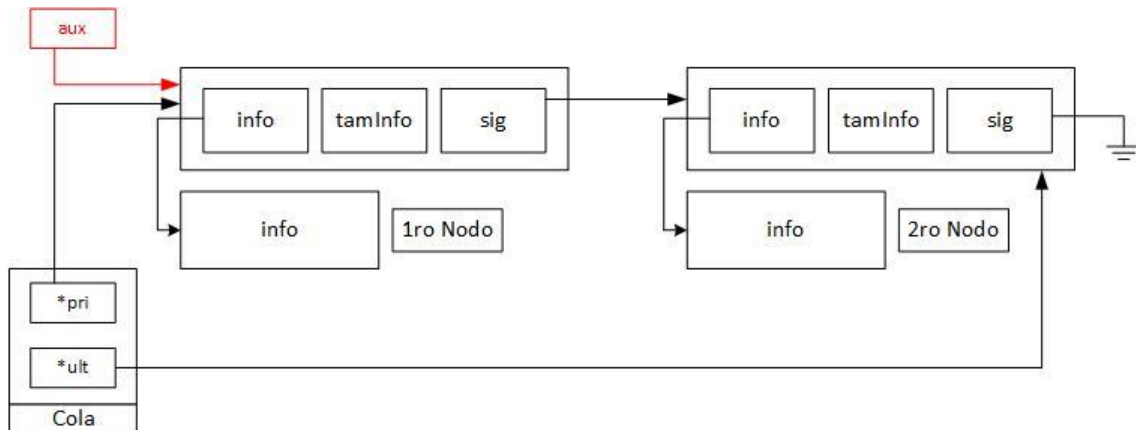
Va nodo por nodo desde el primero hasta el último (cuando no queda ninguno). Se crea una variable auxiliar con el primer elemento para no perder la referencia. Por cada auxiliar se libera la información que guarda y luego el nodo en sí. El puntero a cola termina apuntando a NULL, es decir, la cola queda vacía.

Para una mayor comprensión, se lo representara con los siguientes gráficos:

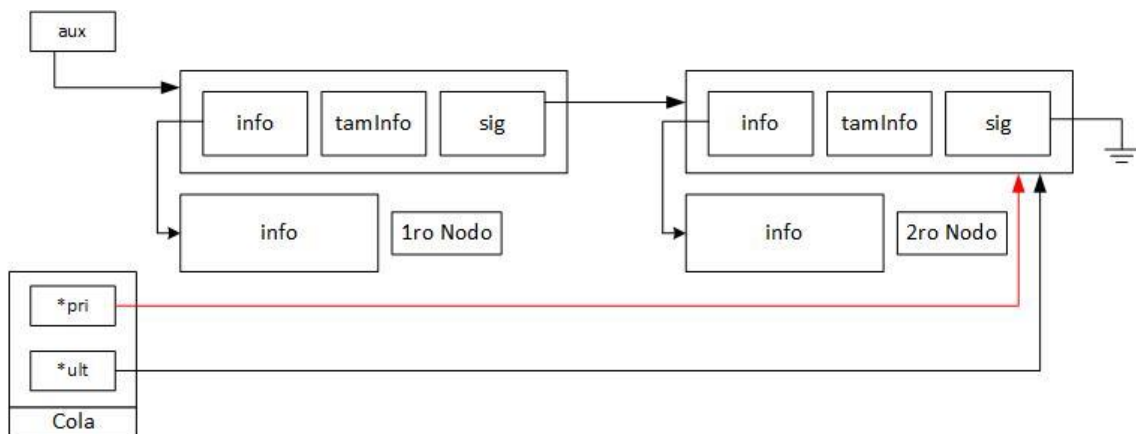
1. Se presenta la siguiente cola con 2 nodos conectados



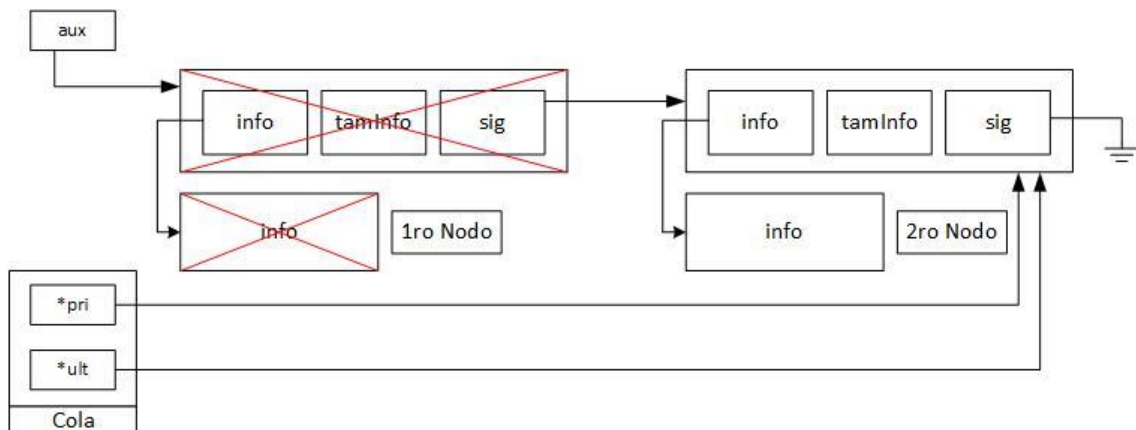
2. Se crea un nodo auxiliar que apunte al primer nodo de la cola. Para no perder la referencia de este al mover el puntero en el siguiente paso.



3. El puntero de la cola *pri, comenzara apuntar al siguiente del primero.



4. Se elimina la información del nodo y el nodo en sí que apunta la variable auxiliar.



5. Este proceso se repite para cada nodo de la pila.

Cola Estática

La cola estática es un concepto demasiado complejo que se realiza con un vector circular. Es un tema que no es evaluado hace año en parciales debido a su complejidad y falta de necesidad. Por lo que no lo enseñare y tan solo adjuntare una clase grabada:

<https://youtu.be/dQw4w9WgXcQ?si=HbuU7nwmWnF235Mf>

TDA Lista

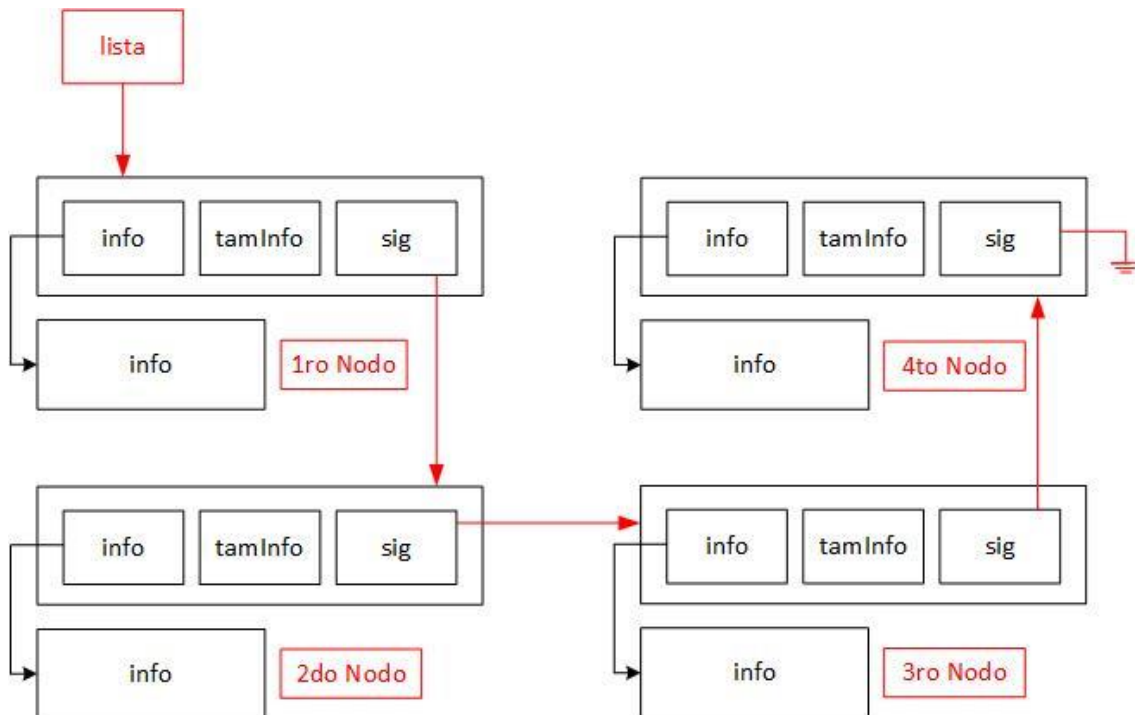
Explicación Problema

El TDA Lista permite modelar colecciones de datos donde los elementos pueden ser insertados, eliminados o consultados en cualquier posición (principio, fin o personalizado), e incluso recorrido. A diferencia de una pila o cola, donde las operaciones están restringidas al tope o al frente, una lista ofrece mayor flexibilidad y libertad en la manipulación de la información.

Puede entenderse también como una combinación de TDA Pila y Cola, donde fusionamos ambos conceptos y funciones.

Estrategia para el Problema

Hay 2 maneras de resolver esta idea, puede ser con estática o dinámica. El modelado de estático no será visto, puesto que ya fue estudiado en la materia "Tópicos de Programación", es el TDA primer visto.



Representación gráfica del funcionamiento interno de una lista dinámica.

La estrategia para implementar una lista dinámica se basa en la construcción de nodos enlazados mediante punteros. Cada nodo contiene un puntero a la información, el tamaño de esa información, y un puntero al siguiente nodo. Esta estructura permite insertar o eliminar elementos en cualquier posición sin necesidad de mover físicamente los datos, como ocurriría en un vector.

Las operaciones se resuelven desplazando un puntero a puntero por los campos *sig* de cada nodo hasta alcanzar la posición deseada. Esta forma de navegación hace posible implementar primitivas como insertar al inicio, al final, en una posición específica, eliminar un nodo, recorrer toda la lista, o insertarla en orden con una función de comparación.

Algunas funciones son un copy past de los anteriores TDA como:

- [¿Está lleno?](#)
- [¿Está vacío?](#)

Estructura

Sigue una estructura idéntica a *Pila*, lo único que cambia es *tPila* por *tLista*.

```
typedef struct sNodo
{
    void *info; // Puntero a la información del nodo (guardada con malloc)
    size_t tamInfo; // Tamaño en bytes de la información
    struct sNodo *sig; // Puntero al siguiente nodo
} tNodo;

typedef tNodo* tLista;
```

El sNodo es como un alias que se pone (sNodo = tNodo), es necesario porque después de ser creado, recién ahí se lo llama como tNodo, entonces esto es necesario para poder auto-referenciarse con el campo *sig. El typedef es otro sinónimo que se le pone para tLista.

Crear Lista

Mismo método que el resto, comienza a apuntar a null.

```
void crearLista(tLista *lista)
{
    *lista = NULL;
}
```

Cuando se crea un puntero, este apuntara a basura si no le asignamos nada. Por lo que, debemos hacer que apunte a la nada (NULL) indicando que está vacío o listo para usarse.

Operaciones Al Principio

Las operaciones al principio siguen prácticamente el mismo mecanismo ya visto en TDA Pila. No cambia nada más que los nombres. Por lo que no se dará una explicación grafica al ser un tema "ya visto".

Insertar Nodo al Principio

```
int insPrinLista(tLista *lista, void *info, size_t tamInfo)
{
    tNodo *nodo;

    // Reservamos memoria para el nodo
    if(!(nodo = (tNodo*)malloc(sizeof(tNodo)))) // Si falla el malloc
    {
        return LISTA_LLENA;
    }

    // Reservamos memoria para la información
    if(!(nodo->info = malloc(tamInfo))) // Si falla el malloc
    {
        free(nodo); // Liberamos el nodo
        return LISTA_LLENA;
    }

    memcpy(nodo->info, info, tamInfo); // Copiamos la información al nuevo nodo
    nodo->tamInfo = tamInfo; // Guardamos el tamaño de la información
    nodo->sig = *lista; // El nuevo nodo apunta al que antes era el primero

    *lista = nodo; // Ahora el nuevo nodo pasa a ser el primero

    return EXITO; // Todo salió bien
}
```

Esta función lo que hace es insertar un nuevo elemento en una lista. Primero reserva memoria para un nuevo nodo y para la información que va a guardar. Si algo falla, libera la memoria y devuelve un error.

Si todo sale bien, copia el contenido de los parámetros al espacio reservado del nuevo nodo, guarda el tamaño de ese contenido, y enlaza ese nuevo nodo al nodo que estaba antes en la cima de la lista. Finalmente, actualiza el puntero de la lista para que apunte al nuevo nodo, que ahora es el nuevo primer nodo.

Ver Nodo del Principio

```
#define MIN(X,Y) (X) > (Y) ? (Y) : (X) // Obtiene el mínimo entre dos valores

int verPrinLista(tLista *lista, void *info, size_t tamInfo)
{
    if (!*lista) // Verifica si la lista está vacía (no hay nodos)
        return PILA_VACIA;

    // Copia la información del nodo del principio hacia el parámetro 'info'
    memcpy(info, (*lista)->info, MIN(tamInfo, (*lista)->tamInfo));

    return EXITO;
}
```

Copiamos el contenido del principio a la dirección parámetro 'info'.

Sacar Nodo del Principio

```
int outPrinLista(tLista *lista, void *info, size_t tamInfo)
{
    tNodo *aux = *lista; // Se guarda el nodo de la lista en un aux

    if(!aux) // Si la lista está vacía, se retorna el código de lista vacía
        return LISTA_VACIA;

    *lista = aux->sig; // Se actualiza el tope de la lista al siguiente nodo

    // Se copian los datos del nodo a eliminar a los parámetros.
    // Se copia el mínimo entre el tamaño guardado y el de los parámetros.
    memcpy(info, aux->info, MIN(tamInfo,aux->tamInfo));

    free(aux->info); // Se libera la memoria del dato del nodo
    free(aux); // Se libera la memoria del nodo en sí

    return EXITO;
}
```

Primero guarda ese nodo en una variable auxiliar para poder manipularlo sin perder su dirección. Luego verifica si la lista está vacía: si no hay nodos, devuelve el error LISTA_VACIA.

Si hay nodos, actualiza el principio de la lista apuntando al siguiente nodo (*lista = aux->sig). Después, copia la información del nodo retirado al espacio de memoria recibido como parámetro. Para no copiar de más, usa la macro MIN(X,Y) y copia solo lo que entre: el tamaño guardado o el tamaño que el usuario pidió, el menor de los dos.

Finalmente libera la memoria reservada tanto para la información (aux->info) como para el nodo en sí (aux).

Desplazamiento en TDA Lista

Las operaciones al final o en posiciones particulares, siguen casi lo mismo que las operaciones al principio anteriormente vistas. La diferencia, es que nos vamos a dirigir hasta la posición que deseemos, por lo que debemos agregar 2 líneas más para movernos, el resto del código, es más de lo mismo, por lo que re utilizamos llamando a las operaciones al principio. Lo único que cambia en cada operación (insertar, ver, sacar), es el agregado de alguno de los siguientes bucles:

```
while(*lista)
    lista = &(*lista)->sig;
```

```
while((*lista)->sig)
    lista = &(*lista)->sig;
```

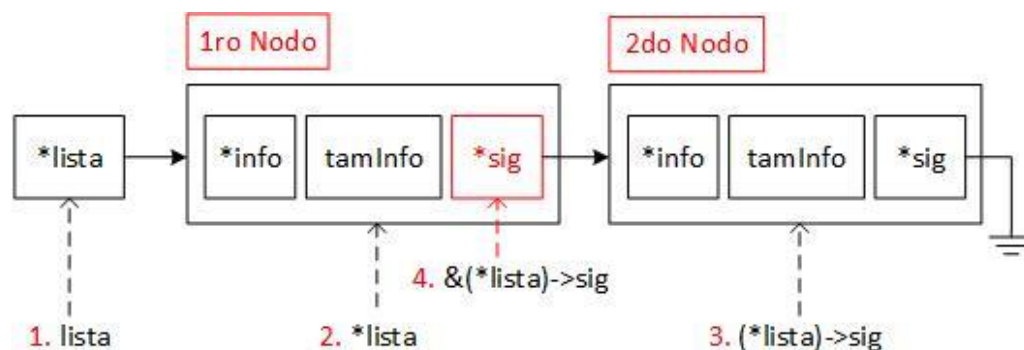
```
while (*lista && pos)
{
    lista = &(*lista)->sig;
    pos--;
}
```

Ante esta situación podemos tomar 2 decisiones:

- Entender a fondo qué hace este código.
- Memorizar la línea de código.

Vamos a explicar los bucles y la operación. No obstante, los ejemplos gráficos paso a paso los daremos a partir del siguiente capítulo "operaciones al Final".

- **Explicación < &(*lista)->sig; >**



La expresión < &(*lista)->sig > o < &((*lista)->sig) >, obtiene la dirección de memoria del campo sig del nodo al que apunta *lista. Vamos paso a paso:

1. **lista** puntero que contiene la dirección del nodo donde apunta.
2. ***lista** accedemos al nodo donde apunta.
3. **(*lista)->sig** es el valor del campo puntero sig de ese nodo, es decir, a dónde apunta el nodo.
4. **&(*lista)->sig** es la dirección de ese campo sig, que vive dentro del nodo actual.
5. **lista = &(*lista)->sig** copia la dirección de memoria contenida en el puntero sig, para comenzar apuntar al mismo lugar

- **Explicación Bucle 1: Posicionarnos en el campo *sig* del último nodo**

```
while(*lista)
    lista = &(*lista)->sig;
```

Recorre la lista enlazada por medio de los campos *sig* hasta llegar al último nodo, que se indica estando el campo *sig* como vacío. En este punto, al estar el campo vacío, es la posición ideal para crear un nuevo nodo (por ejemplo) y conectar el campo al que nos desplazamos con el nuevo nodo. Se puede hacer algo tan simple como: **lista = nuevoNodo;*

Por lo que, lista deja de ser un puntero que apunta al primer nodo, para pasar a ser un puntero posicionado en la dirección del campo *sig* del último nodo. Que, como es el último, tiene contenido nulo.

- **Explicación Bucle 2: Posicionarnos en el campo *sig* del ante ultimo nodo**

```
while((*lista)->sig)
    lista = &(*lista)->sig;
```

Recorre la lista enlazada por medio de los campos *sig* hasta llegar al ante ultimo nodo, que se indica estando nulo el campo *sig* del siguiente nodo. En este punto, nos encontramos en una dirección que apunta al último nodo, por lo que es la situación ideal como para eliminar el ultimo nodo (por ejemplo).

Por lo que, lista deja de ser un puntero que apunta al primer nodo, para pasar a ser un puntero posicionado en la dirección del campo *sig* del ante ultimo nodo.

- **Explicación Bucle 3: Posicionarnos en un el campo *sig* de un nodo específico**

```
while (*lista && pos)
{
    lista = &(*lista)->sig;
    pos--;
}
```

Recorrer la lista enlazada por medio de los campos *sig* hasta llegar a un nodo específico dado por *pos* a partir de 0. Por ejemplo, si la posición querida es 2, nos encontraremos en el campo *sig* del nodo que apunta al nodo 2.

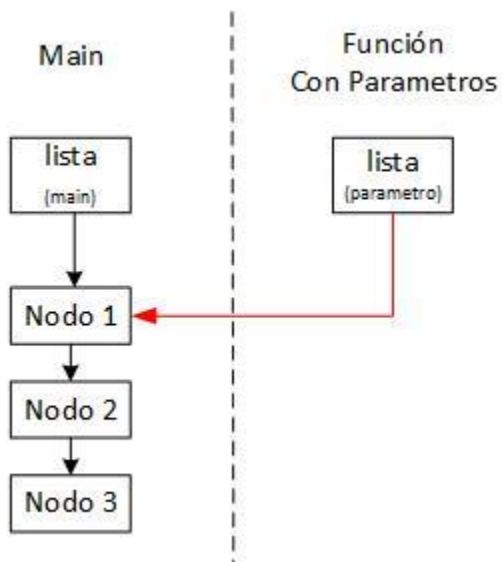
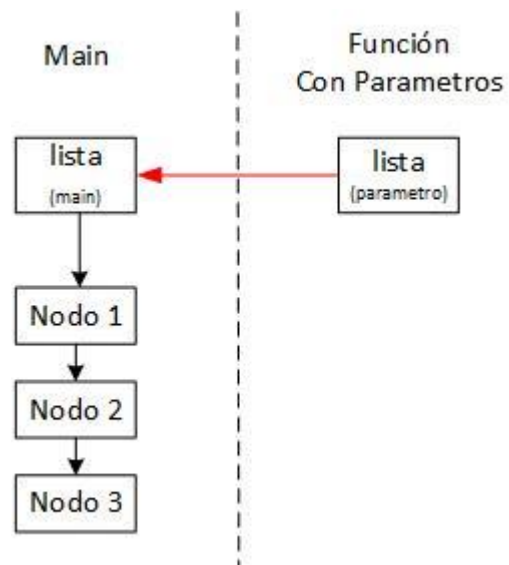
Nota:

Estamos re utilizando el parámetro *lista*, para no crear una nueva variable auxiliar. Puede hacerse con una sola variable (y así lo quieren los docentes).

Tambien, al hacer esto no estamos tocando el apuntador de la variable *lista* creado en el main o bajo otro contexto fuera del código interno. En cambio, estaríamos tocando el apuntador de los campos *sig* en que nos desplazamos.

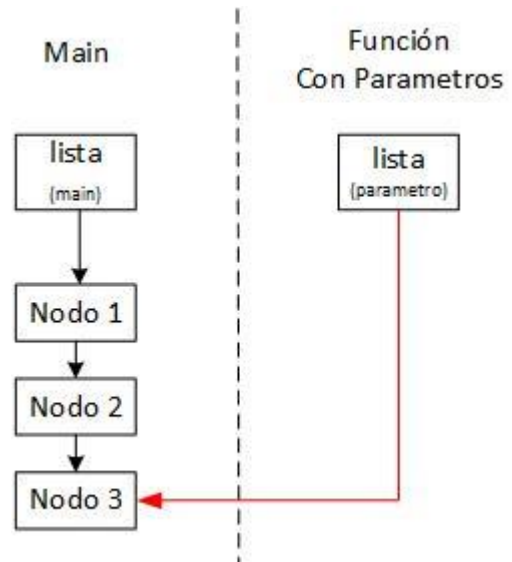
Vamos a darle una explicación gráfica:

1. Se tiene una lista cargada en el main, en que contiene a los siguientes nodos. Al pasar la dirección de la lista a la función, se presenta el siguiente estado inicial. El parámetro "lista" de la función, apunta hacia el contenido de la variable "lista" del main.



2. Modificamos el contenido del parámetro "lista" para que comience apuntar hacia el siguiente nodo.

3. Y así seguimos recorriendo sucesivamente hasta llegar al último nodo, re utilizando el parámetro "lista".



Conclusión:

Estamos re utilizando el parámetro *lista*, si modificamos su contenido (*lista = nuevaDir*), modificamos donde apuntamos. Si accedemos al contenido donde apunta (**lista = nuevaDir*) utilizando el asterisco "*", modificamos el apuntador que se encuentra en el main.

No obstante, en los diagramas gráficos, vamos a representar de forma unificada ambas variables *lista* para mayor simplicidad. Total, ya se entiende el proceso por detrás de todo esto.

Operaciones al Final

Como ya se nombró anteriormente, las operaciones al final son lo mismo que las operaciones al principio, con la diferencia que debemos desplazarnos al final de la lista para realizarlo.

Insertar Nodo al Final

```
int insFinLista(tLista *lista, void *info, size_t tamInfo)
{
    // Avanzamos hasta posicionarnos en el campo sig del último nodo
    while(*lista)
        lista = &(*lista)->sig;
    // Ya estamos en el campo `sig` del último nodo (que es NULL)

    // Como el resto del código es igual, re utilizamos la función
    return insPrinLista(lista, info, tamInfo);
}
```

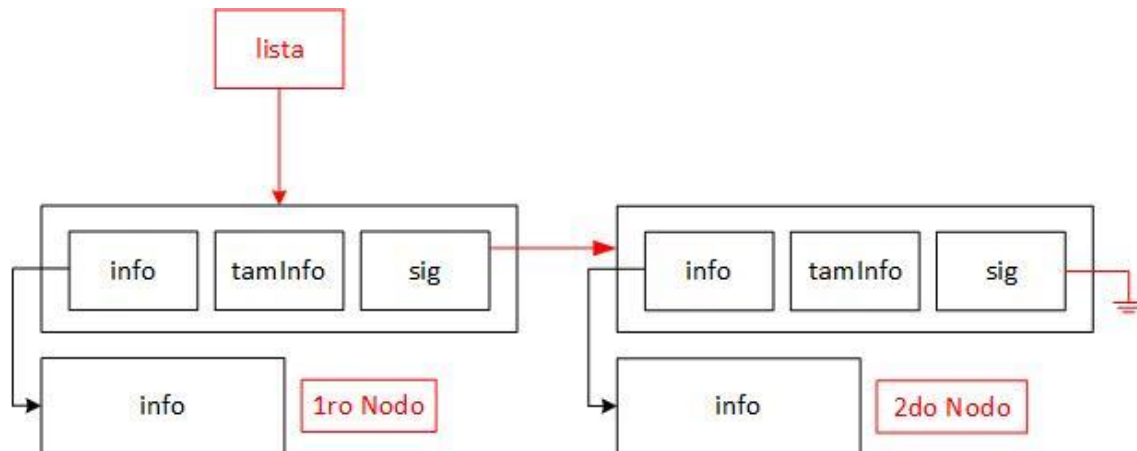
Esta función lo que hace es insertar un nuevo elemento al final que está implementada como una lista enlazada. Primero, nos deslizamos hasta el campo *sig* del último nodo de la lista, ahora la variable *lista* se encuentra en ese campo.

Como el resto del código es idéntico (crear nodo y conectarlo), re utilizamos la función de insertar primero lista, siendo que ahora pasamos el puntero lista para reconectar el nodo anterior con el nuevo.

Si todo sale bien, copia el contenido de los parámetros al espacio reservado del nuevo nodo, guarda el tamaño de ese contenido, y se deja el puntero **sig* como NULL porque ahora este es el último de la lista.

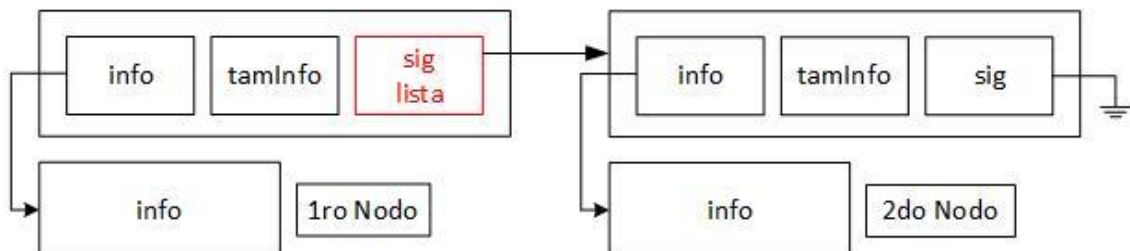
Vamos a darle una representación gráfica:

1. Se presenta la siguiente lista con 2 elementos

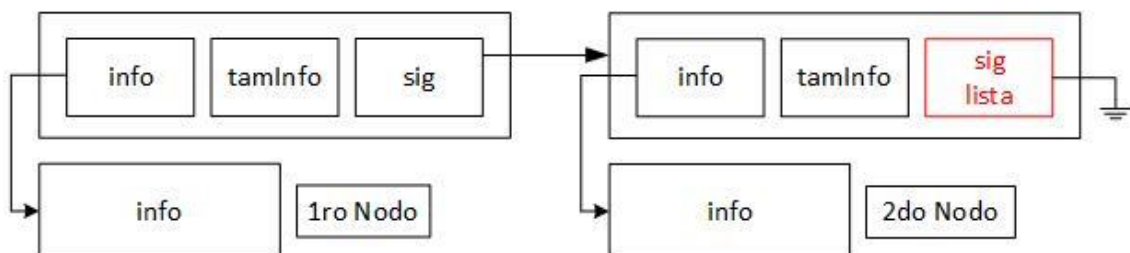


2. Vamos a comenzar a desplazarnos en forma de bucle hasta llegar al último nodo. En este tipo de bucle, finalizara cuando la variable lista se encuentre en NULL, indicando que no hay ningún nodo siguiente.

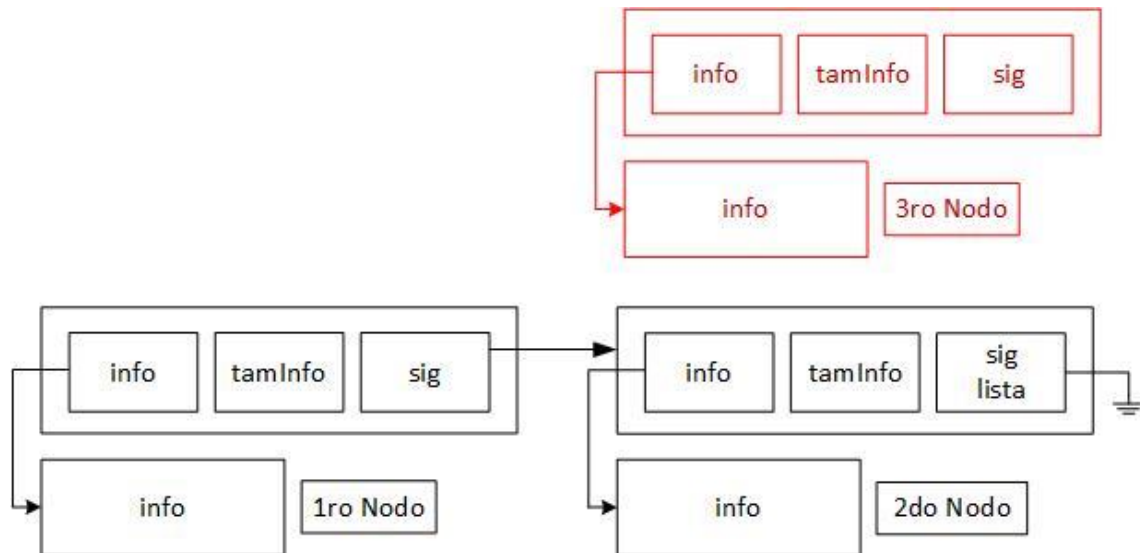
2.1. Se comienza posicionando el puntero *lista*, en el mismo lugar que el campo *sig* del nodo 1. Como su contenido no es nulo, se sigue.



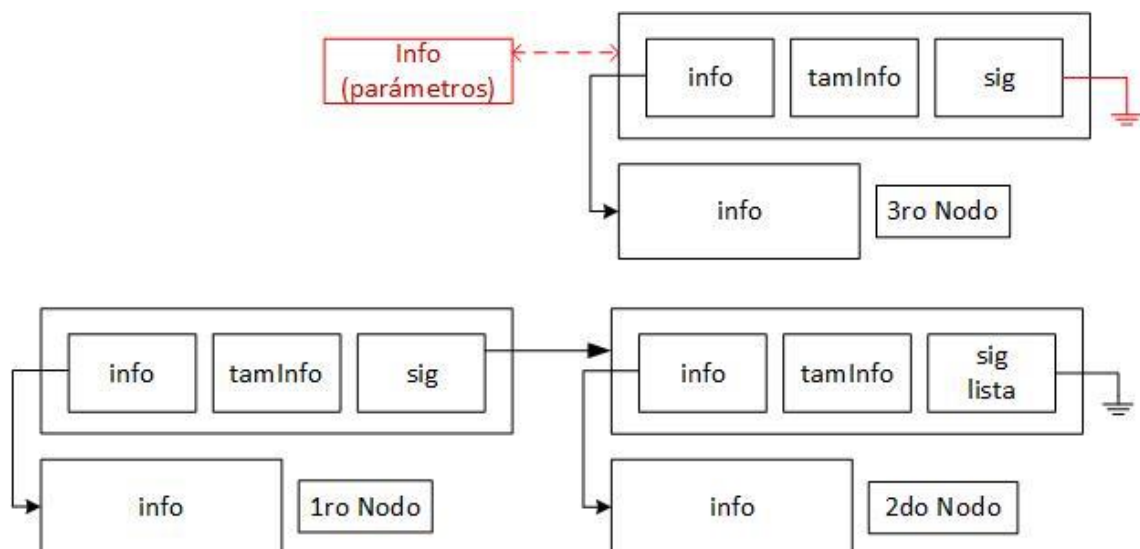
2.2. Se continúa posicionando el puntero lista, nos desplazamos al mismo lugar que el campo sig del nodo 2. Su contenido es nulo, apunta a NULL, por lo que aca finaliza el bucle.



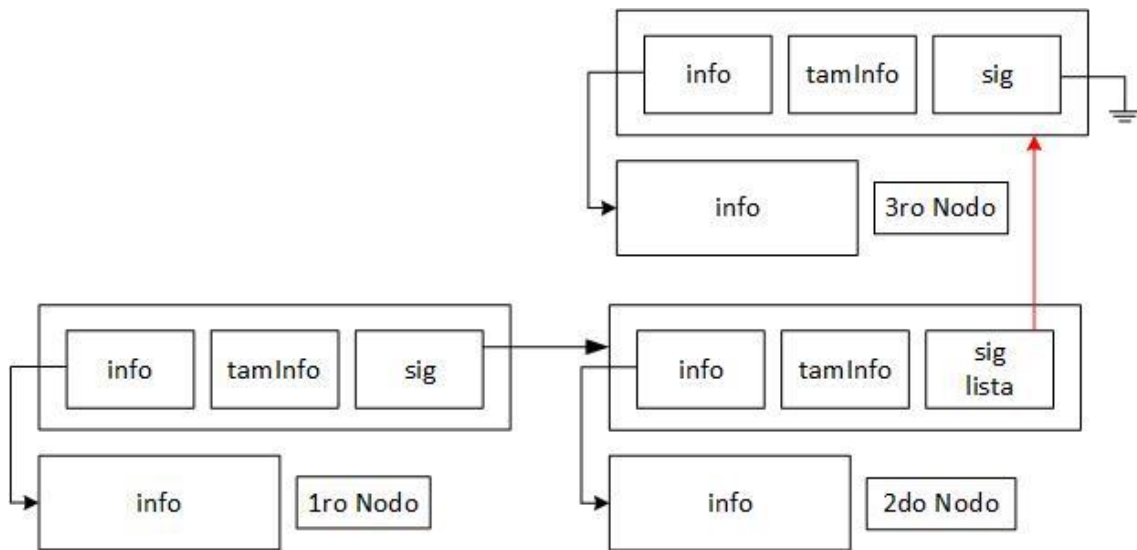
3. Como ya estamos posicionados, estamos en condiciones de llamar la función 'insertar principio lista'. De todas maneras, vamos a continuar la explicación:



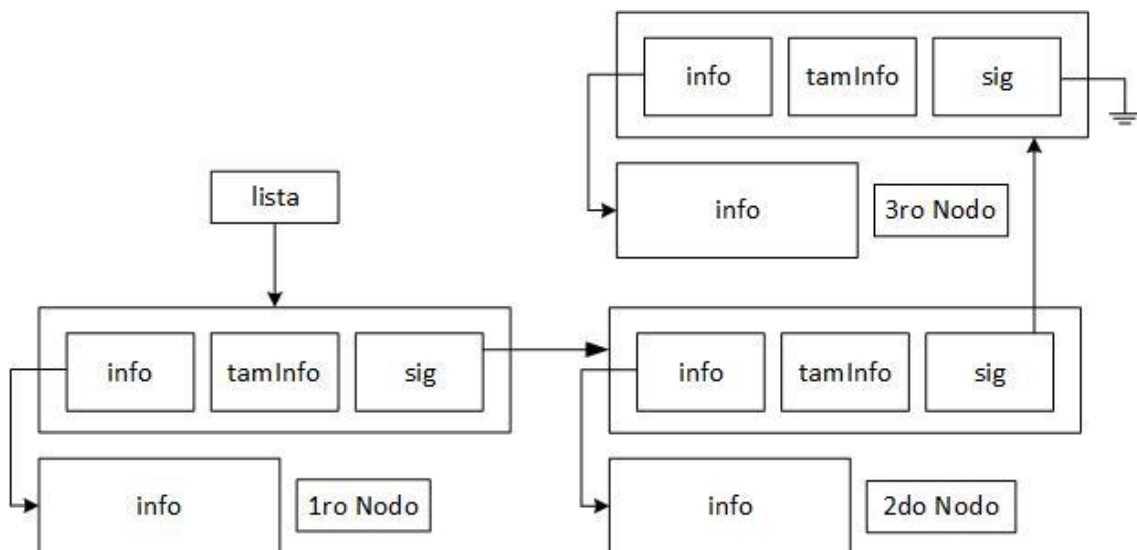
4. Se copian los datos al nuevo nodo creado. Info, tamaño de los parámetros y el nodo siguiente. El puntero a sig, apuntara al mismo lugar que se encuentra lista, en este caso es NULL como debe ser, por lo que no se produce error.



5. Se conecta el nodo anterior con el nuevo nodo



6. Función finalizada, por lo que queda como resultado final:



Ver Nodo del Final

```
int verFinLista(tLista *lista, void *info, size_t tamInfo)
{
    if(!*lista)
        return LISTA_VACIA;

    // Avanzamos hasta posicionarnos en el campo sig del ante último nodo
    while((*lista)->sig)
        lista = &(*lista)->sig;
    // Ya estamos en el campo sig del ante último nodo (que apunta al último)

    // Copiamos la información del último nodo a los parametros
    memcpy(info, (*lista)->info, MIN(tamInfo, (*lista)->info));

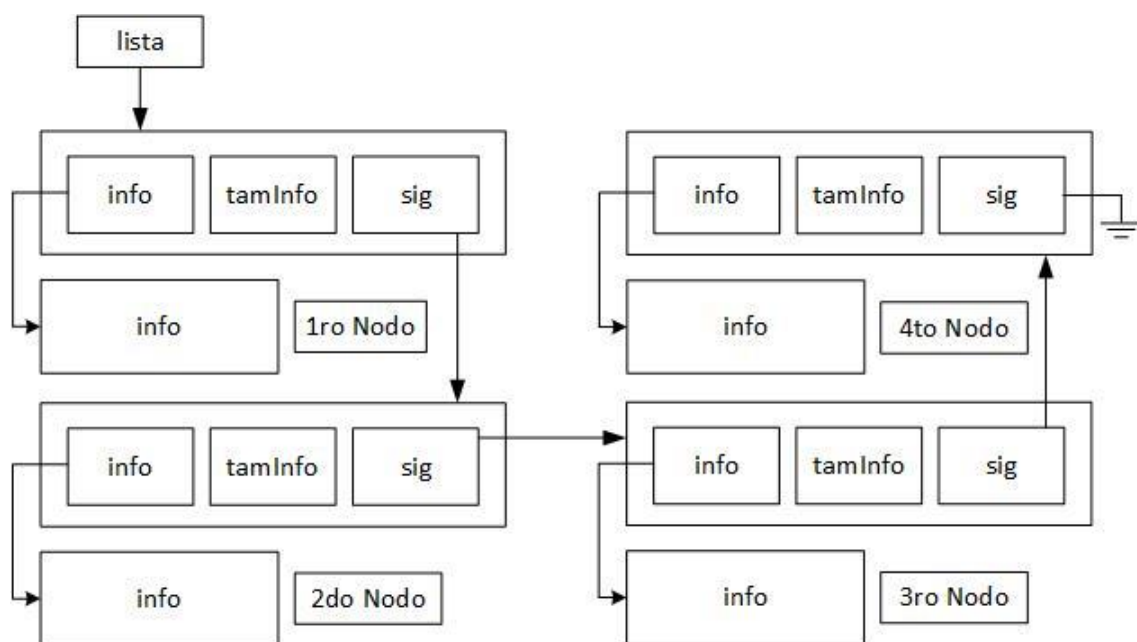
    return EXITO;
}
```

Primero, nos deslizamos hasta el campo *sig* del ante ultimo nodo de la lista, que apunta al nodo último, ahora la variable *lista* se encuentra en ese campo. Una vez posicionados, copiamos la información al parámetro *info*, luego retornamos el éxito.

En este caso, no podemos re utilizar la función de *verPrinLista* por temas de eficiencia. Para insertar se utiliza todo el código de la secuencia, en este caso no como comprobar si está vacía la lista.

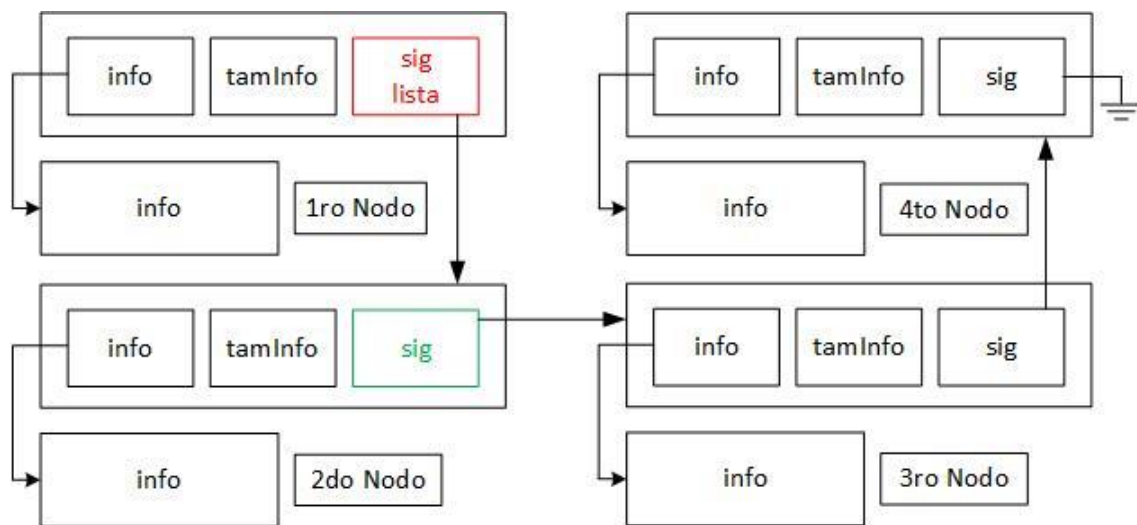
Vamos a darle una representación gráfica:

1. Se tiene la siguiente lista, se quiere obtener el contenido del último nodo:

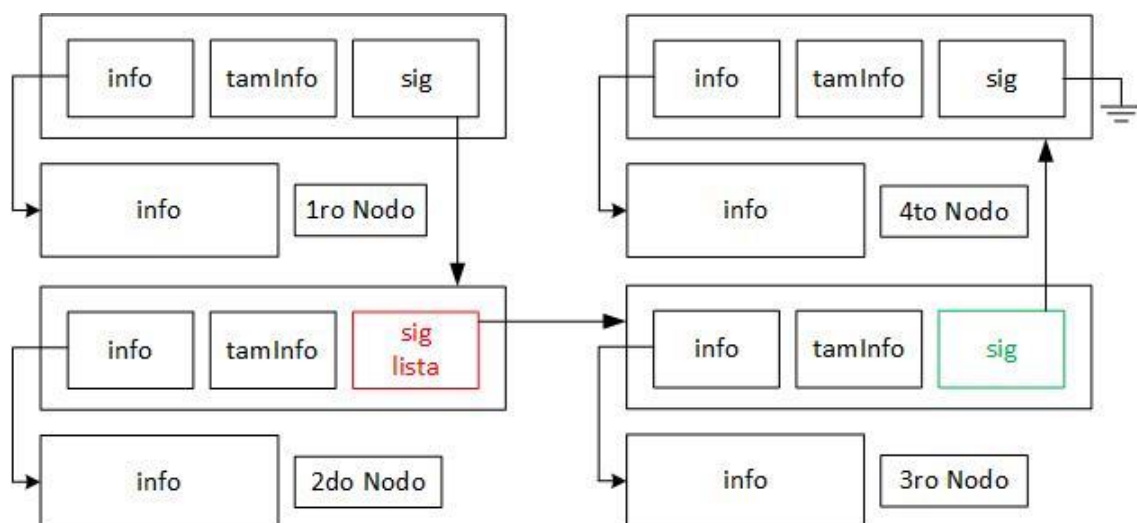


2. Vamos a comenzar a desplazarnos en forma de bucle hasta llegar al último nodo. *Lista* se pondrá en el mismo lugar que *sig* para simplificar, pero solo tienen el mismo contenido, no están posicionados en el mismo lugar. Como estamos con otro tipo de bucle, este finalizará cuando el campo *sig* del siguiente nodo, se encuentre en NULL.

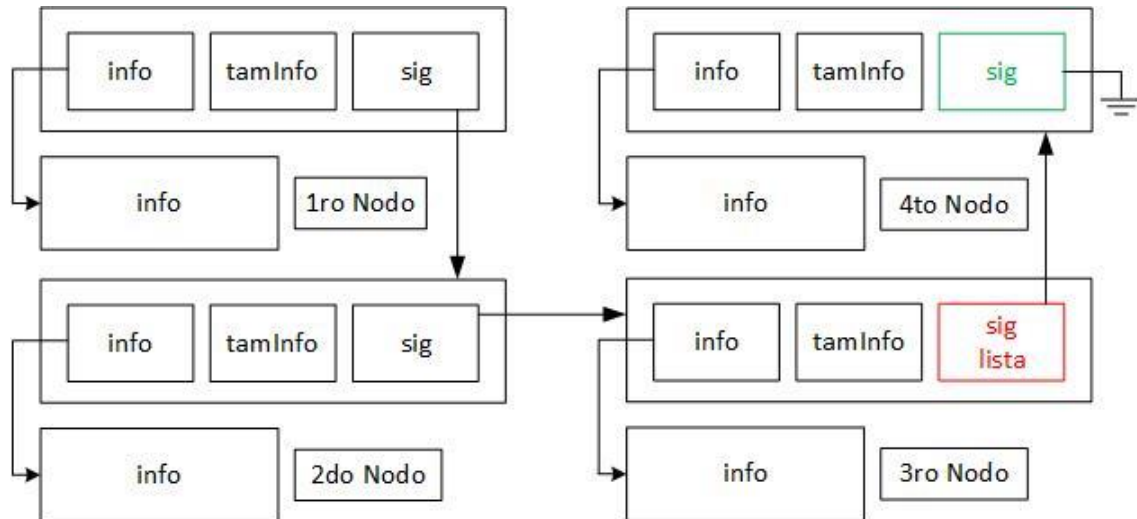
2.1. Se comienza posicionando el puntero *lista*, en el mismo lugar que el campo *sig* del nodo 1. Como el contenido del campo *sig* del siguiente nodo no es nulo, se continua.



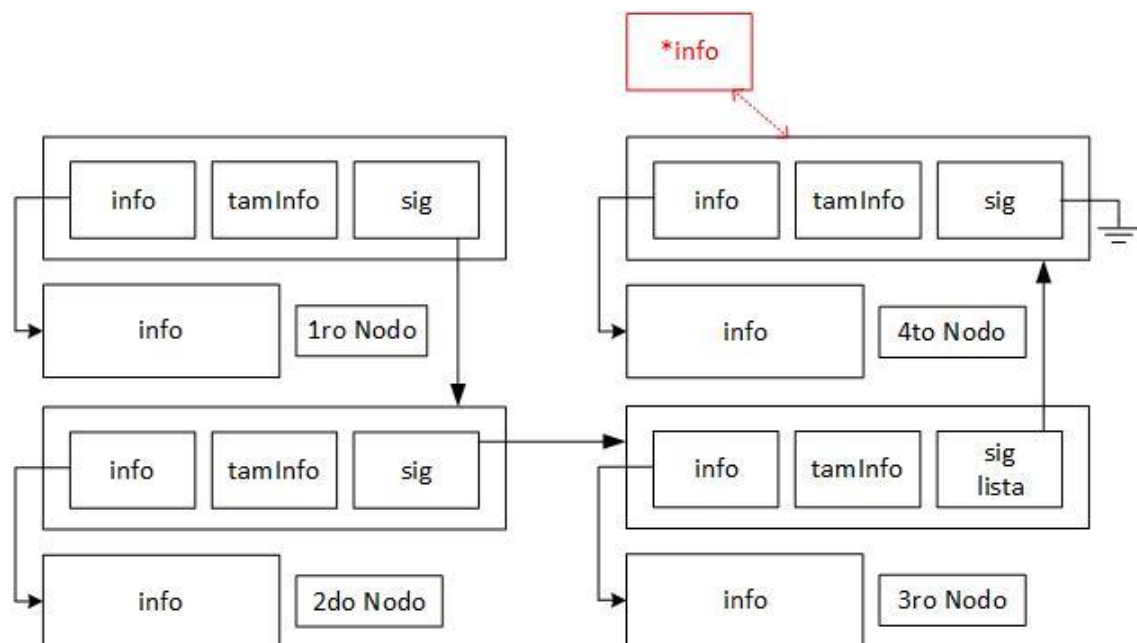
2.2. Se continúa posicionando el puntero *lista*, en el mismo lugar que el campo *sig* del siguiente nodo (nodo 2). Como el contenido del campo *sig* del siguiente nodo no es nulo, se continua.



2.3. Se continúa posicionando el puntero *lista*, en el mismo lugar que el campo *sig* del siguiente nodo (nodo 2). Como el contenido del campo *sig* del siguiente nodo es nulo, se finaliza. El puntero *lista* se queda apuntando al último nodo.



3. Se copia el contenido de este último nodo, a dirección *info* dada por parámetro.



Sacar Nodo del Final

La función *sacarUltimoLista* copia el contenido que se encuentra en el último nodo al parámetro *info*, para luego eliminar el nodo.

```
int outFinLista(tLista *lista, void *info, size_t tamInfo)
{
    if (!*lista)
        return LISTA_VACIA;

    // Avanzamos hasta posicionarnos en el campo sig del ante último nodo
    while ((*lista)->sig)
        lista = &(*lista)->sig;
    // Ya estamos en el campo sig del ante último nodo (que apunta al último)

    // Copiamos la información del último nodo a los parámetros
    memcpy(info, (*lista)->info, MIN(tamInfo, (*lista)->tamInfo));

    // Liberamos el nodo
    free((*lista)->info);
    free(*lista);

    // El campo sig del ante ultimo nodo debe ser nulo
    *lista = NULL;

    return EXITO;
}
```

Nos desplazamos por la lista enlazada hasta llegar al campo *sig* del ante-ultimo nodo, en que apunta al último nodo. Copiamos el contenido del último nodo al parámetro 'info'. Liberamos la memoria del contenido y nodo en sí, y luego hacemos que el campo *sig* sea nulo indicando que es el último; re utilizamos la función sacar primero de la lista.

Operaciones en Posiciones Determinadas

Se realizan operaciones en una posición determinada, tiene un actuar similar a un vector estático. Nosotros decidimos si queremos que comience a contar desde 0 o 1, esto depende del programador. No obstante, en el parcial se indicará si se quiere que comience a leer desde alguno de esos 2 números. Por mi parte, como costumbre a los vectores, comencare a contar desde cero.

Las operaciones, son más de lo mismo.

Insertar Nodo en Posición Determinada

```
int insPosLista(tLista *lista, void *info, size_t tamInfo, size_t pos)
{
    // Nos desplazamos hasta la posición deseada o hasta el final de la lista
    while(*lista && pos)
    {
        lista = &(*lista)->sig; // Avanzamos al siguiente nodo
        pos--;                    // Reducimos la posición restante
    }

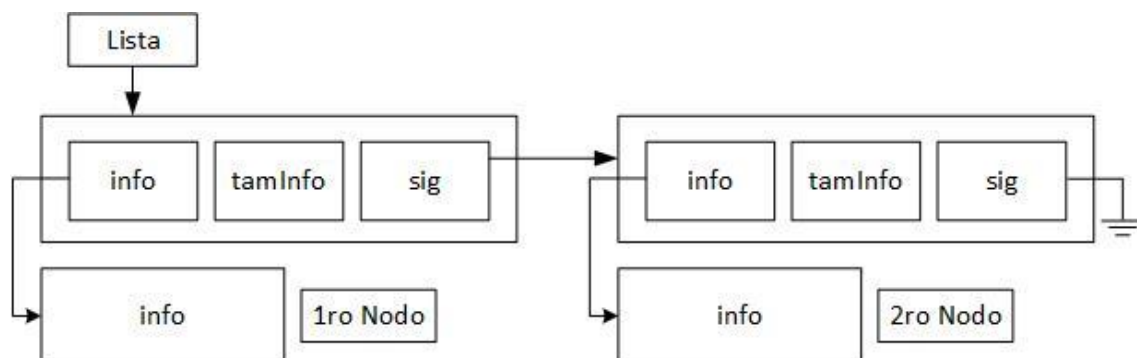
    // No recorrió todo
    if(pos)
    {
        return POS_INVALIDA;
    }

    // Re utilizamos el código
    return insPrinLista(lista, info, tamInfo);
}
```

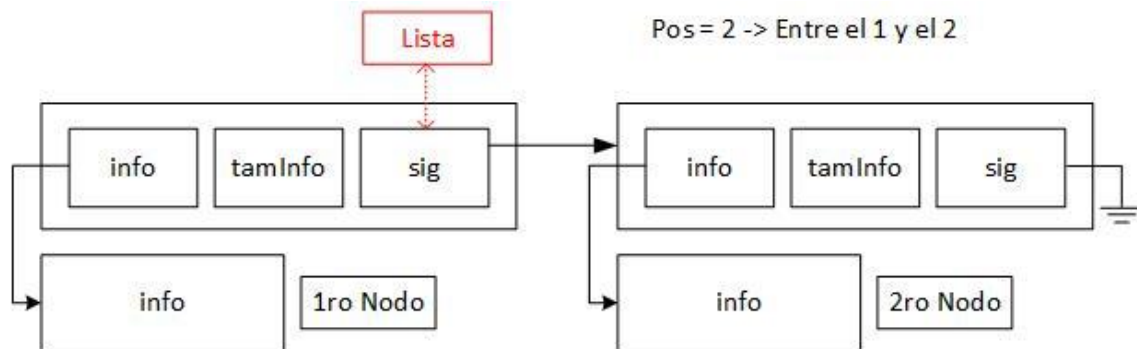
Primero, nos deslizamos hasta la posición deseada. Si la variable pos no es cero, se detuvo antes de llegar donde queremos, entonces surgió un error y se notifica que no existe la posición. Como estamos con insertar, no es necesario verificar si **lista* es null. Luego, como ya se dijo anteriormente, es más de lo mismo, por lo que re utilizamos la función de *insertar principio de lista*.

Vamos a darle una explicación gráfica:

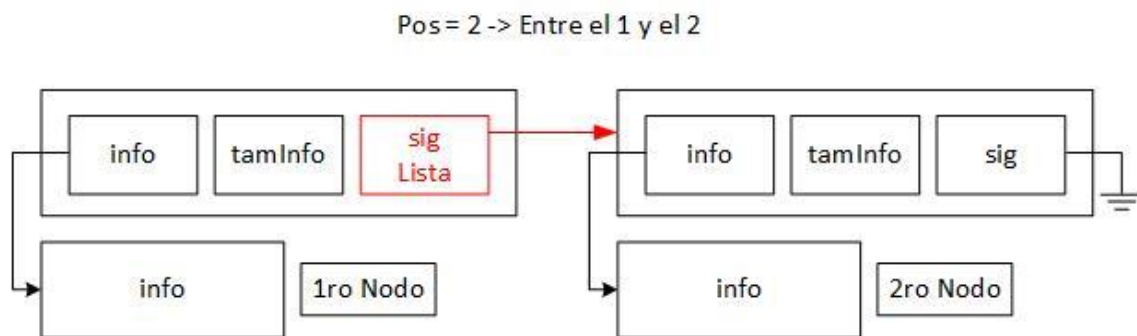
1. Se presenta la siguiente lista con 2 nodos:



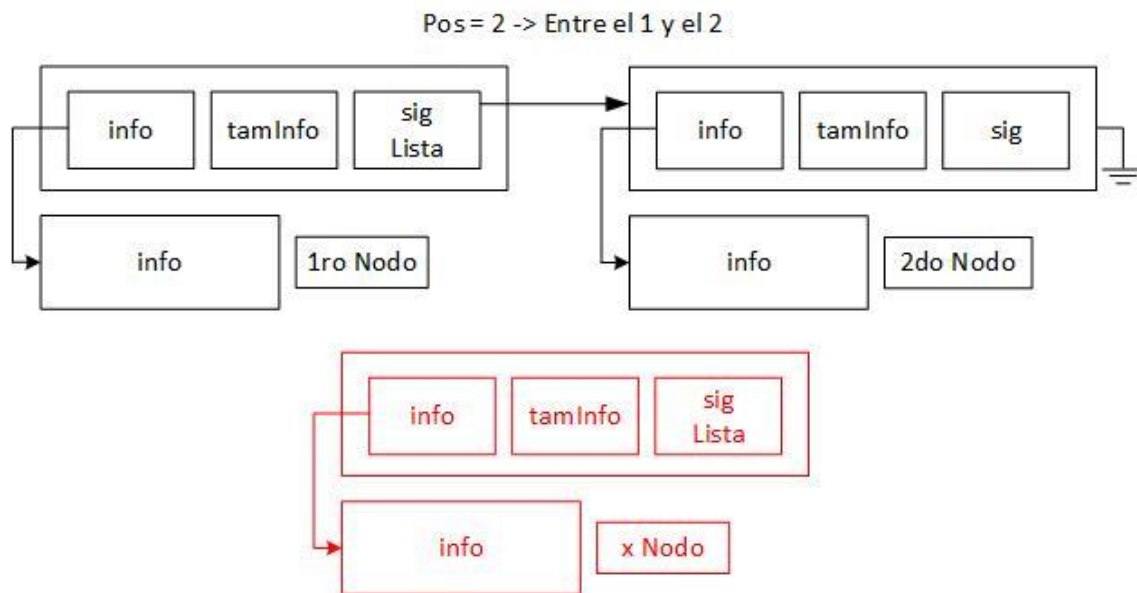
2. Se desea insertar un nuevo nodo en la posición 2. Recorremos la lista hasta llegar a tal posición. La lista, ahora se encuentra en el mismo que el campo *sig* del nodo 1 (el anterior que el 2).



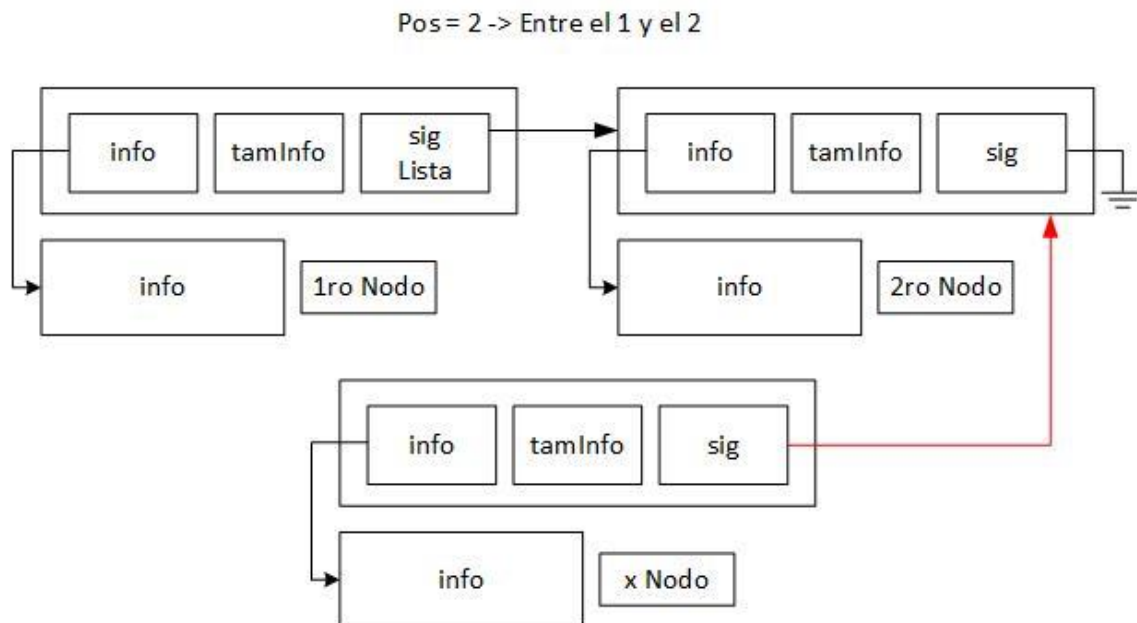
3. Como están en la misma posición, vamos a poner ambas direcciones en un mismo cuadro para simplificar el grafico.



4. Se crea un nuevo nodo donde ya insertamos sus respectivos datos e información.

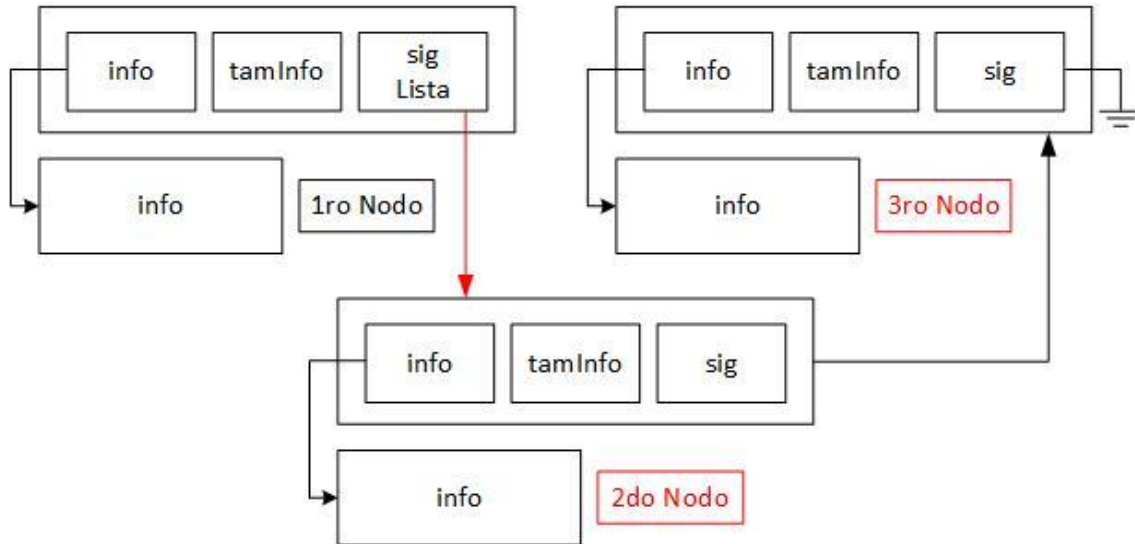


5. El nuevo nodo, va a comenzar apuntar donde apunta *lista*.



6. El anterior nodo va a comenzar a apuntar hacia el nuevo nodo. Para esto, el campo *lista* va a comenzar a apuntar hacia el nuevo.

Pos = 2 -> Entre el 1 y el 2



7. Quedando como resultado final, de manera más clara:



Ver Nodo en Posición Determinada

```
int verPosLista(tLista *lista, void *info, size_t tamInfo, size_t pos)
{
    // Avanzamos hasta la posición deseada
    while(*lista && pos)
    {
        lista = &(*lista)->sig; // Avanzamos al siguiente nodo
        pos--;                  // Reducimos la posición restante
    }

    // No recorrió todo
    if(!*lista || pos)
        return POS_INVALIDA;

    // Copiamos la información del último nodo a los parametros
    memcpy(info, (*lista)->info, MIN(tamInfo, (*lista)->info));

    // Re utilizamos la función
    return EXITO;
}
```

Primero, nos deslizamos hasta la posición deseada.

Luego, si la variable pos no es cero, se detuvo antes de llegar donde queremos, entonces surgió un error y se notifica que no existe la posición. También, puede que justo donde nos posicionamos es el final de la lista y estamos apuntando a NULL, por lo que se debe verificar también.

Luego, se copia la información con memcpy a la información de los parámetros, y se retorna el éxito.

Sacar Nodo en Posición Determinada

```
int outPosLista(tLista *lista, void *info, size_t tamInfo, size_t pos)
{
    tNodo *aux;

    // Avanzamos hasta la posición deseada
    while (*lista && pos)
    {
        lista = &(*lista)->sig; // Avanzamos al siguiente nodo
        pos--;                  // Reducimos la posición restante
    }

    // No recorrió todo
    if (!*lista || pos)
        return ERROR_POS;

    // Conectamos el nodo anterior con el siguiente
    *lista = aux->sig;

    // Copiamos la información del último nodo a los parametros
    memcpy(info, aux->info, MIN(tamInfo, aux->tamInfo));

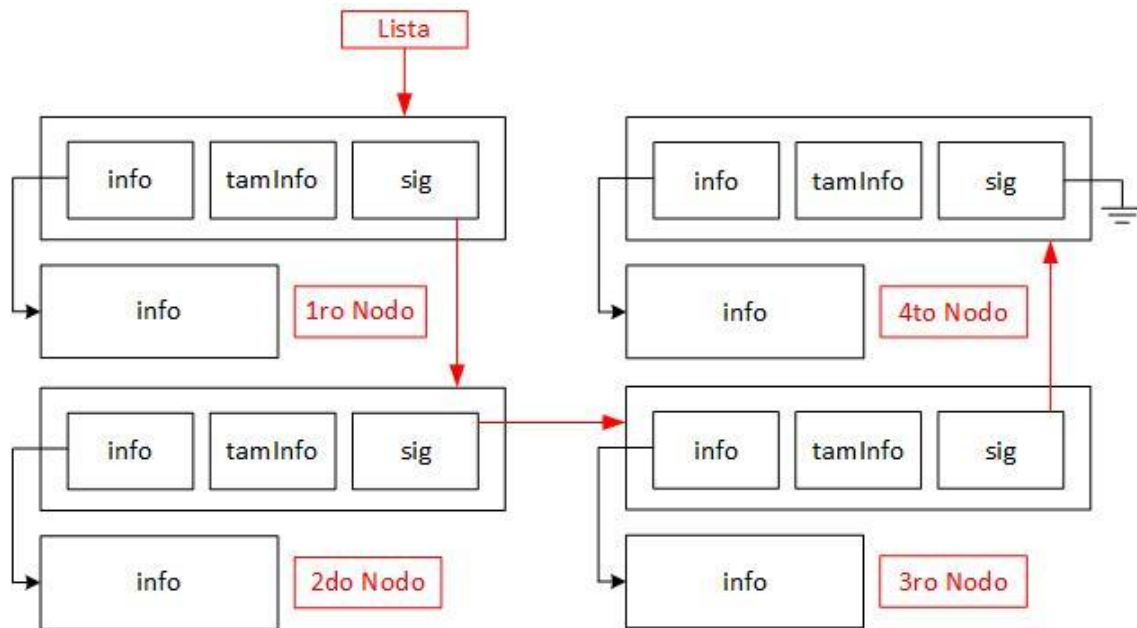
    // Liberamos el nodo
    free(aux->info);
    free(aux);

    return EXITO;
}
```

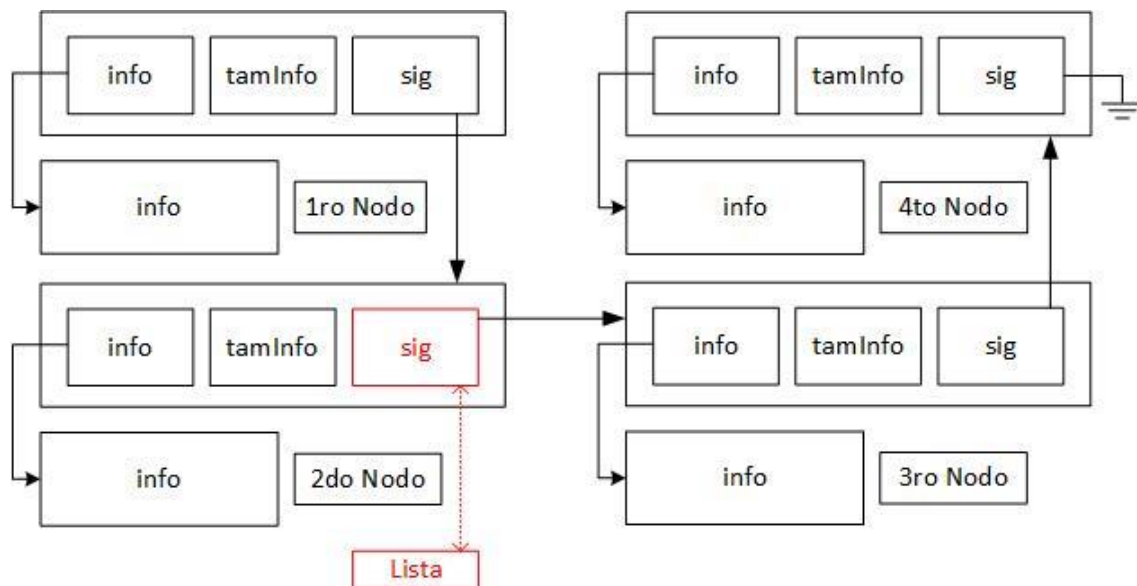
Primero, nos deslizamos hasta la posición deseada. Si la variable pos no es cero, se detuvo antes de llegar donde queremos, entonces surgió un error y se notifica que no existe la posición. También, puede que justo donde nos posicionamos es el final de la lista y estamos apuntando a NULL, por lo que se debe verificar también. Luego, como ya se dijo anteriormente, es más de lo mismo, por lo que re utilizamos la función de *sacar principio de lista*.

Vamos a realizar un análisis grafico:

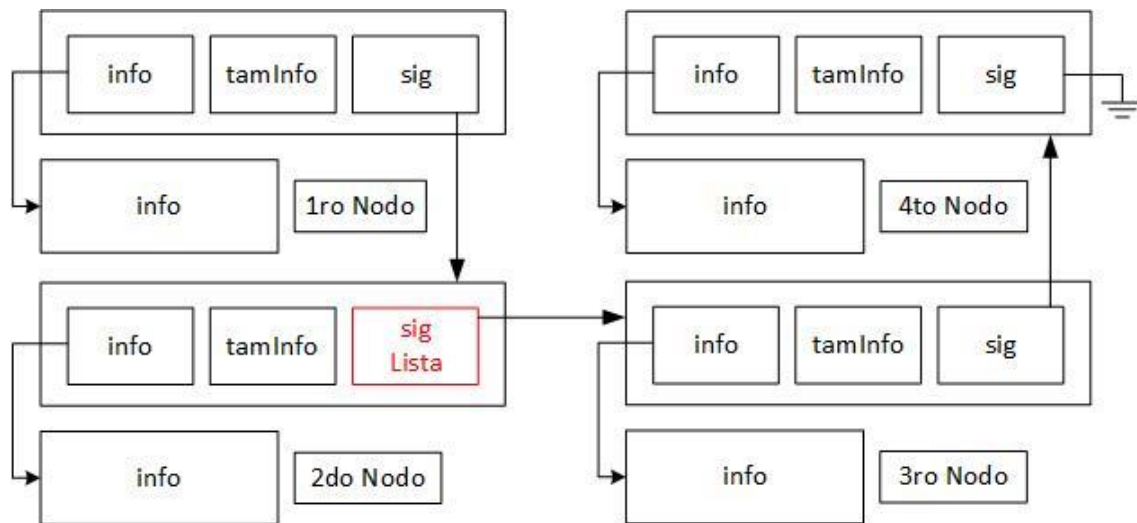
1. Se presenta la siguiente lista



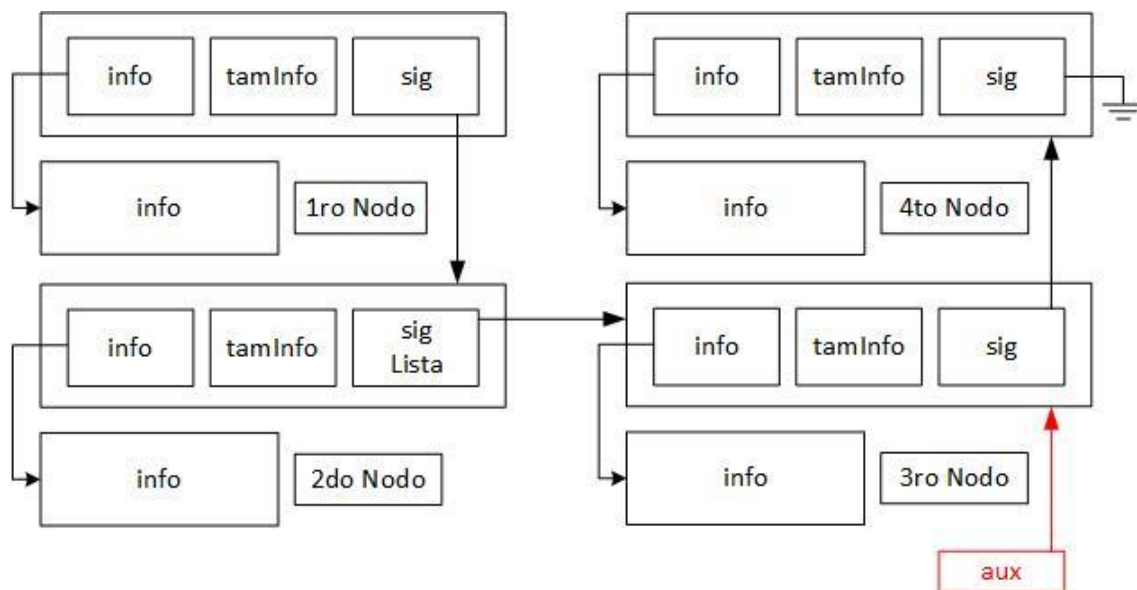
2. Se quiere eliminar la posición 3. Por lo que vamos a posicionarnos en el campo sig del nodo 2. Ahora *lista* se encuentra posicionado en el mismo lugar apuntando al nodo 3.



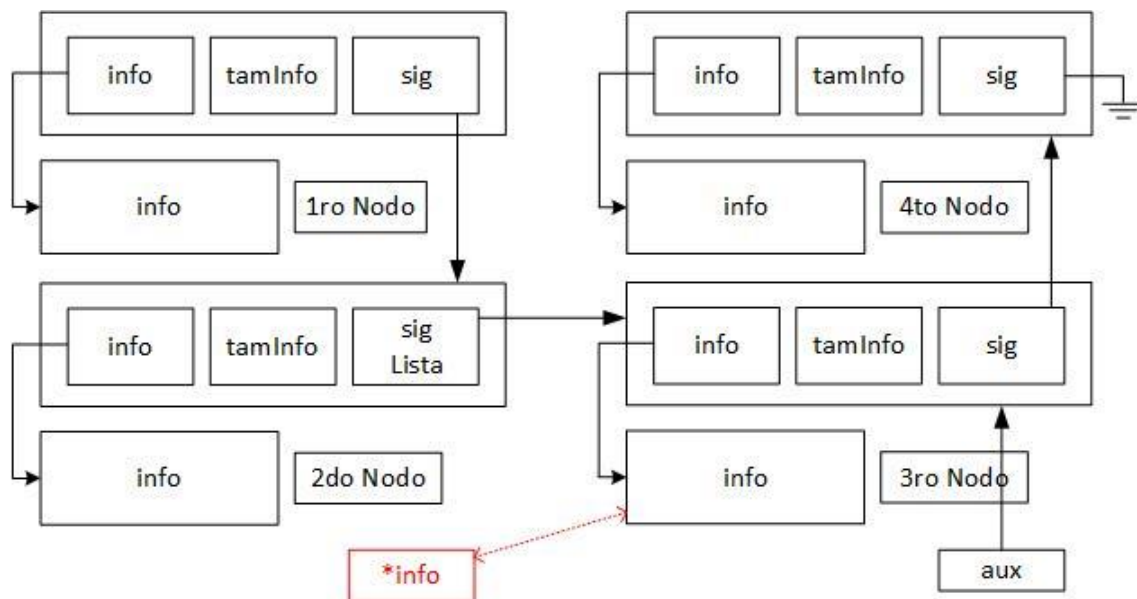
3. Para ejemplificar el diagrama, vamos a ponerlos ambos en el mismo lugar.



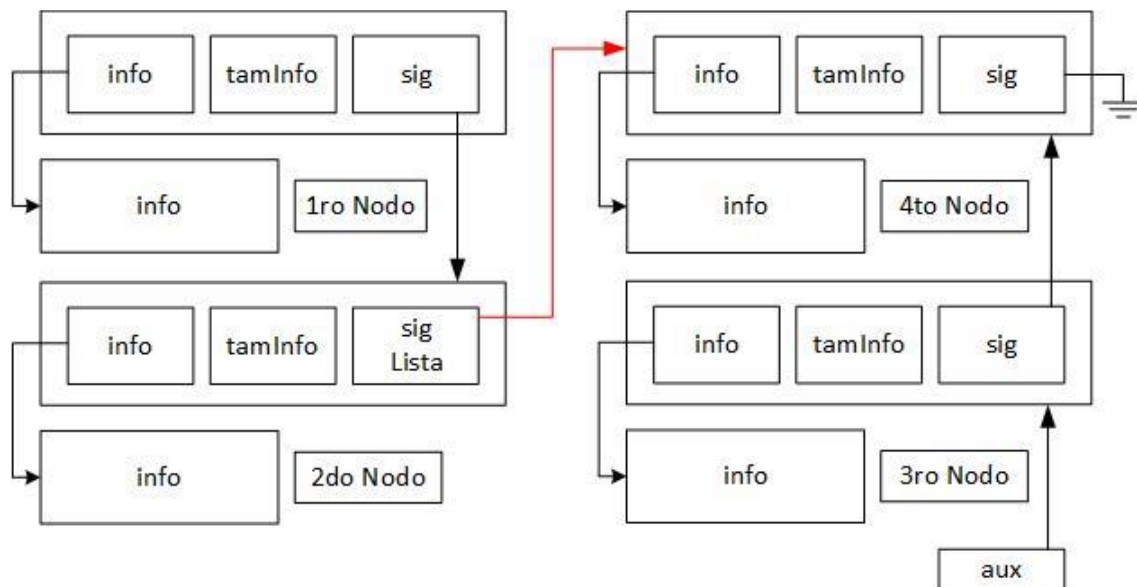
4. Se crea una variable auxiliar que apunte hacia donde apunte lista (el nodo que se quiere eliminar)



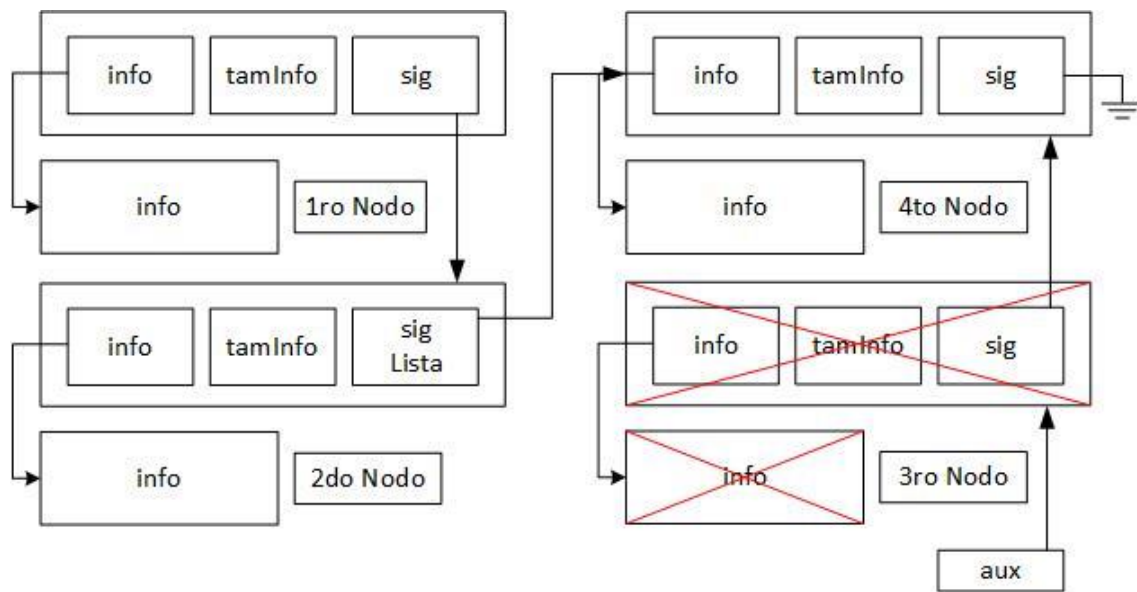
5. Se copia el contenido del nodo hacia la dirección 'info' del parámetro.



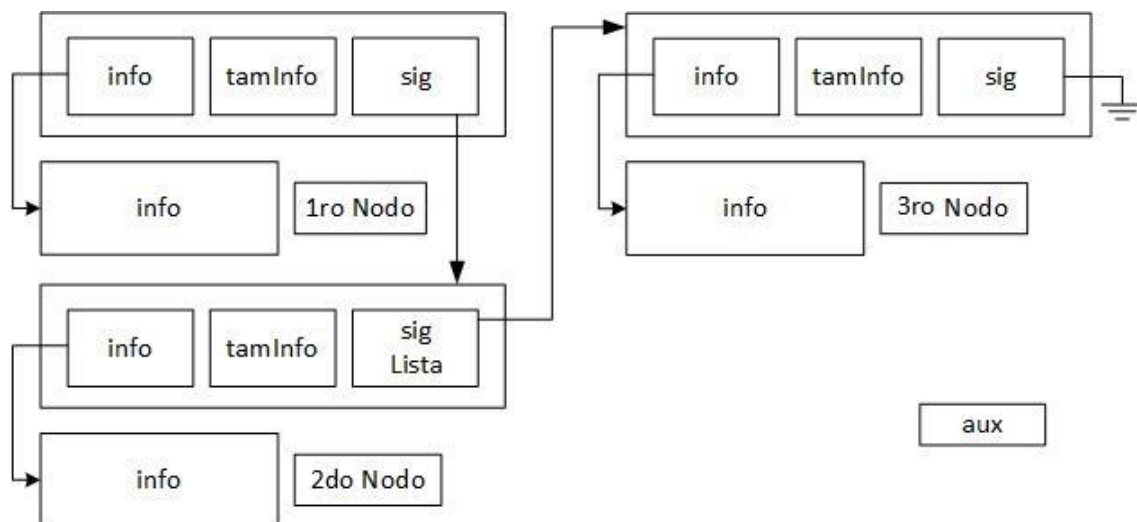
6. La lista y a su vez el campo sig del nodo anterior del que se eliminara, comenzara apuntar hacia el nodo siguiente de este.



7. Se libera la memoria del nodo que apunta la variable auxiliar aux.



8. Quedando como resultado:



Insertar Ordenado

La función *insertarEnListaOrdenada* permite insertar un nuevo nodo en una lista de forma ordenada, es una función global para cualquier tipo de dato. Esta función ya fue vista en "Tópicos de Programación", pero se lo adapta para utilizar una lista dinámica.

```
int insOrdLista(tLista *lista, void *info, size_t tamInfo, int
cmp(void*,void*))
{
    int resulCmp; // Resultado de la comparación

    // Recorremos la lista hasta encontrar la posición correcta
    while (*lista && (resulCmp = cmp(info, (*lista)->info)) > 0)
        lista = &(*lista)->sig;

    // Si encontramos un elemento igual, no se inserta (evitamos duplicados)
    if (*lista && resulCmp == 0)
        return ELEMENTO_DUPLICADO;

    // Re utilizamos insertar primero
    return insPrinLista(lista,info,tamInfo);
}
```

Recorre la lista comparando el nuevo dato con los existentes, y se detiene cuando encuentra la posición donde debería insertarse. Si encuentra un elemento igual, no lo inserta y devuelve un código de duplicado. Si la posición es válida, re utilizamos la función de insertar al principio.

Map, Filter, Reduce y Busqueda

Map

Esta función permite recorrer una lista, aplicarle una acción a cada nodo (como mostrar o sumar un numero) y aparte darle una dirección de memoria como parámetro. Ya fue vista en Tópicos de Programación, pero será adaptado para nodos, aparte se le dará un parámetro extra **param*.

```
void mapLista(tLista *lista, void accion(void*, void*), void *param)
{
    // Mientras haya nodos en la lista
    while(*lista)
    {
        // Aplica la función 'accion' al dato del nodo actual
        // Tambien, se le pasa un parámetro adicional
        accion((*lista)->info, param);

        // Avanza al siguiente nodo en la lista
        lista = &(*lista)->sig;
    }
}
```

El **parámetro** **param* sirve para enviar un dato adicional que es necesario al momento de recorrer, como, por ejemplo: un puntero a un archivo para escribir en él. Es una dirección de memoria genérica, por lo que puede ser cualquier tipo de dato, incluyendo estructuras. En caso que no se quiere enviar un parámetro, tan solo se escribe NULL en el argumento.

No se la pasa el tamaño del nodo, siendo que se supone que ya es conocido de ante mano, y se sabe si es tipo número, cadena, flotante, etc... En la función acción se decide.

Ejemplo:

Se tiene una lista de alumnos guardado en un TDA Lista, se quiere guardar cada alumno en un archivo con toda su estructura

```
typedef struct
{
    long int dni;
    char alumno[50];
} tAlumno;
```

Se utilizará el siguiente main:

```
int main()
{
    tLista lista;
    FILE *arch;

    if (!(arch = fopen("alumnos.dat", "wb")))
    {
        return ERROR_ARCH;
    }

    crearLista(&lista); // Se crea la lista

    /*
    while()
    {
        Se cargan los alumnos en la lista mediante insertar...
    }
    */

    mapLista(&lista, aluTxt, arch); // Aplicamos la función a cada nodo
    vaciarLista(&lista); // Liberamos la memoria
    fclose(arch); // Cerramos el archivo

    return EXITO;
}
```

Como se puede ver, en este caso, enviamos como parámetro un puntero de un archivo, se utilizará para escribir los alumnos en el archivo y no perder la dirección.

```
void aluBin(void *info, void *param)
{
    tAlumno *alumno = (tAlumno*)info;
    FILE *arch = (FILE*)param;

    fwrite(alumno, sizeof(tAlumno), 1, arch);
}
```

Esta es la función que será ejecutada para cada nodo.

Reduce

Se aplica una función acumulativa o de reducción a los nodos de la lista, de manera que se recorre la lista para ser reducida a un solo valor inicial. No se eliminan los nodos o el contenido, solo se obtiene y se interactúa para ser acumulado a un valor inicial.

```
void reduceLista(tLista lista, void redu(void*,void*,void*), void *result, void *param)
{
    // Mientras haya nodos en la lista
    while (*lista)
    {
        // Aplica la función 'redu' al dato del nodo actual
        redu((*lista)->info, result, param);

        // Avanza al siguiente nodo
        lista = &(*lista)->sig;
    }
}
```

La función redu será ejecutada en cada nodo de la lista, recibe 3 parametros:

1. **Información* -> Contenido del nodo
2. **result* -> Valor inicial para el acumulador
3. **param* -> Mismo concepto visto anteriormente con *map*

El valor inicial debe ser definido de antemano de ejecutar la función reduce.

Ejemplo:

Se tiene una lista de alumnos semejante. Solo que esta vez no se quiere escribir en un archivo (a pesar que se puede), sino que se quiere obtener el promedio de las notas.

```
typedef struct
{
    long int dni;
    char alumno[50];
    float nota; // Del 1 al 10
} tAlumno;
```

Para obtener el promedio de las notas de los alumnos, se debe tener la suma total de notas y aparte la cantidad. El problema, es que no acepta 2 argumentos que sea como acumulador, no en teoría, no se debería utilizar **param* para eso. Por lo que, se creará una estructura promedio donde será enviado como argumento **result*.

```
typedef struct asd
{
    float sumaTotal;
    size_t cant;
} tProm;
```

Se tiene el siguiente main, donde creamos la lista de alumnos, la variable de promedio. Enviamos la lista a *reduce* para obtener el promedio de notas y luego imprimimos el resultado.

```
int main()
{
    tLista listAlu;
    tProm promAlu = {0,0};

    crearLista(&listAlu); // Se crea la lista

    /*
    while()
    {
        Se cargan los alumnos en la lista mediante insertar...
    }
    */

    reduceLista(&listAlu, obtPromAlu, &promAlu, NULL);
    printf("El promedio es %.2f\n", promAlu.sumaTotal/promAlu.cant);
    vaciarLista(&listAlu); // Liberamos la memoria

    return EXIT0;
}
```

Se tiene la siguiente función que será ejecutada en todos los nodos de la lista, con esto es suficiente para rellenar los campos a tal fin de obtener el promedio.

```
void obtPromAlu(void *info, void *result , void *param)
{
    tAlumno *alumno = (tAlumno*)info;
    tProm *promAlu = (tProm*)result;

    promAlu->cant++;
    promAlu->sumaTotal += alumno->nota;
}
```

Filter

Filter es una función que recorre la lista y a cada nodo se le ejecuta una función que devuelve true o false. Según el resultado de la función al nodo, este se eliminará o se lo dejará en la lista (false = eliminado). De ahí el nombre de "filtro".

```
tLista *filterLista(tLista *lista, int filtro(void *, void *), void *param)
{
    // Guardamos la dirección original del primer puntero
    tLista *inicio = lista;

    while (*lista) // Mientras haya nodos en la lista
    {
        // Si el nodo cumple la condición, avanzamos
        if ( filtro((*lista)->info, param) )
        {
            lista = &(*lista)->sig;
        }
        else // Si no cumple, lo eliminamos
        {
            tNodo *elim = *lista; // Guardamos el nodo a eliminar
            *lista = elim->sig;    // Lo salteamos (conectamos al siguiente)
            free(elim->info);      // Liberamos la memoria de la info
            free(elim);           // Liberamos el nodo en sí
        }
    }

    return inicio; // Retornamos el inicio de la lista (puede ser NULL)
}
```

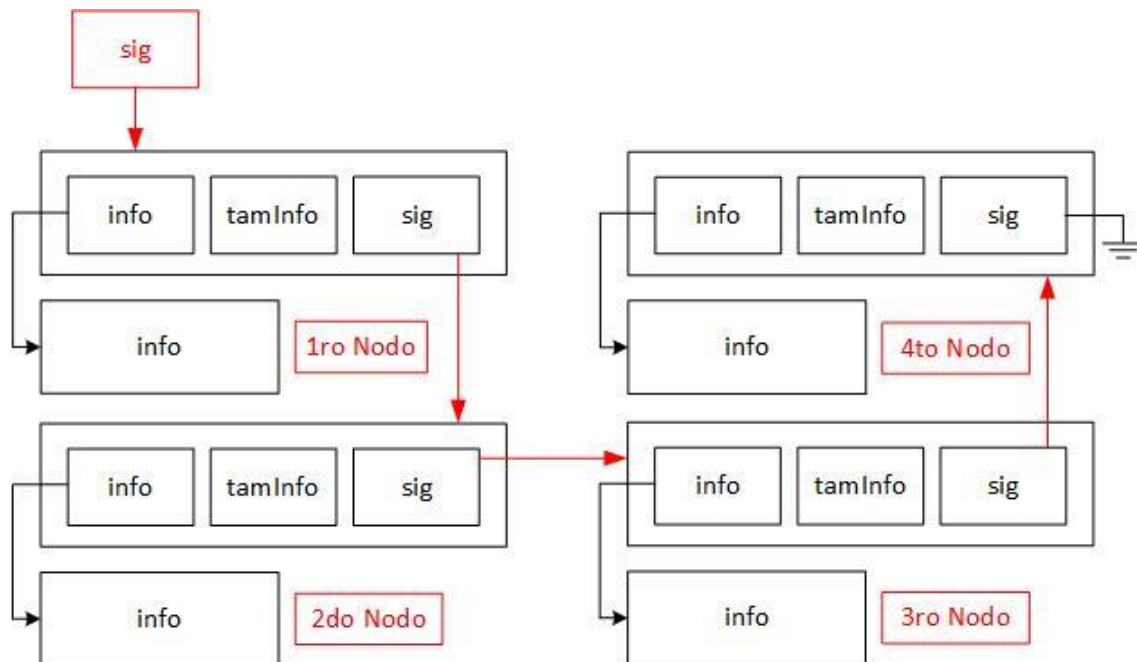
La función **retorna** la dirección de memoria original de la lista. La razón por la que devuelve la dirección a diferencia de map y reduce, es porque se está modificando la lista original (se elimina nodos o no), a diferencia de los otros que solo se recorre para ejecutar una función. Entonces, si se eliminan todos los nodos, filter retorna NULL, indicando que está vacía la lista.

Su algoritmo, **consiste en** recorrer la lista hasta finalizar. Si se cumple la condición de filtro (true), quiere decir que la condición en ese nodo está aprobada y pasa al siguiente, en caso contrario lo eliminamos. Para eliminarlo debemos conectar el nodo anterior al siguiente del eliminado, y liberar la dirección de memoria; mismo proceso que la función de [sacar nodo en posición determinada](#).

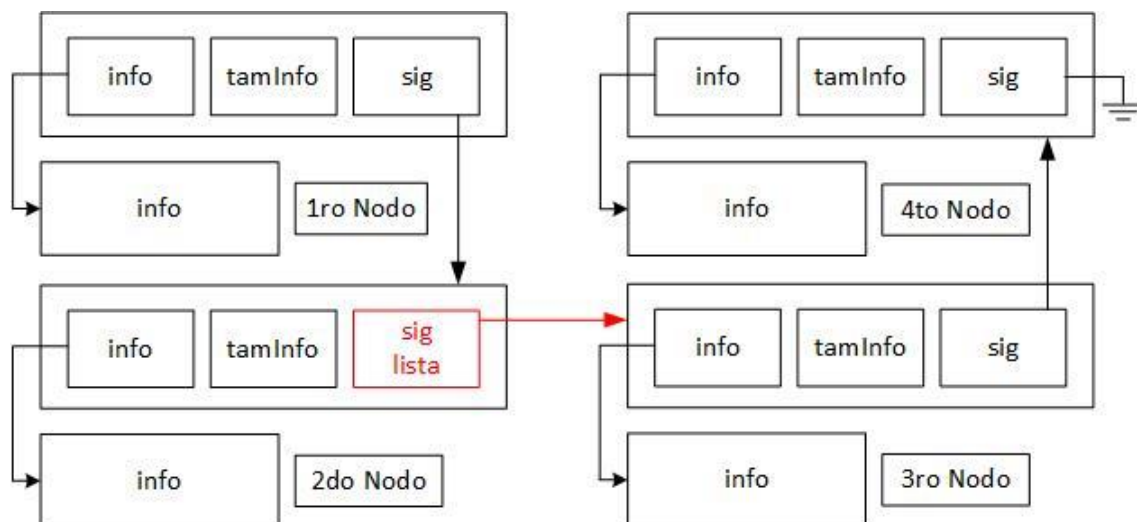
En cuanto a los **argumentos**, son los mismos que recibe reduce y map. Por lo que no se dará mucha más explicación.

Vamos a dar una explicación grafica del proceso de eliminado(salteo) del nodo luego de que la condición sea falsa (línea 15):

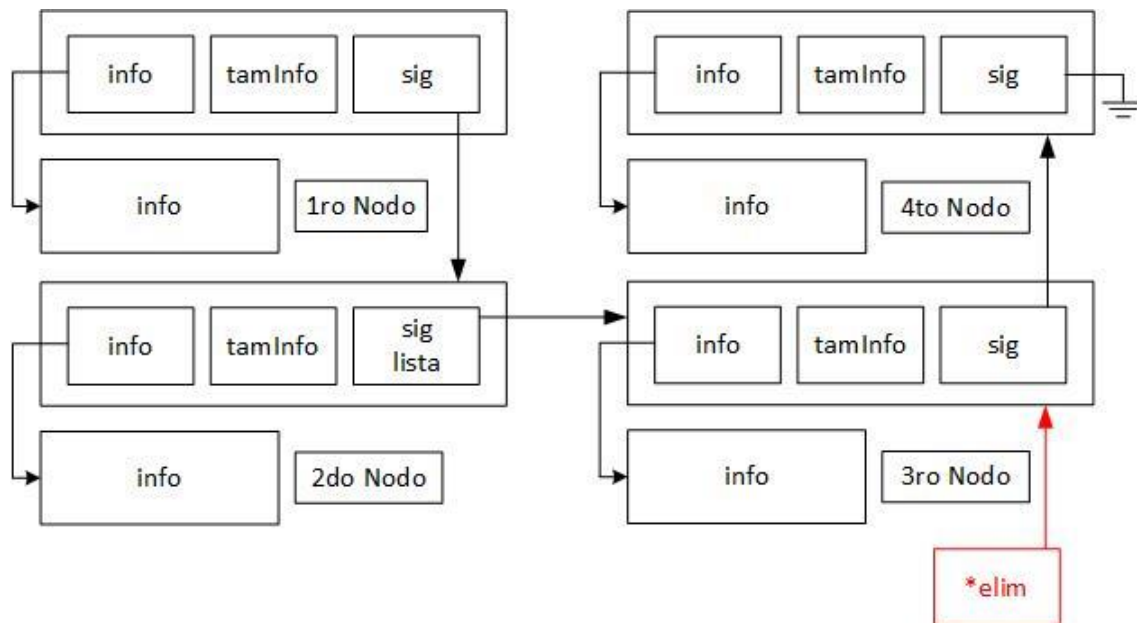
1. Se representa la siguiente lista con sus 4 nodos.



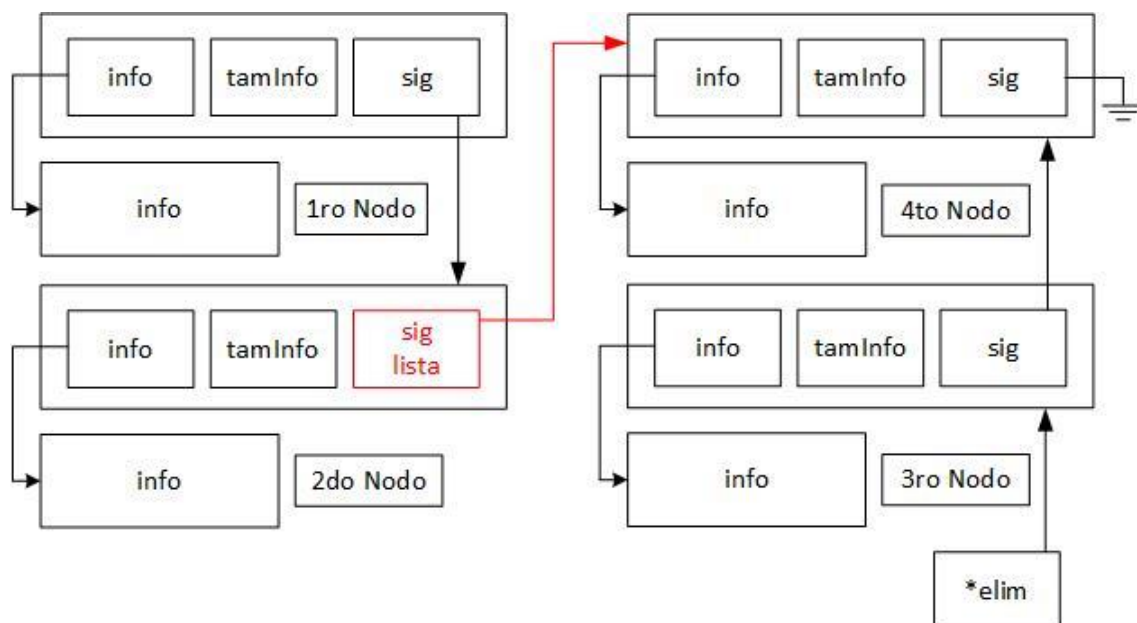
2. Se comienza a recorrer la lista y ejecutar una función para evaluar una condición sobre eliminar o no. Se llega a la conclusión, que el 3° nodo debe ser eliminado.



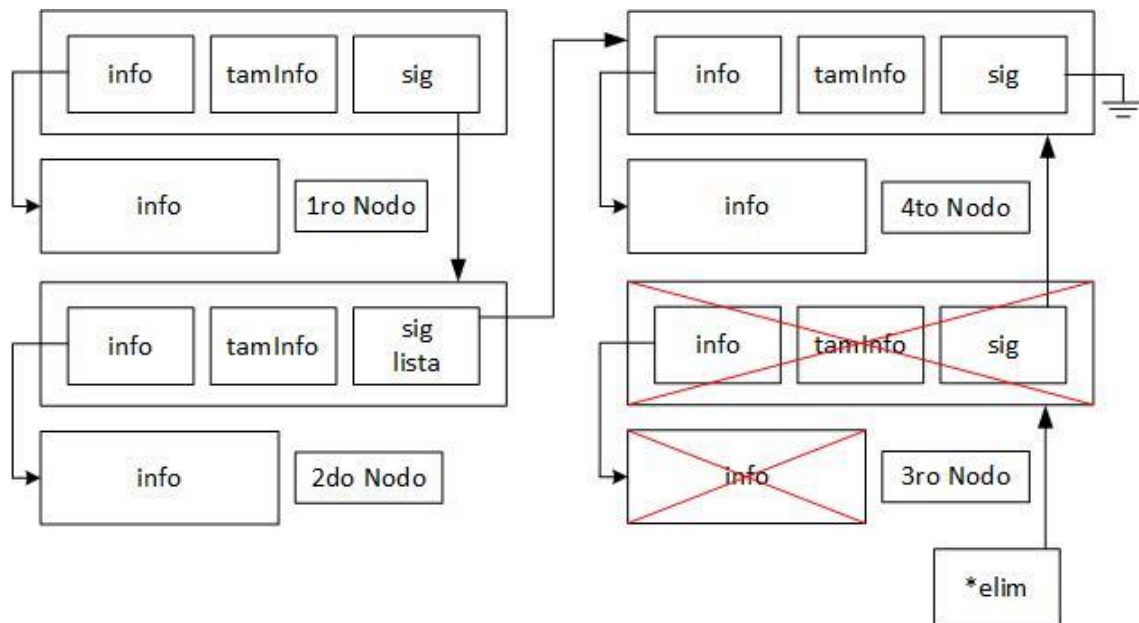
3. Se crea una función auxiliar para apuntar al nodo a eliminar y no perder su dirección de memoria.



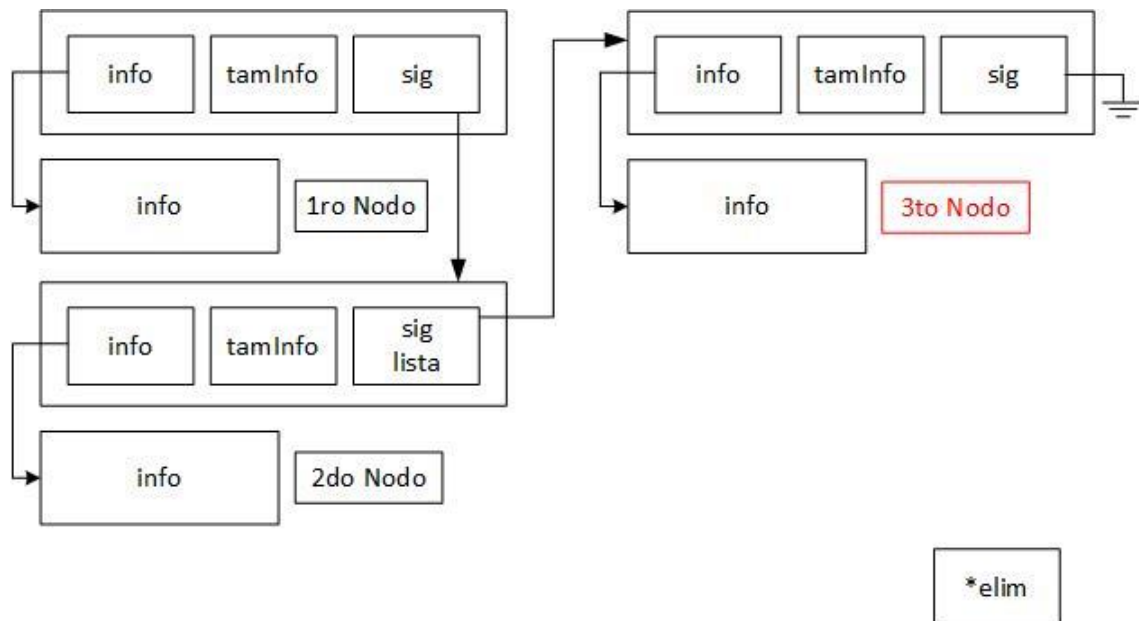
4. El 2º nodo comenzara apuntar al nodo siguiente del eliminado, saltando el nodo eliminado.



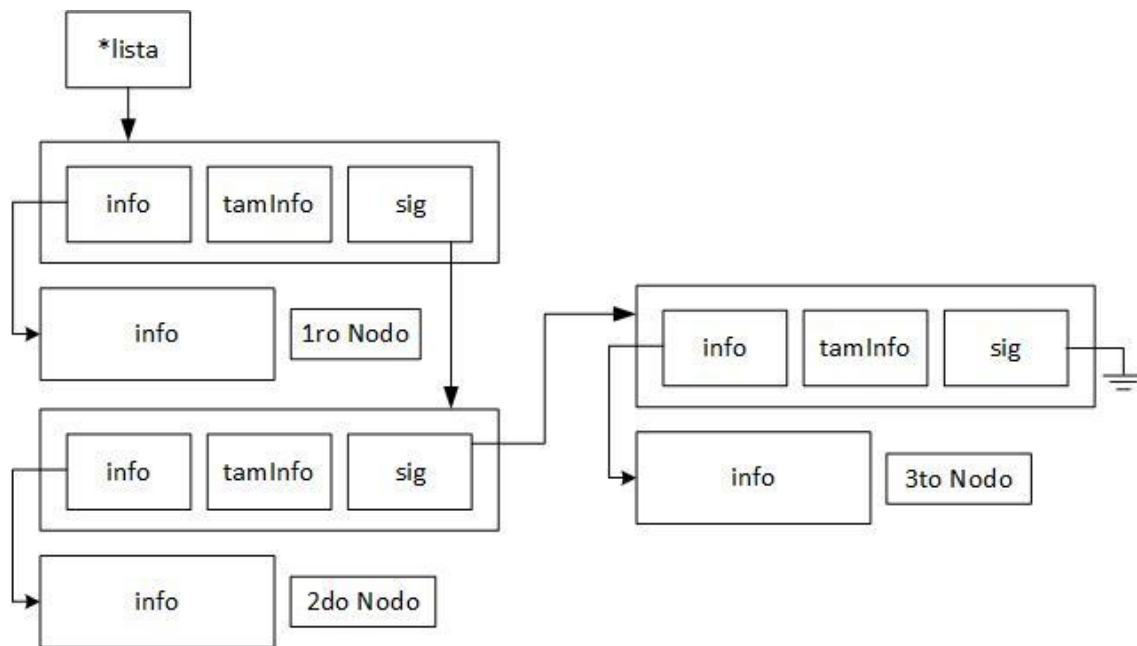
5. Aprovechando la variable auxiliar, se elimina la dirección de memoria del contenido del nodo eliminado y el nodo en sí.



6. El nodo es eliminado, quedando la siguiente lista como resultado final.



7. Pasando la lista a limpio, queda el siguiente resultado final.



Ejemplo:

Se tiene la misma lista de alumnos con su misma estructura. Se quiere eliminar aquellos alumnos que tienen una nota menor a 7.

```
typedef struct
{
    long int dni;
    char alumno[50];
    float nota; // Del 1 al 10
} tAlumno;
```

Se hará uso del siguiente main.

```
int main()
{
    tLista lista;

    crearLista(&lista); // Se crea la lista

    /*
    while()
    {
        Se cargan los alumnos en la lista mediante insertar...
    }
    */

    // Aplicamos la función a cada nodo
    filterLista(&lista, filtroNota, NULL);
    vaciarLista(&lista); // Liberamos la memoria

    return EXIT0;
}
```

Para crear la función condicional sobre cual alumno se queda o se va de la lista, se hace uso de la siguiente función que será ejecutada en cada nodo.

```
void filtroNota(void *info, void *param)
{
    tAlumno *alumno = (tAlumno*)info;

    // Nota >= 7 => No se elimina
    // Nota < 7 => Se elimina
    return alumno->nota >= 7 ? 1 : 0;
}
```

Búsqueda

Esta función nos permite realizar una búsqueda en la lista, y en la primera coincidencia nos retorna la posición (a partir de cero). Una vez la posición, podemos hacer lo que queramos con la información, como eliminarlo o mostrar.

```
int busqClaveLista(tLista *lista, int cmp(void *, void*), void *clave)
{
    // Contador de posiciones
    int pos = 0;

    // Mientras haya nodos en la lista
    while(*lista)
    {
        if(cmp((*lista)->info, clave) == 0)
        {
            return pos;
        }

        lista = &(*lista)->sig; // Avanza al siguiente nodo en la lista
        pos++; // Aumentamos el contador
    }

    // Si no encontró la posición
    return POS_INVALIDA;
}
```

También, podemos crear la función de buscar la última clave, es más de lo mismo:

```
int busqUltClaveLista(tLista *lista, int cmp(void *, void*), void *param)
{
    // Contador de posiciones
    int pos = 0, posUlt = POS_INVALIDA;

    // Mientras haya nodos en la lista
    while(*lista)
    {
        if(cmp((*lista)->info, param) == 0)
        {
            posUlt = pos;
        }

        lista = &(*lista)->sig; // Avanza al siguiente nodo en la lista
        pos++; // Aumentamos el contador
    }

    return posUlt;
}
```

También, si tenemos una lista ordenada, podemos realizar una búsqueda un poco más rápida. Sabemos hasta qué punto de la lista, ya no se encontrará la clave, por lo que podemos descartarlo.

```
int busqClaveListaOrd(tLista *lista, int cmp(void *, void *), void *clave)
{
    int pos = 0;

    // Recorre mientras haya elementos y la clave buscada sea mayor
    while (*lista && cmp((*lista)->info, clave) < 0)
    {
        lista = &(*lista)->sig;
        pos++;
    }

    // Si la lista no terminó, devuelve la posición
    // Si terminó (clave no encontrada), devuelve POS_INVALIDA
    return *lista ? pos : POS_INVALIDA;
}
```

Sacar Duplicados

Eliminamos todos los elementos duplicados (excluyendo el primero)

```
int outDupliLista(tLista *lista, int cmp(void*, void*))
{
    // Verificamos si la lista está vacía
    if (!*lista)
        return LISTA_VACIA;

    int cant = 0; // Contador de elementos eliminados

    // Recorremos la lista desde el principio
    while (*lista)
    {
        tNodo *anterior = (*lista); // Nodo anterior al que estamos revisando
        tNodo *revisar = (*lista)->sig; // Nodo siguiente que se va a comparar

        // Recorremos todos los nodos siguientes al nodo actual
        while (revisar)
        {
            // Si encontramos un duplicado
            if (cmp((*lista)->info, revisar->info) == 0)
            {
                // Desenlazamos el nodo duplicado
                anterior->sig = revisar->sig;

                // Liberamos la memoria del nodo duplicado
                free(revisar->info);
                free(revisar);

                // Avanzamos al siguiente nodo después del eliminado
                revisar = anterior->sig;

                // Contamos el duplicado eliminado
                cant++;
            }
            // Si no es duplicado, simplemente avanzamos en la lista
            else
            {
                anterior = revisar;
                revisar = revisar->sig;
            }
        }

        // Pasamos al siguiente nodo base para seguir comparando
        lista = &(*lista)->sig;
    }

    return cant;
}
```

Invertir Lista

Para invertir el sentido de una lista, debe utilizarse si o si recursividad:

```
// Función principal que invierte la lista
void invLista(tLista *lista)
{
    // Llama a la versión recursiva pasándole: nodo actual y anterior
    // Retorna el nuevo primer nodo (anteriormente el ultimo)
    *lista = _invLista(*lista, NULL);
}

// Función recursiva que invierte los enlaces de la lista
tNodo* _invLista(tNodo *act, tNodo *ant)
{
    // Si el nodo actual es NULL, llegamos al final de la lista
    if (!act)
        // En ese punto, el anterior (ant) es el nuevo primer nodo
        return ant;

    // Guardamos el siguiente nodo antes de romper el enlace
    tNodo *sig = act->sig;
    // Invertimos el enlace: el siguiente del actual será el anterior
    act->sig = ant;

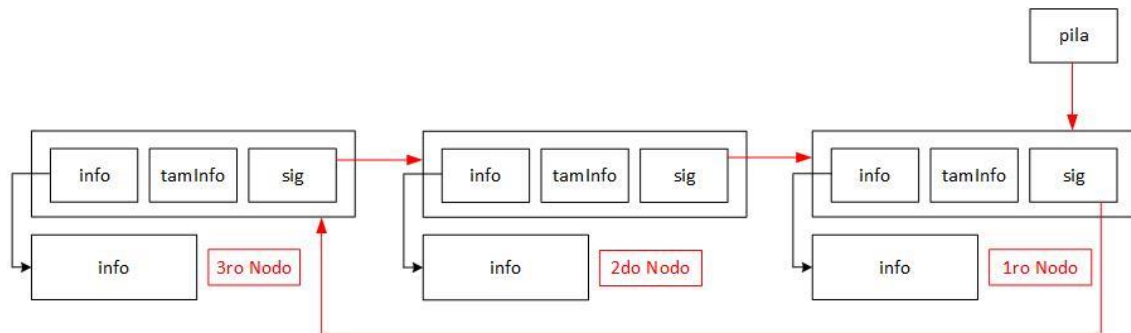
    // Llamada recursiva:
    // - Avanzamos al siguiente nodo (sig)
    // - El actual pasa a ser el anterior (act)
    return _invLista(sig, act);
}
```

Esta función recorre la lista de forma recursiva y va invirtiendo los punteros *sig* en cada nodo. Al llegar al final, retorna el nuevo primer nodo (el último original), y lo asigna a **lista*.

TDA Circular

TDA Pila Circular

Estrategia Problema



Representación gráfica del funcionamiento interno de una pila circular.

Una pila dinámica circular es una variante del TDA Pila donde se organizan formando un ciclo, es decir, el primero nodo insertado apunta al último (que es donde se harán interacciones). No tiene un fin como tal debido a su desplazamiento circular, como al igual se dice que tiene un principio, por eso no existe en teoría insertar ultimo o primero. Sin embargo, las interacciones de ver tope y sacar, se hacen al nodo que apunta el primer nodo insertado. El puntero de pila, se queda estático apuntando siempre al primer nodo insertado.

- Ver Tope -> Se obtiene el contenido del nodo que es apuntado por el primer nodo insertado.
- Sacar Pila -> Se quita la pila que es apuntado por el primer nodo insertado.
- Poner en Pila -> Se crea un nodo que apuntara al último nodo insertado, y el primer nodo insertado apuntara a este nuevo nodo.

Coincidencias con la Pila Aprendida

Pila circular y normal tienen muchas coincidencias, por lo que algunas no serán explicadas, tan solo busca sus capítulos. Vamos a listar sus coincidencias:

- [Estructura tPila](#)
- [Pila Llena](#)
- [Crear Pila](#)
- [Pila Vacía](#)

Poner en Pila Circular

```
int ponerPilaCir(tPila *pila, void *info, size_t tamInfo)
{
    tNodo *nue;

    // Reservamos memoria para el nuevo nodo
    if ( !(nue = malloc(sizeof(tNodo))) )
    {
        return ERROR_MALL;
    }

    // Reservamos memoria para la información
    if ( !(nue->info = malloc(tamInfo)) )
    {
        free(nue);
        return ERROR_MALL;
    }

    // Copiamos la información y su tamaño
    memcpy(nue->info, info, tamInfo);
    nue->tamInfo = tamInfo;

    // Si la pila tiene elementos
    if (*pila)
    {
        nue->sig = (*pila)->sig; // El nuevo nodo apunta al último insertado
        (*pila)->sig = nue; // El primero insertado, apunta al nuevo nodo
    }
    // Si la pila esta vacia
    else
    {
        nue->sig = nue; // Primer nodo en la pila: se apunta a sí mismo
        *pila = nue; // El puntero de pila apunta al único nodo
    }

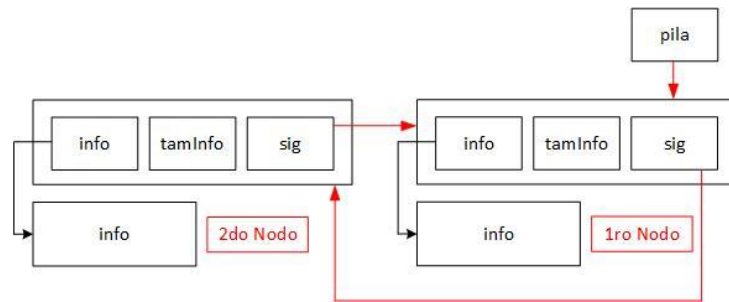
    return EXITO;
}
```

Se crea un nuevo y se copia su información como ya sabemos. Luego se crea una condicional para saber cómo insertarlo según si está vacío o no:

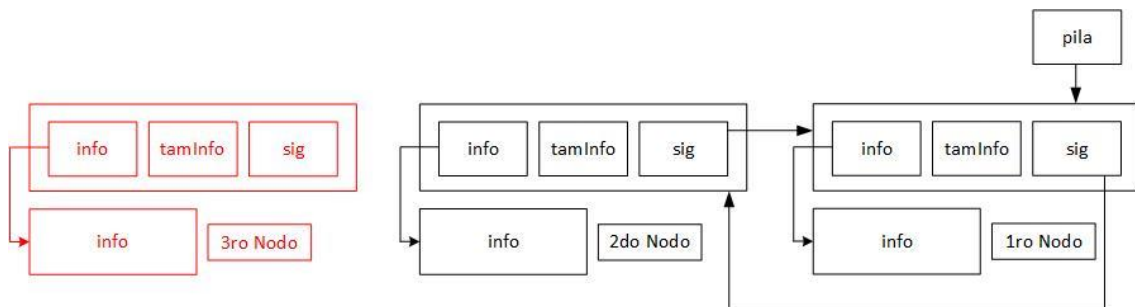
- La pila tiene elementos -> El nuevo nodo comienza apuntar hacia el ultimo elemento insertado, y el primer elemento insertado apunta hacia el nuevo nodo.
- La pila está vacía -> El nuevo nodo comienza apuntar así si mismo porque es el único elemento, y la pila comienza apuntar hacia este.

Vamos a darle una explicación gráfica:

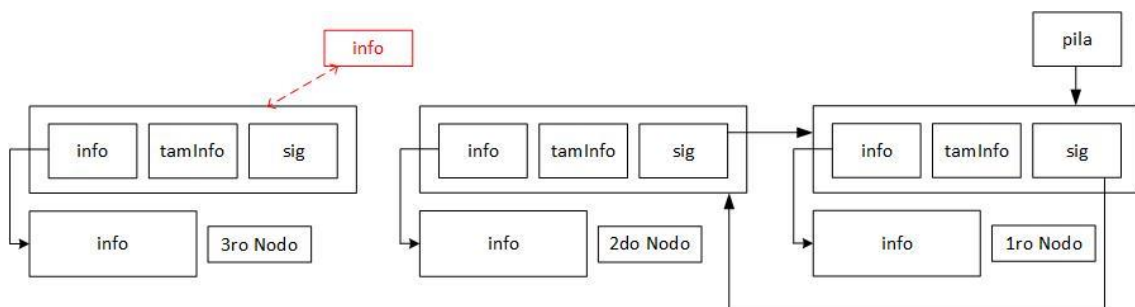
1. Se presenta la siguiente pila circular con sus 2 nodos. Se desea insertar un nuevo nodo a la pila.



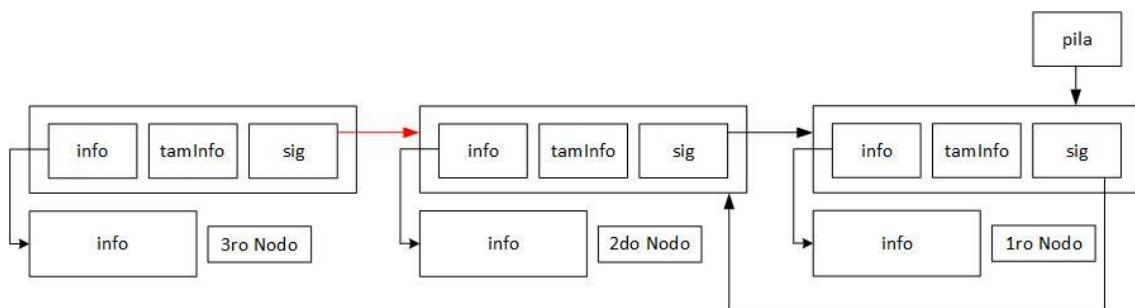
2. Se crea un nuevo nodo



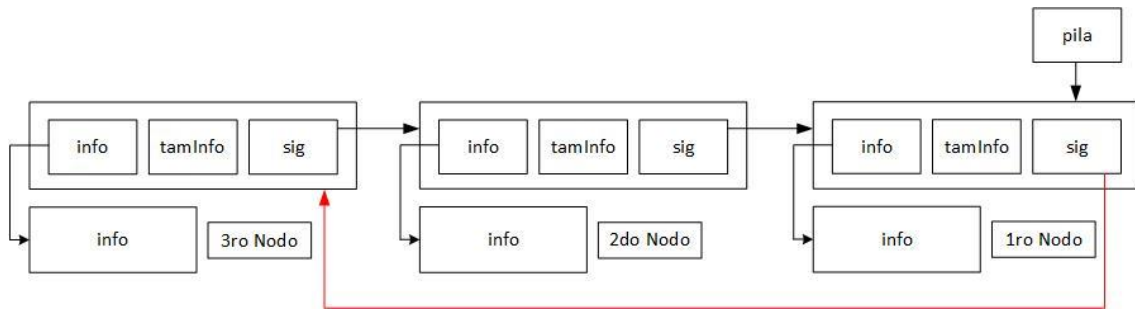
3. Se copia la información hacia la pila



4. El nuevo nodo comienza a apuntar hacia el ultimo elemento insertado, que coincide con el sig del nodo que apunta pila.



5. El primer nodo insertado que apunta la pila, comienza apuntar hacia el nuevo nodo insertado



Ver Tope

```
#define MIN(X,Y) X > Y ? Y : X // Obtiene el mínimo entre dos valores

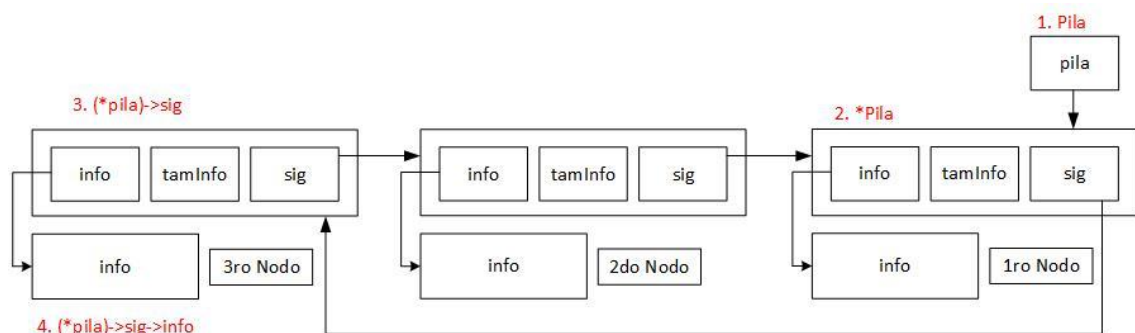
int verTopePilaCir(tPila *pila, void *info, size_t tamInfo)
{
    // Verifica si la pila está vacía (no hay nodos)
    if (!*pila)
    {
        return PILA_VACIA;
    }

    // Copia la información del último nodo (tope) hacia 'info'
    memcpy(info, (*pila)->sig->info, MIN(tamInfo, (*pila)->sig->tamInfo));

    return EXITO;
}
```

En este caso, se desea leer el contenido del tope, el tope es el último elemento insertado que apunta el nodo apuntado por pila. Sería el nodo que creamos en "Poner en Pila Circular".

Para realizar esta función, es casi idéntica la vista en Pila normal, lo único que cambie es: `< (*pila)->info >` por `< (*pila)->sig->info >`, esto hace referencia hacia el nodo que estamos buscando apuntar. Porque, en caso de no cambiarlo, apuntamos hacia el primer elemento insertado.



Sacar de Pila Circular

```
#define MIN(X,Y) X > Y ? Y : X // Obtiene el mínimo entre dos valores

int sacarDePilaCir(tPila *pila, void *info, size_t tamInfo)
{
    tNodo *aux;

    if (!*pila) // Si la pila está vacía, no hay nada para extraer
    {
        return PILA_VACIA;
    }

    aux = (*pila)->sig; // Pasamos al tope

    // Copiamos la información desde el nodo hacia los parámetros
    memcpy(info, aux->info, MIN(tamInfo, aux->tamInfo));

    // Si la pila tiene un solo nodo
    if (aux == *pila)
    {
        // Queda vacía la pila
        *pila = NULL;
    }
    // Si la pila tiene más de un nodo
    else
    {
        // El ultimo nodo, apuntara al siguiente del nodo eliminado
        (*pila)->sig = aux->sig;
    }

    // Liberamos la memoria del nodo y su información
    free(aux->info);
    free(aux);

    return EXITO;
}
```

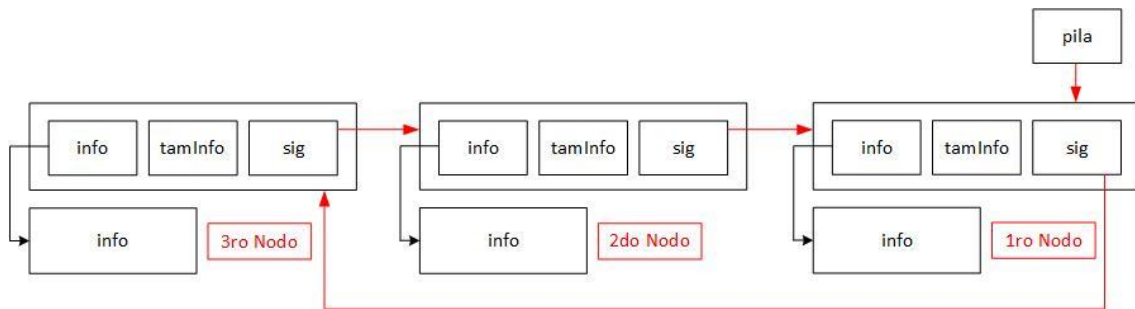
Vemos el tope de la pila, y una vez echo quitamos este nodo. Para eliminarlo se comprueba primero si hay solo un nodo en la pila, según el resultado:

- La pila está vacía -> Estamos quitando el único nodo que tiene la pila, por lo que queda vacía y debemos indicarlo apuntando a NULL.
- La pila tiene elementos -> El primer nodo insertado comienza a apuntar hacia el siguiente nodo del que vamos a eliminar.

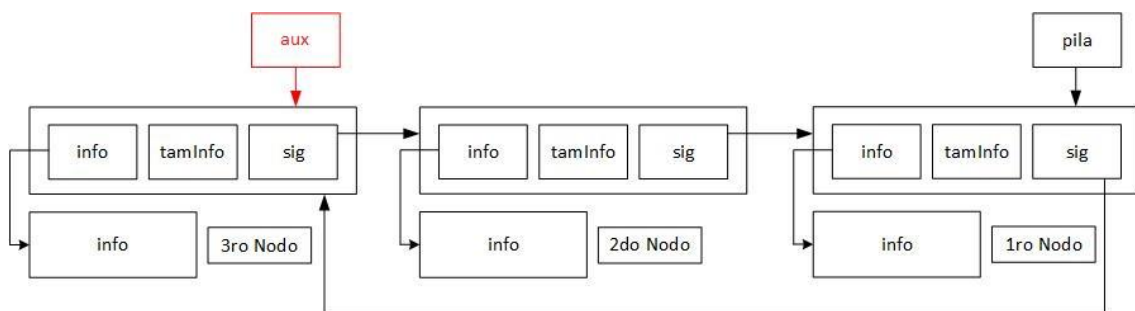
Una vez echo la condicional, liberamos la memoria del nodo.

Vamos a darle una explicación gráfica:

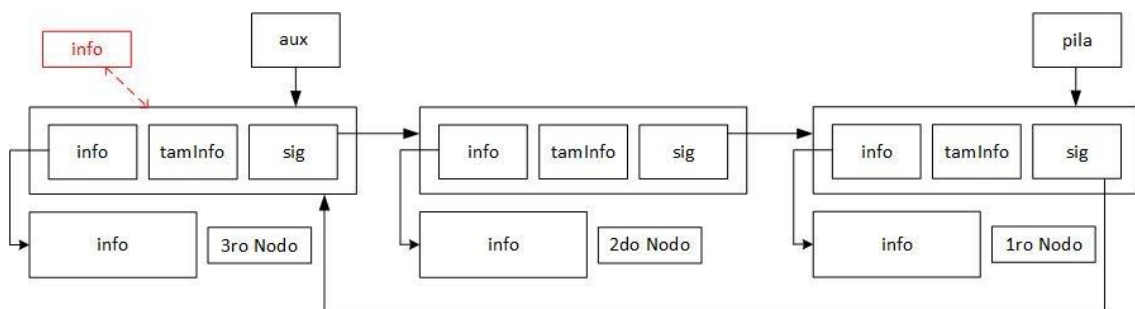
1. Se presenta la siguiente pila con sus 3 nodos. Se desea quitar el tope.



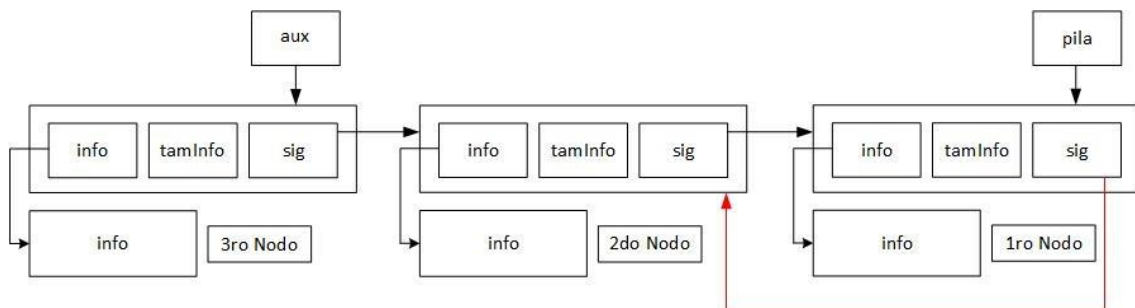
2. Se crea un auxiliar que apunta hacia el tope para no perder la dirección



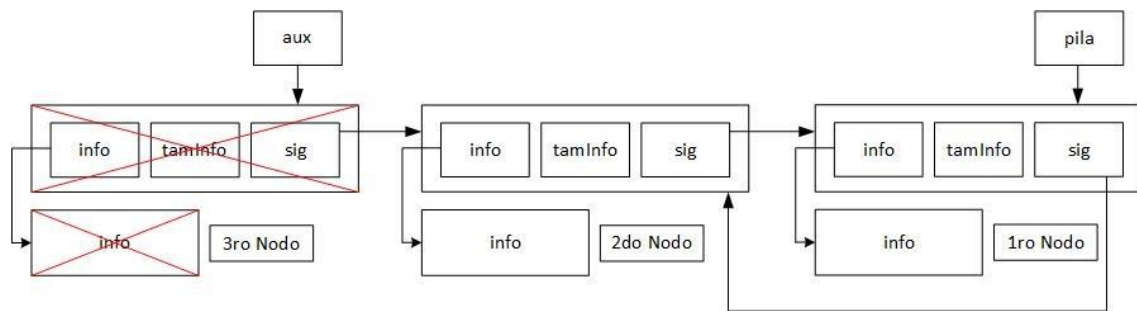
3. Se copia la información del tope hacia los parámetros



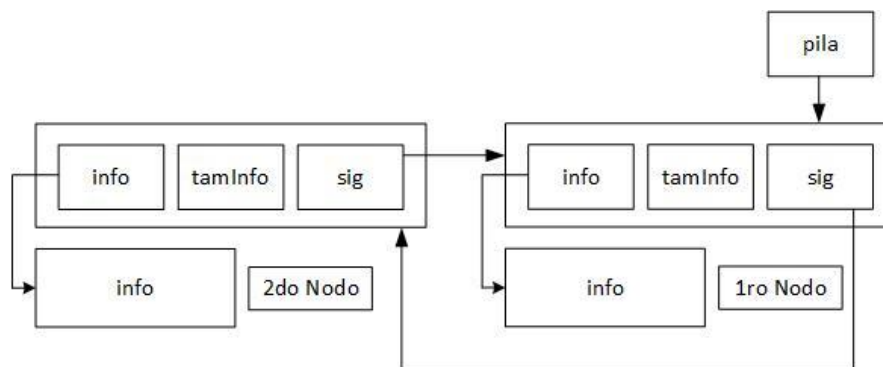
4. El primer nodo insertado, comienza a apuntar hacia el siguiente de aux. Salteando el ultimo nodo.



5. Se libera la información de memoria del nodo eliminado



6. Quedando como resultado final



Vaciar Pila Circular

Como tiene un funcionamiento circular el TDA, no hay un nodo que el puntero *sig* tenga contenido *null* para determinar el fin de la lista enlazada. Por lo que, se deberán hacer un funcionamiento diferente.

```
void vaciarPilaCir(tPila *pila)
{
    while (*pila) // Mientras haya nodos en la pila
    {
        tNodo *aux = (*pila)->sig; // aux apunta al último nodo

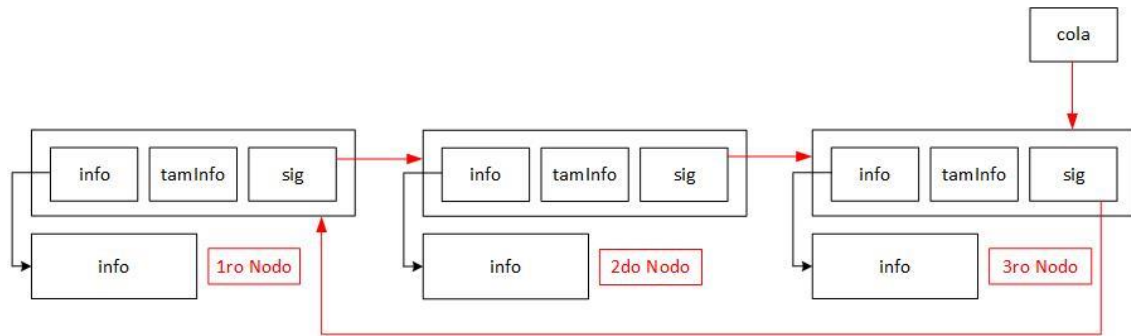
        if (aux == *pila) // Si solo queda un nodo
        {
            // La pila queda vacía
            *pila = NULL;
        }
        else
        {
            // Salteamos al siguiente nodo
            (*pila)->sig = aux->sig;
        }

        free(aux->info); // Liberamos la memoria de la información
        free(aux);       // Liberamos el nodo
    }
}
```

Se agrega una condicional if mostrado en el código para determinar si llegamos al fin (el primer nodo insertado). Para esto, se evalúa si la variable aux (que es la que se ira desplazando) es igual a la dirección que apunta la pila, si coinciden, quiere decir que se llegó al último nodo y la pila apunta a null. Si no coinciden, el primer nodo insertado para apuntar al siguiente nodo del que será eliminado (que apunta aux).

TDA Cola Circular

Estrategia Problema



Representación gráfica del funcionamiento interno de una cola circular.

TDA Cola Circular es una variante de la cola ya aprendida. Sin embargo, su funcionamiento se asemeja más a TDA Pila Circular que a una cola. Tiene un significado muy similar al anterior TDA ya visto.

Lo que los diferencia, es que el puntero cola se va a ir desplazamiento mientras se insertan nodos, no se queda estático en el primer nodo insertado. Luego, el funcionamiento es casi igual. Incluso, su estructura cambia, esta vez se tiene solo un puntero y no dos.

Coincidencias con Pila Circular

Cola circular y pila tienen muchas coincidencias, por lo que algunas no serán explicadas, tan solo busca sus capítulos. Vamos a listar sus coincidencias:

- Estructura tPilaCir = Estructura tColaCir
- Crear Pila Cir = Crear Cola Cir
- Pila Cir Llena = Cola Cir Llena
- Pila Cir Vacía = Cola Cir Vacía
- [Vaciar Pila Cir](#) = Vaciar Cola Cir
- [Ver Tope Pila Cir](#) = Ver Primero Cola Cir
- [Sacar de Pila Cir](#) = Sacar de Cola Cir

Poner en Cola Circular

```
int ponerEnCola(tCola *cola, void *info, size_t tamInfo)
{
    tNodo *nue;

    // Reservamos memoria para el nuevo nodo
    if ( !(nue = malloc(sizeof(tNodo))) )
    {
        return ERROR_MALL;
    }

    // Reservamos memoria para la información del nuevo nodo
    if ( !(nue->info = malloc(tamInfo)) )
    {
        free(nue);
        return ERROR_MALL;
    }

    // Copiamos la información al nodo nuevo
    memcpy(nue->info, info, tamInfo);
    nue->tamInfo = tamInfo;

    // Si la cola tiene elementos
    if (*cola)
    {
        // El nuevo apunta al primer insertado
        nue->sig = (*cola)->sig;

        // El último insertado apunta al nuevo nodo
        (*cola)->sig = nue;
    }
    // Si la cola está vacía
    else
    {
        // El nuevo nodo se apunta hacia si mismo
        nue->sig = nue;
    }

    // El nuevo nodo se convierte en el último
    *cola = nue;

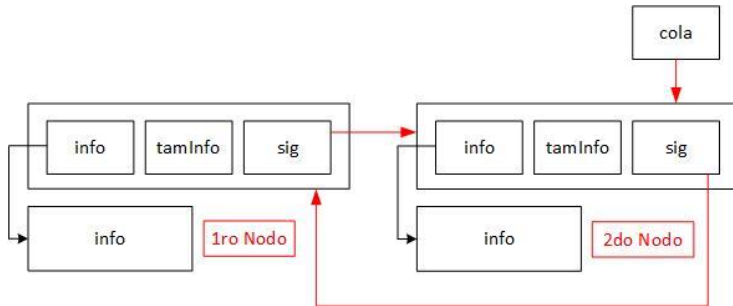
    return EXITO;
}
```

El código a diferencia de Pila Circular, es casi idéntico. De hecho, no se elimina nada, tan solo se mueve una operación de lugar. `< *cola = nue; >` pasa de estar dentro de la condicional if, a estar fuera de la misma, esa es la única diferencia.

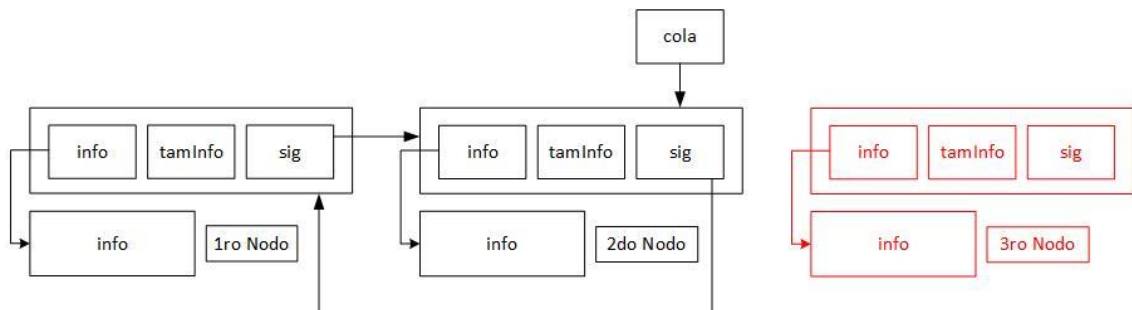
`< *cola = nue; >` Esto se coloca fuera de la condicional, para que el puntero de cola se desplace hacia el nuevo nodo, cada vez que se insertar alguno.

Vamos a darle una explicación grafica

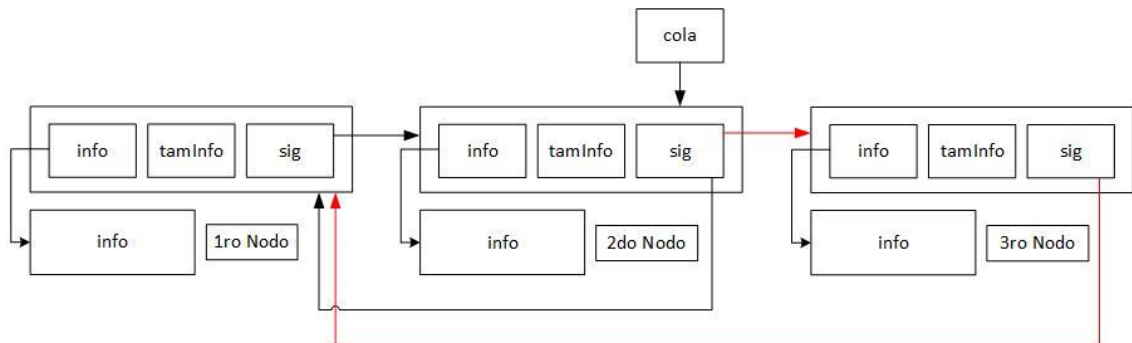
1. Se presenta la siguiente cola circular, se desea insertar un tercer nodo



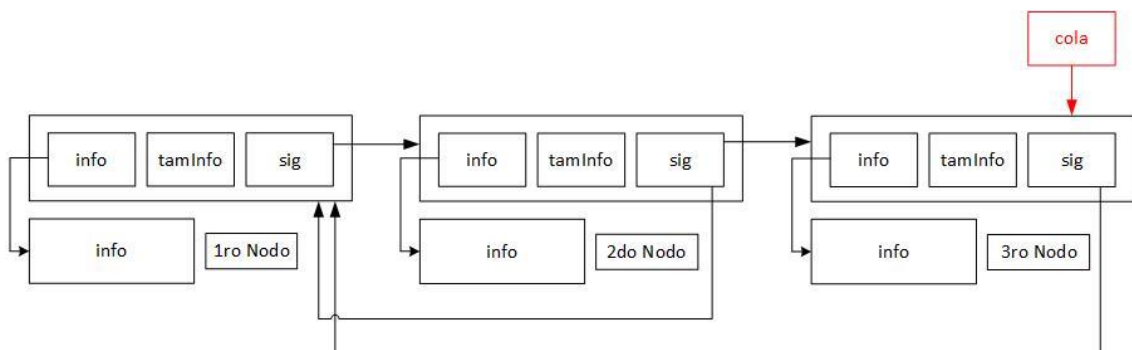
2. Se crea un nuevo nodo y se copia la información de los parámetros al nodo.



3. El nuevo nodo apunta al primer nodo insertado, que coincide con el *sig* del nodo que es apuntado por la cola. También, el nodo apuntado por la cola, comienza apuntar al nuevo nodo.



4. La cola comienza apuntar hacia el nuevo nodo.



TDA Lista Doble Mente Enlazada

Problemática y Estrategia



Representación gráfica del funcionamiento interno de una lista doble mente enlazada. Por cuestiones de plasticidad, a partir de ahora no se representará el contenido de info.

La ventaja de agregar un puntero que apunte al nodo anterior es que permite desplazarse tanto hacia adelante como hacia atrás dentro de la lista. Esta característica otorga una gran flexibilidad, ya que el puntero principal de la lista no necesita estar necesariamente en el primer nodo: **puede ubicarse en cualquier posición**, como el último nodo o incluso en uno intermedio.

El objetivo de esta estructura es **mejorar la eficiencia** en determinadas operaciones que, en una lista simplemente enlazada, serían más costosas. Por ejemplo, **retroceder** desde una posición actual, eliminar nodos sin recorrer desde el inicio, o insertar con mayor facilidad en ambas direcciones, son tareas que se optimizan gracias al doble enlace de los nodos.

Todas las funciones vistas en TDA Lista Simplemente Enlazada, pueden ser implementadas aca. Por eso mismo, vamos a ver funciones repetidas y excluir bastantes a diferencia de lista simplemente enlazada.

Estructura

```
typedef struct sNodo
{
    void *info; // Dirección malloc donde guarda la información
    size_t tamInfo; // Tamaño de la Información
    struct sNodo *sig; // Dirección nodo siguiente
    struct sNodo *ant; // Dirección nodo anterior
} tNodo;

typedef tNodo *tLista;
```

El sNodo es como un alias que se pone (sNodo = tNodo), es necesario porque después de ser creado, recién ahí se lo llama como tNodo, entonces esto es necesario para poder auto-referenciarse con el campo **sig* y **ant*. El typedef es otro sinónimo que se le pone para tLista.

Crear Lista

Mismo método que el resto, comienza a apuntar a null.

```
void crearLista(tLista *lista)
{
    *lista = NULL;
}
```

Cuando se crea un puntero, este apuntara a basura si no le asignamos nada. Por lo que, debemos hacer que apunte a la nada (NULL) indicando que está vacío o listo para usarse.

Desplazamiento

El patrón aprendido de `< lista = &(*lista)->sig >` no se repetirá en este caso. Esto debido a que ahora el puntero *lista* del main o de otro contexto fuera del código interno, puede apuntar a cualquier nodo de la lista, debemos estar retocando el apuntador original, por lo que no debemos perder la referencia. En este caso, se utiliza casi el mismo concepto, pero con una variable auxiliar que comience apuntando al mismo lugar que lista:

```
tNodo *aux = *lista;
while (aux != NULL)
    aux = aux->sig;
```

```
tNodo *aux = *lista;
while (aux != NULL)
    aux = aux->ant;
```

```
tNodo *aux = *lista;
while (aux != NULL && pos > 0)
{
    aux = aux->sig;
    pos--;
```

Tenemos 3 formas de desplazarnos que quizás ya son reconocidas por el usuario, por lo que no se dará mucha explicación:

- Desplazarnos al último elemento
- Desplazarnos al primer elemento
- Desplazarnos en una posición determinada.

Operaciones al Principio

Insertar al Principio

```
int insPrinLista(tLista *lista, void *info, size_t tamInfo)
{
    tNodo *act = *lista, *nue;

    // Si la lista no está vacía
    if (act)
        // Nos posicionamos al principio
        while (act->ant)
            act = act->ant;

    if (!(nue = malloc(sizeof(tNodo))))
    {
        return ERROR_MALLOC;
    }

    if (!(nue->info = malloc(tamInfo)))
    {
        free(nue);
        return ERROR_MALLOC;
    }

    // Copio los datos de los parámetros
    memcpy(nue->info, info, tamInfo);
    nue->tamInfo = tamInfo;

    // Enlazo el nuevo nodo al principio de la lista
    nue->sig = act;
    nue->ant = NULL;

    // Si la lista no está vacía
    if (act)
    {
        // Enlazamos el nue como el anterior del que ya estaba primero
        act->ant = nue;
    }

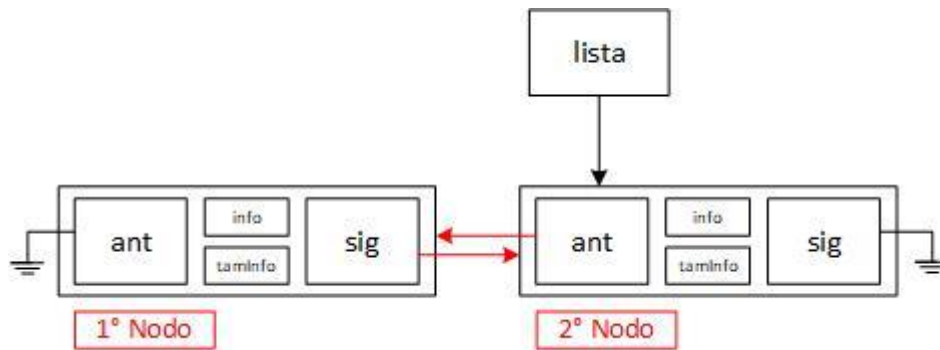
    // Actualizo el puntero de la lista para que apunte al nuevo nodo
    *lista = nue;

    return EXITO;
}
```

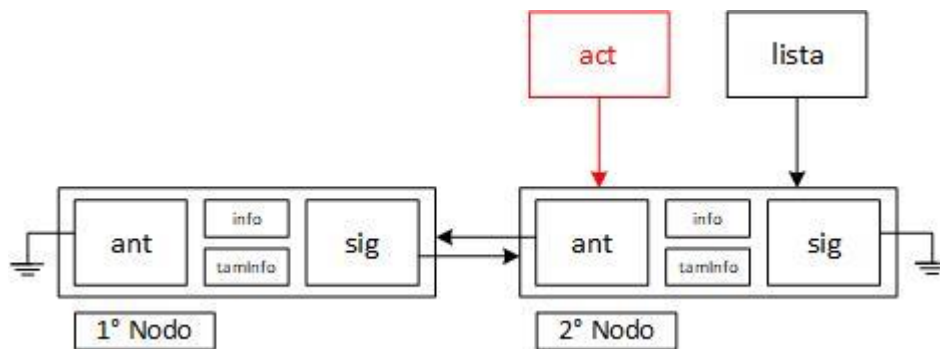
Como ya sabemos, el puntero de la lista puede estar posicionado en cualquier lado, por lo que debemos asegurar ir al principio, luego de comprobar que la lista no está vacía claro. Para `< nue->sig = act; >` no importa si la lista está vacía, siendo que, si no lo pone en NULL, por lo q no es necesario una condicional.

Vamos a darle una explicación gráfica:

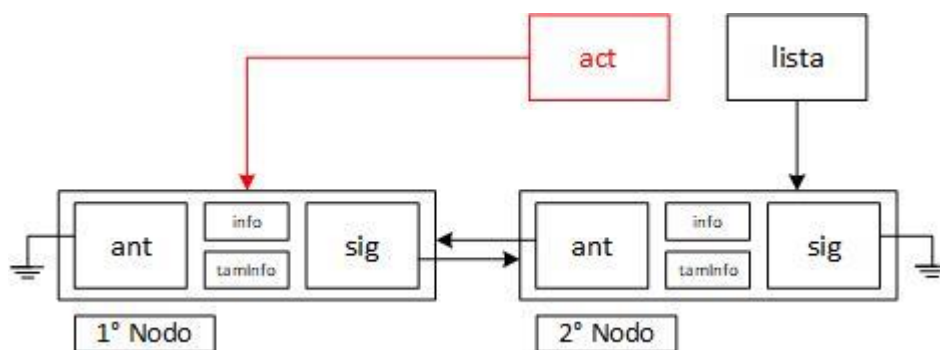
1. Se presenta la siguiente lista doblemente enlazada, con el puntero de lista posicionado en el 2° nodo insertado.



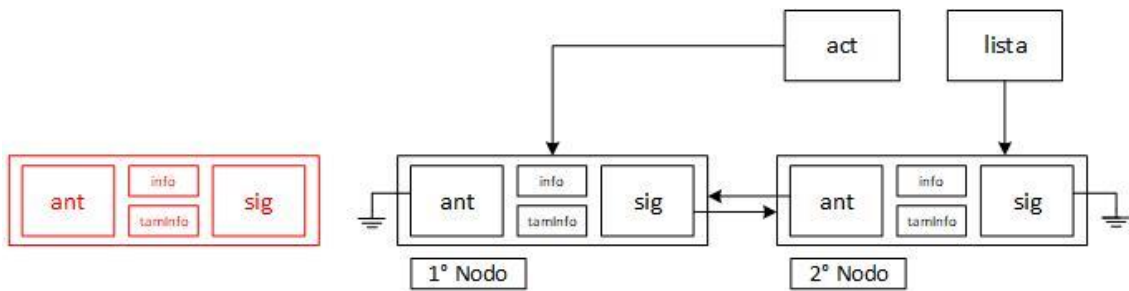
2. Se crea una variable auxiliar que apunte al mismo lugar que lista.



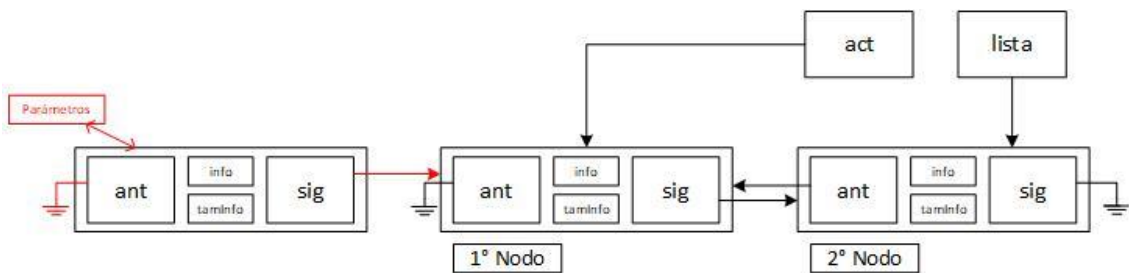
3. Se desplaza el puntero auxiliar al inicio de la lista.



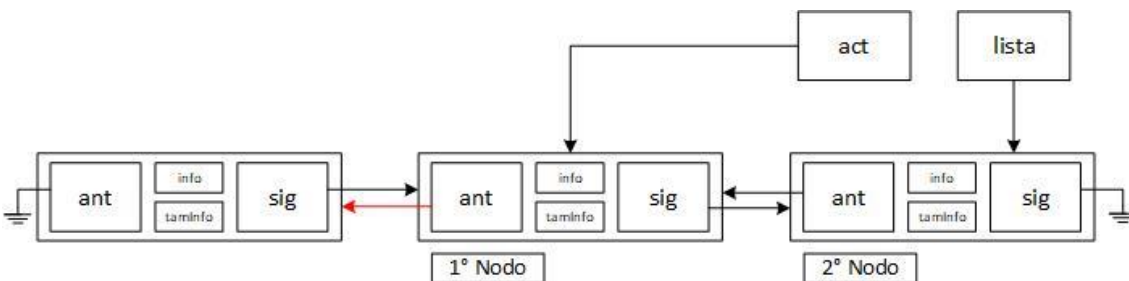
4. Se crea un nuevo nodo



5. Se copia la información de los parámetros al nuevo nodo y se corrige los punteros *ant* y *sig*



6. Se corrige el puntero *ant* del anterior primero nodo



7. La lista comenzara apuntar hacia el nuevo nodo insertado



Ver Principio

```
int verPrinLista(tLista *lista, void *info, size_t tamInfo)
{
    tNodo *act = *lista;

    // Si la lista está vacía, no hay nada que ver
    if (!act)
    {
        return LISTA_VACIA;
    }

    // Ir al principio de la lista
    while (act->ant)
    {
        act = act->ant;
    }

    // Copiar la información del nodo a los parámetros
    memcpy(info, act->info, MIN(tamInfo, act->tamInfo));

    return EXITO;
}
```

Como ya sabemos, el puntero de la lista puede estar posicionado en cualquier lado, por lo que debemos asegurar ir al principio, luego de comprobar que la lista no está vacía claro. Una vez posicionados, copiamos la información. No tiene mucha más lógica.

Sacar del Principio

```
int outPrinLista(tLista *lista, void *info, size_t tamInfo)
{
    tNodo *act = *lista;

    // Si la lista está vacía, no hay nada para quitar
    if (!act)
    {
        return LISTA_VACIA;
    }

    // Ir al principio de la lista
    while (act->ant)
    {
        act = act->ant;
    }

    memcpy(info, act->info, MIN(tamInfo, act->tamInfo));

    // Avanzar al siguiente nodo para actualizar la lista
    *lista = act->sig;

    // Si existe un siguiente nodo, su anterior ya no será el
    // nodo eliminado, será null porque ya no existirá
    if (act->sig)
    {
        act->sig->ant = NULL;
    }

    // Liberar la memoria del nodo eliminado
    free(act->info);
    free(act);

    return EXITO;
}
```

Como ya sabemos, el puntero de la lista puede estar posicionado en cualquier lado, por lo que debemos asegurar ir al principio, luego de comprobar que la lista no está vacía claro. Una vez ahí, copiamos la información a los parámetros. Luego, corregimos los punteros de la lista y liberamos memoria.

Explicación gráfica:

1. Se presenta la siguiente lista doblemente enlazada, se desea quitar el primer nodo



2. Creamos un puntero auxiliar para lista



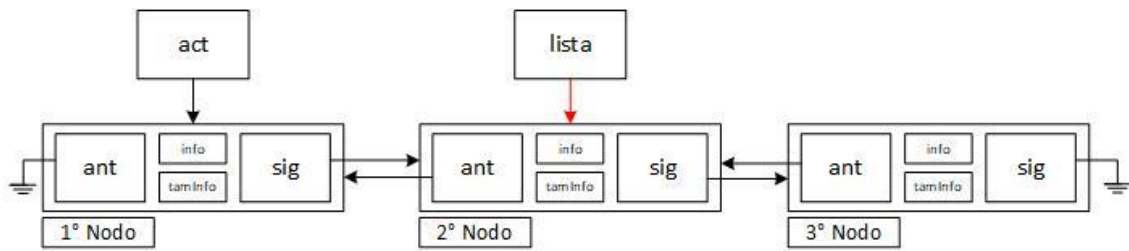
3. Desplazamos el puntero auxiliar al inicio de la lista



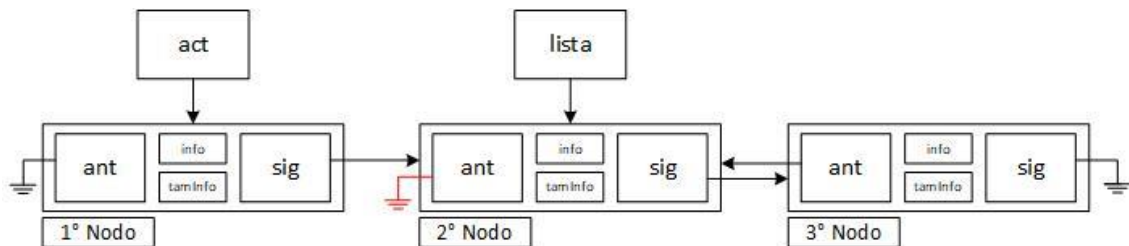
4. Copiamos el contenido del nodo a los parámetros



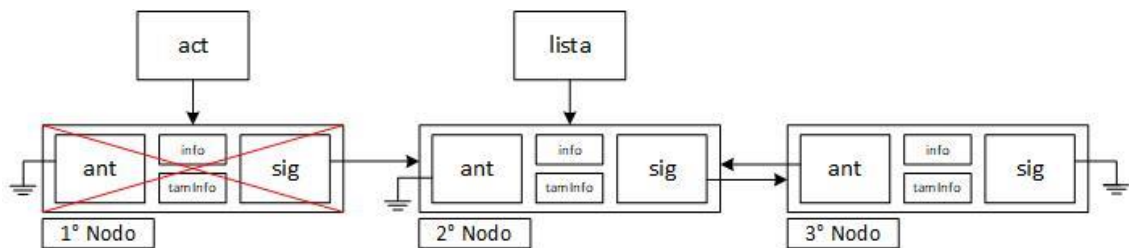
5. Desplazamos el puntero *lista*, al nodo siguiente del que vamos a eliminar



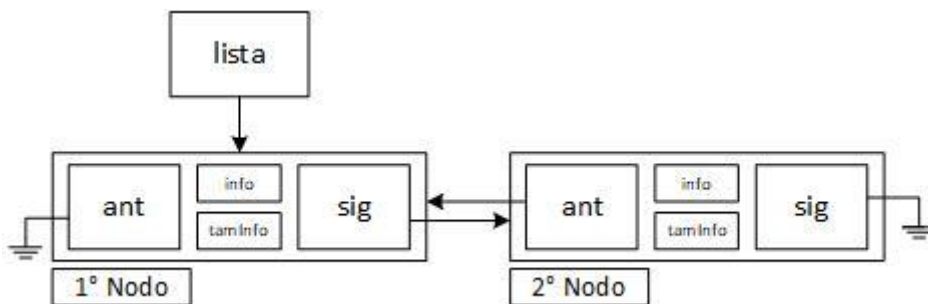
6. El nodo siguiente del que vamos a eliminar, debe apuntar a null



7. Eliminamos el primer nodo apuntado por *act*



8. Quedando como resultado final:



Operaciones al Final

Las operaciones al final, son casi lo mismo que las operaciones al inicio, solo que cambian algunos pocos datos. Por lo que no daremos una explicación grafica.

Insertar al Final

```
int insFinLista(tLista *lista, void *info, size_t tamInfo)
{
    tNodo *act = *lista, *nue;

    // Si la lista no está vacía
    if (act)
    {
        // Nos posicionamos al final
        while (act->sig)
            act = act->sig;
    }

    if (!(nue = malloc(sizeof(tNodo))))
    {
        return ERROR_MALLOC;
    }

    if (!(nue->info = malloc(tamInfo)))
    {
        free(nue);
        return ERROR_MALLOC;
    }

    // Copio los datos de los parámetros
    memcpy(nue->info, info, tamInfo);
    nue->tamInfo = tamInfo;

    // Enlazo el nuevo nodo al final de la lista
    nue->sig = NULL;
    nue->ant = act;

    // Si la lista no está vacía
    if (act)
    {
        // Enlazamos el nue como el siguiente del que ya estaba ultimo
        act->sig = nue;
    }

    // Actualizo el puntero de la lista para que apunte al nuevo nodo
    *lista = nue;

    return EXITO;
}
```

Como se puede ver, lo único q cambia respecto a insertar al inicio, es intercambiar los *sig* por *ult*, el resto es igual.

Ver Final

```
int verFinLista(tLista *lista, void *info, size_t tamInfo)
{
    tNodo *act = *lista;

    // Si la lista está vacía, no hay nada que ver
    if (!act)
    {
        return LISTA_VACIA;
    }

    // Ir al final de la lista
    while (act->sig)
    {
        act = act->sig;
    }

    // Copiar la información del nodo, respetando el menor tamaño
    memcpy(info, act->info, MIN(tamInfo, act->tamInfo));

    return EXITO;
}
```

Como se puede ver, lo único q cambia respecto a ver al inicio, es intercambiar los *sig* por *ult* del bucle para desplazarnos, el resto es igual.

Sacar del Final

```
int outFinLista(tLista *lista, void *info, size_t tamInfo)
{
    tNodo *act = *lista;

    // Si la lista está vacía, no hay nada para quitar
    if (!act)
        return LISTA_VACIA;

    // Ir al último nodo
    while(act->sig)
        act = act->sig;

    memcpy(info, act->info, MIN(tamInfo, act->tamInfo));

    // Avanzar al anterior nodo para actualizar la lista
    *lista = act->ant;

    // Si había un nodo anterior, su sig debe ser NULL
    if (act->ant)
        act->ant->sig = NULL;

    // Liberar la memoria del nodo a eliminar
    free(act->info);
    free(act);

    return EXITO;
}
```

Como se puede ver, lo único q cambia respecto a insertar al inicio, es intercambiar los *sig* por *ult*, el resto es igual.

Operaciones en Posiciones Determinadas

Sacar Nodo en Posición Determinada

```
int outPosLista(tLista *lista, void *info, size_t tamInfo, size_t pos)
{
    tNodo *act = *lista;

    // Ir al inicio de la lista
    while(act->ant)
        act = act->ant;

    // Recorremos hasta la posición deseada o hasta que la lista termine
    while(act && pos > 0)
    {
        act = act->sig;
        pos--;
    }

    // Si no se llega a la posición deseada, la posición no es válida
    if (!act || pos)
        return POSICION_INVALIDA;

    // Copiamos la información del nodo eliminado
    memcpy(info, act->info, MIN(tamInfo, act->tamInfo));

    // Si existe un nodo anterior, lo reconectamos con el siguiente
    if(act->ant)
        act->ant->sig = act->sig;

    // Si existe un nodo siguiente, lo reconectamos con el anterior
    if(act->sig)
        act->sig->ant = act->ant;

    // Si el nodo que eliminamos era justo el que apuntaba *lista,
    // actualizamos *lista para que siga apuntando a algún nodo válido
    if (*lista == act)
        *lista = act->sig ? act->sig : act->ant;

    // Liberamos la memoria del nodo
    free(act->info);
    free(act);

    return EXIT0;
}
```

Nos posicionamos al inicio de la lista, para comenzar a contar las posiciones. Recorre la lista hasta llegar a la posición deseada, si no existe retorna error. Cuando encontremos la posición, copia la información del nodo a los parámetros. Reconectamos los nodos vecinos del nodo que vamos a eliminar. En caso de que justo el nodo que eliminamos es donde está la cabecera lista, la movemos de lugar. Por último, liberamos la memoria.

Vaciar Lista

```
void vaciarLista(tLista *lista)
{
    tNodo *act = *lista, *aux;

    // Si la lista ya está vacía, no hay nada que hacer
    if (!act)
        return;

    // Asegurarse de estar al principio
    while (act->ant)
        act = act->ant;

    // Liberar cada nodo desde el principio hacia el final
    while (act)
    {
        aux = act->sig;    // Guardar el siguiente nodo
        free(act->info);   // Liberar la información
        free(act);        // Liberar el nodo
        act = aux;        // Pasar al siguiente
    }

    // Dejar el puntero de la lista en NULL
    *lista = NULL;
}
```

Hay varias formas de resolver esta operación. Podemos, por ejemplo: guardar la posición donde estamos, comenzar a recorrer a una dirección y liberar memoria, luego volver donde estábamos y comenzar a recorrer a la otra dirección.

Yo opte por una solución menos eficiente que la anterior explicada, pero más sencilla de aplicar y aprender. Nos posicionamos al inicio de la lista, y comenzamos a recorrer hacia la izquierda mientras liberamos.

Map, Filter, Reduce y Búsqueda

Map (recorrer)

Ahora que tenemos un puntero extra en los nodos, podemos recorrer de derecha a izquierda y viceversa. No se dará mucha explicación.

Esta función permite recorrer una lista, aplicarle una acción a cada nodo (como mostrar o sumar un numero) y aparte darle una dirección de memoria como parámetro. Ya fue vista en Tópicos de Programación, pero será adaptado para nodos, aparte se le dará un parámetro extra **param*.

```
void mapListaIzqDer(tLista *lista, void accion(void*, void*), void *param)
{
    tNodo *act = *lista;

    // Si la lista está vacía
    if(!act)
        return;

    // Nos posicionamos al principio de la lista
    while(act->ant)
        act = act->ant;

    // Comenzamos a recorrer de inicio a fin de la lista
    while(act)
    {
        accion(act->info, param);
        act = act->sig;
    }
}
```

```
void mapListaDerIzq(tLista *lista, void accion(void*, void*), void *param)
{
    tNodo *act = *lista;

    // Si la lista está vacía
    if(!act)
        return;

    // Nos posicionamos al final de la lista
    while(act->sig)
        act = act->sig;

    // Comenzamos a recorrer de fin a inicio de la lista
    while(act)
    {
        accion(act->info, param);
        act = act->ant;
    }
}
```


Reduce

Se aplica una función acumulativa o de reducción a los nodos de la lista, de manera que se recorre la lista para ser reducida a un solo valor inicial. No se eliminan los nodos o el contenido, solo se obtiene y se interactúa para ser acumulado a un valor inicial.

```
void reduceLista(tLista *lista, void redu(void*, void*, void*), void *acum, void *param)
{
    tNodo *act = *lista;

    // Si la lista está vacía
    if(!act)
        return;

    // Nos posicionamos al principio de la lista
    while(act->ant)
    {
        act = act->ant;
    }

    // Comenzamos a recorrer de inicio a fin de la lista
    while(act)
    {
        // Aplica la función 'redu' al dato del nodo actual
        redu(act->info, acum, param);
        act = act->sig;
    }
}
```

La función redu será ejecutada en cada nodo de la lista, recibe 3 parámetros:

1. **Información* -> Contenido del nodo
2. **result* -> Valor inicial para el acumulador
3. **param* -> Mismo concepto visto anteriormente con *map*

El valor inicial debe ser definido de antemano de ejecutar la función reduce.

Filter

Filter es una función que recorre la lista y a cada nodo se le ejecuta una función que devuelve true o false. Según el resultado de la función al nodo, este se eliminará o se lo dejará en la lista (false = eliminado). De ahí el nombre de "filtro".

```
void filterLista(tLista *lista, int filtro(void *, void *), void *param)
{
    tNodo *act = *lista, *elim;

    if (!act) // Lista vacía, nada que hacer
        return;

    // Nos aseguramos de comenzar desde el primer nodo
    while (act->ant)
        act = act->ant;

    while (act) // Mientras haya nodos
    {
        // Si el nodo cumple con el filtro, simplemente avanzamos
        if (filtro(act->info, param))
        {
            act = act->sig;
        }
        // Si no lo cumple, lo eliminamos
        else
        {
            // Guardamos el nodo a eliminar y avanzamos antes de eliminarlo
            elim = act;
            act = act->sig;

            // Reconectamos los vecinos del nodo eliminado
            if (elim->ant)
                elim->ant->sig = elim->sig;

            if (elim->sig)
                elim->sig->ant = elim->ant;

            // Si eliminamos el nodo apuntado por la cabecera, actualizamos
            if (*lista == elim)
                *lista = elim->sig ? elim->sig : elim->ant;

            // Liberamos memoria
            free(elim->info);
            free(elim);
        }
    }
}
```

Se asegura de comenzar desde el primer nodo de la lista. Luego, recorre cada nodo y evalúa su contenido con la función filtro. Si el nodo **no cumple** con la condición, se elimina liberando su memoria y reconectando los nodos vecinos. Además, si el nodo eliminado era el que apuntaba la cabecera de la lista (*lista), se actualiza para que siga apuntando a un nodo válido.

Búsqueda por Clave

```
int busqClaveLista(tLista *lista, int cmp(void*, void*), void *clave)
{
    int pos = 0;
    tNodo *act = *lista;

    // Si la lista se encuentra vacía
    if(!act)
        return POS_INVALIDA;

    // Nos desplazamos al inicio
    while(act->ant)
        act = act->ant;

    while(act)
    {
        if(cmp(act->info, clave) == 0)
            return pos;

        act = act->sig; // Avanza al siguiente nodo
        pos++; // Aumentar contador
    }

    return POS_INVALIDA;
}
```

Aunque **OJO**: Si por ejemplo estas ejecutando una función de eliminar duplicados y estas usando esta función, es un error. Porque siempre te lleva al principio. Como solución, es recomendado crear 2 funciones de búsqueda:

- Búsqueda desde el inicio
- Búsqueda desde la posición actual (oculta para el usuario)

La primera función ya la tenemos, por lo que nos falta la otra:

```
int _busqActClaveLista(tLista *lista, int cmp(void*, void*), void *clave)
{
    int pos = 0;
    tNodo *act = *lista;

    // Si la lista se encuentra vacía
    if(!act)
        return POS_INVALIDA;

    while(act)
    {
        if(cmp(act->info, clave) == 0)
            return pos;

        act = act->sig; // Avanza al siguiente nodo
        pos++; // Aumentar contador
    }

    return POS_INVALIDA;
}
```

Elimina por Clave con Lista Ordenada

También, si nos encontramos con una lista ya ordenada, podemos optimizar muchísimo el proceso de insertar, siendo que no nos debemos desplazar al final o inicio, puede encontrarse ya en una posición intermedia:

```
int elimClaveListaOrd(tLista *lista, void *info, size_t tamInfo, int
cmp(void*, void*), void *clave)
{
    tNodo *act = *lista;

    // Si la lista esta vacía
    if(!act)
        return LISTA_VACIA;

    // si clave de pos actual < clave => avanzamos
    while(act->sig && cmp(act->info, clave) < 0)
        act = act->sig;

    // si clave de pos actual > clave => retrocedemos
    while(act->ant && cmp(act->info, clave) > 0)
        act = act->ant;

    // Si el valor posición alcanzada no coincide, no existe la clave
    if(cmp(act->info, clave) != 0)
        return POSICION_INVALIDA;

    // Si existe un nodo anterior, lo reconectamos con el siguiente
    if(act->ant)
        act->ant->sig = act->sig;

    // Si existe un nodo siguiente, lo reconectamos con el anterior
    if(act->sig)
        act->sig->ant = act->ant;

    // Si el nodo que eliminamos era justo el que apuntaba *lista,
    // actualizamos *lista para que siga apuntando a algún nodo válido
    if (*lista == act)
        *lista = act->sig ? act->sig : act->ant;

    // Copiamos la información del nodo a los parámetros
    memcpy(info, act->info, MIN(tamInfo, act->tamInfo));

    // Liberamos la memoria del contenido del nodo y el nodo en sí
    free(act->info);
    free(act);

    return EXIT0;
}
```

Sacar Duplicados

Esta función puede ser un poco compleja a pesar que se asemeja a la lista simple.

```
void outDupliLista(tLista *lista, int cmp(void *, void*))
{
    // Variable para obtener la posición o si fue invalido
    int pos;
    tNodo *act = *lista;

    // Si la lista se encuentra vacía
    if(!act)
        return;

    // Nos desplazamos al inicio
    while(act->ant)
    {
        act = act->ant;
    }

    // Recorremos hasta el ante ultimo nodo
    while(act->sig)
    {
        // Buscamos si hay otro elemento duplicamos
        // Comenzamos a buscar desde la posición siguiente
        if((pos = _busqActClaveLista(&act->sig, cmp, act->info)) != POS_INV)
        {
            // Comenzar a sacar desde posición siguiente
            outPosLista(&act->sig, NULL, 0, pos);
        }
        else
        {
            // Avanzamos la lista
            act = act->sig;
        }
    }
}
```

< **busqActClaveLista** > Recibe como argumento una *lista*, por lo que no podemos enviar *act* por ser **lista*, así que como solución debemos enviarlo con **&**.

También, prestar atención que no estamos enviados la función de búsqueda desde el inicio, si no desde la posición actual. Porque si no borraría todos los elementos excepto el primero (si se tiene varios elementos distintos), por lo que sería un error.

Insertar Ordenado

```
int insOrdLista(tLista *lista, void *info, size_t tamInfo, int
cmp(void*,void*))
{
    tNodo *act = *lista, *nue;

    // Si la lista está vacía, se inserta al principio directamente
    if(!act)
    {
        insPrinLista(lista, info, tamInfo);
        return EXITO;
    }

    // Si info de pos actual < info => Avanzamos
    while(act->sig && cmp(act->info, info) < 0)
        act = act->sig;

    // Si info de pos actual > info => Retrocedemos
    while(act->ant && cmp(act->info, info) > 0)
        act = act->ant;

    // Se crea el nuevo con su información básica
    if (!(nue = malloc(sizeof(tNodo))))
        return ERROR_MALLOCC;

    if (!(nue->info = malloc(tamInfo)))
    {
        free(nue);
        return ERROR_MALLOCC;
    }

    memcpy(nue->info, info, tamInfo);
    nue->tamInfo = tamInfo;

    // Si el nuevo elemento es menor que el actual, se inserta antes
    if(cmp(act->info, info) > 0)
    {
        nue->sig = act;        // El sig del nuevo es el actual
        nue->ant = act->ant;    // El ant del nuevo es el anterior del actual
        // Si existe el nodo anterior, se conecta al nuevo
        if(act->ant)
            act->ant->sig = nue; // El sig del nodo anterior apunta al nuevo
        act->ant = nue;        // El anterior del actual ahora es el nuevo
    }
    // Si es mayor o igual, se inserta después
    else
    {
        nue->ant = act;        // El ant del nuevo es el actual
        nue->sig = act->sig;    // El sig del nuevo es el siguiente del actual
        // Si existe el nodo siguiente, se conecta al nuevo
        if (act->sig)
            act->sig->ant = nue; // El ant del nodo siguiente apunta al nuevo
        act->sig = nue;        // El siguiente del actual ahora es el nuevo
    }

    // Se actualiza la cabecera de la lista
    *lista = nue;

    return EXITO;
}
```

La función, aunque algo extensa, es comprensible en su lógica:

Primero, se verifica si la lista está vacía. En ese caso, simplemente se inserta el nuevo nodo utilizando la función correspondiente (`insPrinLista`), ya que no hay necesidad de buscar una posición específica.

Si la lista ya contiene elementos, se recorre hacia adelante o hacia atrás, según sea necesario, para encontrar la posición adecuada donde insertar el nuevo nodo respetando el orden definido por la función de comparación.

Una vez localizada la posición, se crea un nuevo nodo y se reserva memoria tanto para él como para la información que almacenará. Luego, se decide si debe insertarse antes o después del nodo actual comparando los contenidos. En ambas situaciones, se actualizan cuidadosamente los punteros `ant` y `sig` de los nodos vecinos para mantener la integridad de la estructura doblemente enlazada.

Finalmente, se actualiza la cabecera de la lista para que apunte al nuevo nodo insertado. Esto implica que la cabecera no necesariamente representa el primer nodo, sino el último nodo insertado.

Recursividad

Definición

- **Concepto**

La recursividad no es una funcionalidad misma, si no a la capacidad de resolver los mismos problemas de la materia, pero con un estilo de programación diferente. La recursividad es la técnica mediante la cual una función se invoca a sí misma directa o indirectamente para resolver un problema, eliminando completamente los bucles formales (for, while, etc...).

- **Ventajas y Desventajas**

Su utilización tiene ventajas como reducir en gran escala la cantidad de líneas de una función y casos especiales como en algoritmos (como la búsqueda binaria), donde se mejora la eficiente. Pero, en la mayoría de casos como un programa más genérico, su uso es ineficiente en cuanto procesador y memoria en programas, ya que siempre se queda en espera la función anterior. Por lo que no es recomendados en casos cuando existen estructuras que requieren muchas iteraciones, como los arrays, donde una solución iterativa es más eficiente que una recursiva.

- **Ejemplo (factorial):**

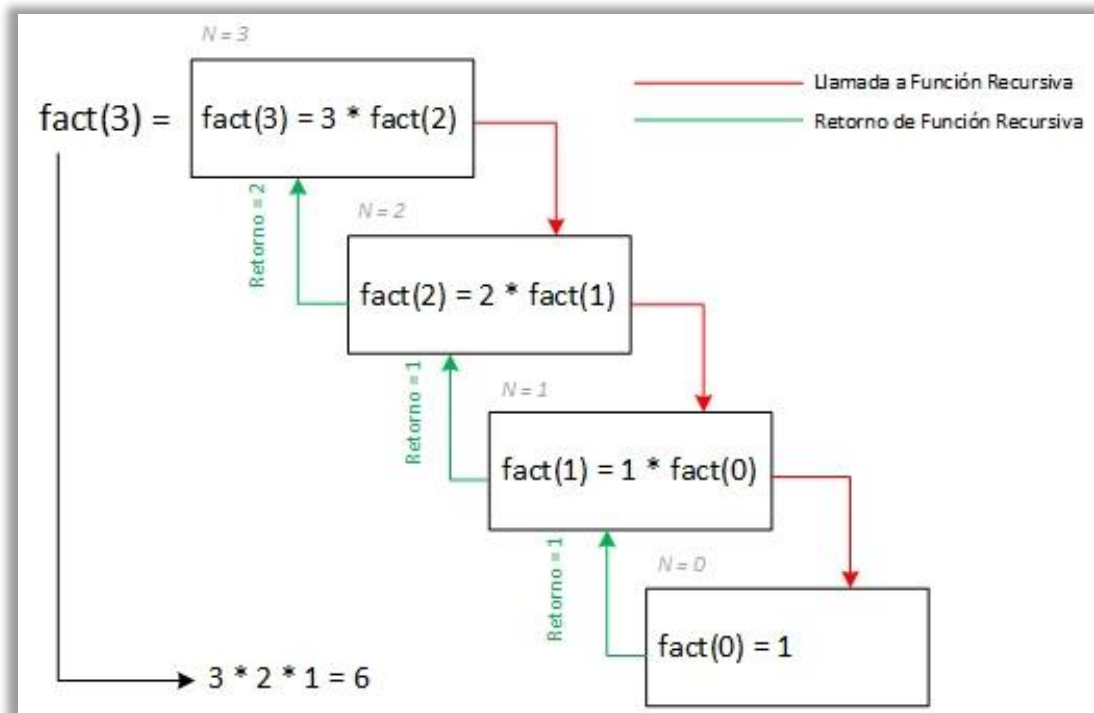
Se tiene la función matemática:

$$N! = \begin{cases} N * (N-1)! & N \geq 1 \\ 1 & N == 0 \end{cases}$$

Su función por medio de la iteración (sin recursividad) es la siguiente:

```
unsigned long long fact(unsigned num)
{
    unsigned long long acum = 1;
    while(num > 0)
    {
        acum *= num--;
    }
    return acum;
}
```

Si queremos implementarlo con recursividad debe ser de la siguiente manera:



El cálculo del factorial de un número N se descompone en múltiples llamadas a la función. Cada llamada corresponde a la evaluación de $N \cdot (N-1)!$ reduciendo el valor de N en cada nivel hasta llegar al caso base $N = 0$, donde $0! = 1$.

Cada línea roja representa la invocación a una función un valor N decreciente. Cada línea verde indica el valor de regreso calculado por el nivel anterior, comienza en `fact(0)`. Cada bloque representa la operación recursiva.

Este diagrama se repite en toda función recursiva, con la diferencia que el resultado de la función retornada puede ser operado o tan solo se propaga un resultado hasta volver al inicio.

Entendiendo ya ese diagrama, se puede implementar la función factorial:

```
unsigned long long fact(unsigned num)
{
    if(num == 0) // El factorial de 0 es 1
        return 1;
    return num * fact(num - 1);
}
```

Incluso, se puede reducir aún más las líneas de código:

```
unsigned long long fact(unsigned num)
{
    return num ? num * fact(num - 1) : 1;
}
```

[Funciones recursivas varias provistas de tópicos de programación.](#)

Operaciones para TDA Lista Simple

Insertar al Final

Cuando nos queremos desplazar por la lista con recursividad, es casi lo mismo que la función original. Tan solo, se cambia el bucle por una condición recursiva.

```
int insFinLista(tLista *lista, void *info, size_t tamInfo)
{
    // Avanzamos hasta posicionarnos en el campo sig del último nodo
    if(*lista)
        return insFinLista(&(*lista)->sig, info, tamInfo);

    // Como el resto del código es igual, re utilizamos la función
    return insPrinLista(lista, info, tamInfo);
}
```

Como se puede ver, lo único que cambia a diferencia de insertar final o en posición, es el desplazamiento recursivo.

**lista* apunta al nodo siguiente del que se encuentra posicionado, por lo que, si el campo es NULL, quiere decir que estamos en el último. No hay siguientes. Ahora, para retornar: pasamos la misma dirección de información y tamaño de la misma, el nodo que se va a enviar es el valor del campo **sig* del nodo siguiente que nos encontramos posicionados.

Ver Ultimo

Sigue un concepto similar.

```
int verFinLista(tLista *lista, void *info, size_t tamInfo)
{
    // Si la lista está vacía, no hay nada que ver
    if (!*lista)
        return LISTA_VACIA;

    // Si este es el último nodo, copiamos la info
    if (!(*lista)->sig)
    {
        memcpy(info, (*lista)->info, MIN(tamInfo, (*lista)->tamInfo));
        return EXITO;
    }

    // Si no es el último, seguimos recursivamente
    return verFinLista(&(*lista)->sig, info, tamInfo);
}
```

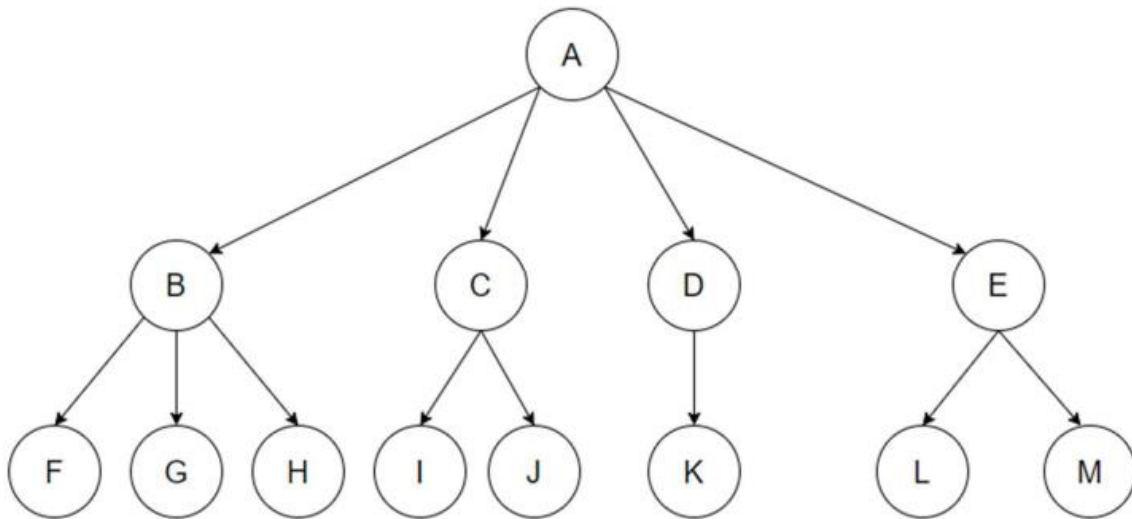
El problema que tiene esta función, es que estamos constantemente llamando la condicional si este vacío. Por lo que, se puede crear una nueva función `_verFinLista`, únicamente para ejecutar en la función del usuario si esta vacío y luego ejecutar el resto del código.

Como se puede ver, el código es casi lo mismo al insertar, solo le agregamos si esta vacío la lista y el tipo de desplazamiento. Para crear la función **outFinLista**, tan solo cambiamos la acción,

TDA Árbol Binario

Problemática y Estrategia

[Visualizador de Arboles](#)



Representación gráfica de un Árbol Binario.

- **Definición Árbol**

Un Árbol es una estructura de datos jerárquica donde este compuesto por:

- Nodos => Los nodos mismos aprendidos
- Aristas => Punteros que conectan los nodos

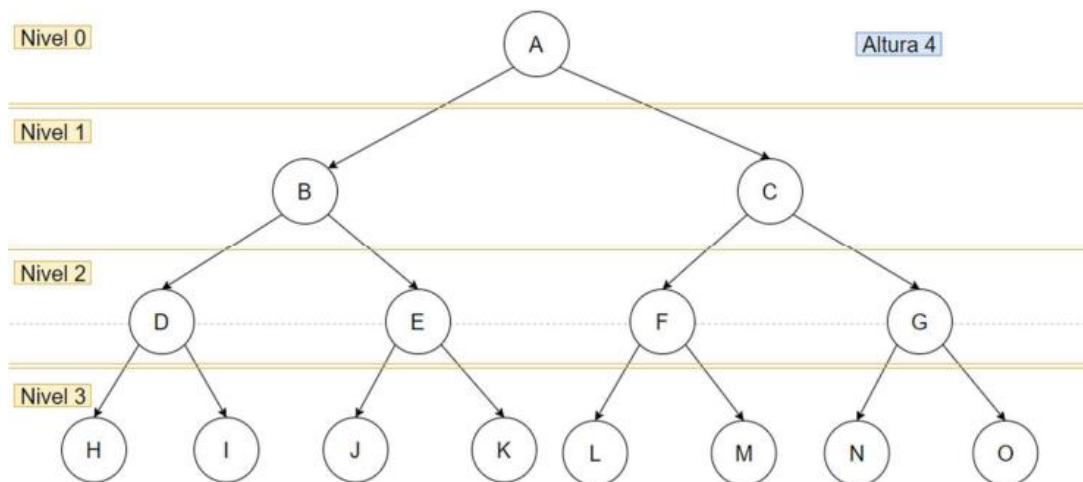
Todo árbol tiene una raíz (nodo inicial) donde se van a expandir más nodos. No debe haber ciclos; para llegar de un nodo a cualquier otro, solo debe existir un camino para llegar. Nosotros, vamos a utilizar arboles binarios ordenados: Cada nodo debe tener 2 caminos como mucho (0, 1 o 2), incluyendo la raíz.

- **Elementos de un Árbol**

- Nodo Padre o Raíz → Origen del árbol, no tiene padres
- Nodos Hijos u Hojas → Solo tiene padres, no tiene hijos o caminos.
- Nodos Intermedios → Tiene tanto padres como hijos
- Nodos Hermanos → Comparten el mismo padre

• Algunos Conceptos

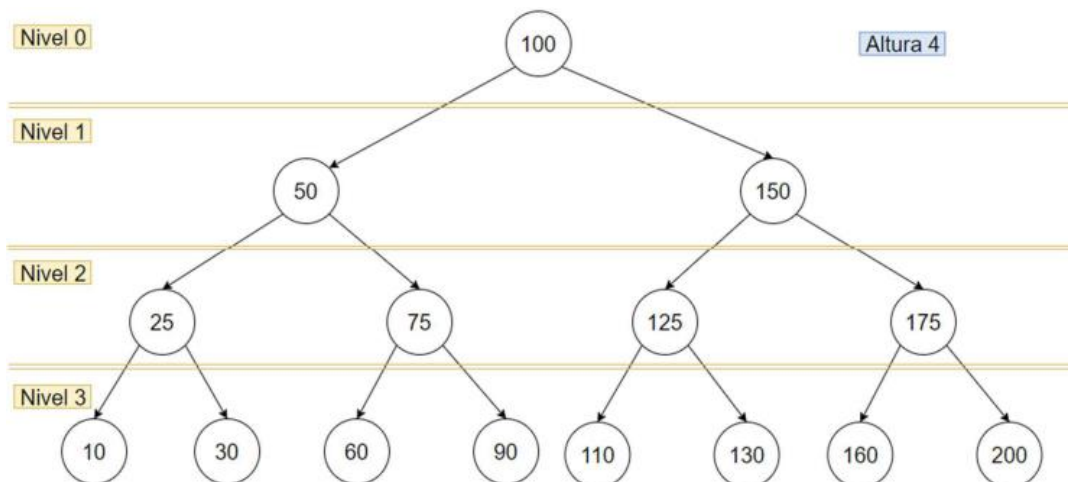
- Nivel del Nodo → Cuantos saltos se dan desde la raíz para llegar al nodo que nos interesa. La raíz siempre comienza desde cero.
- Altura del Árbol → Máxima altura o nivel más bajo que se puede llegar desde de la raíz.
- Árbol Completo → Tiene todos los nodos en el nivel más bajo, sin faltar alguno



• Características

Cada nodo tiene un ID para realizar búsquedas binarias con mayor eficiencia. Sigue la siguiente lógica:

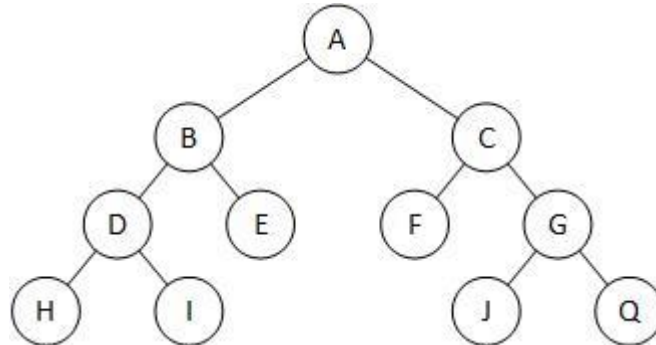
- Hijo izquierdo va un valor menor al actual
- Hijo derecho va un valor mayor al actual



Según el valor inicial que le demos a la raíz, podemos obtener mediante la fórmula $2^h - 1$, la cantidad máxima de nodos en el árbol. También, quiere decir que mientras menor sea la altura del árbol, más eficiente será la búsqueda.

Recorrer a Nivel Semántico

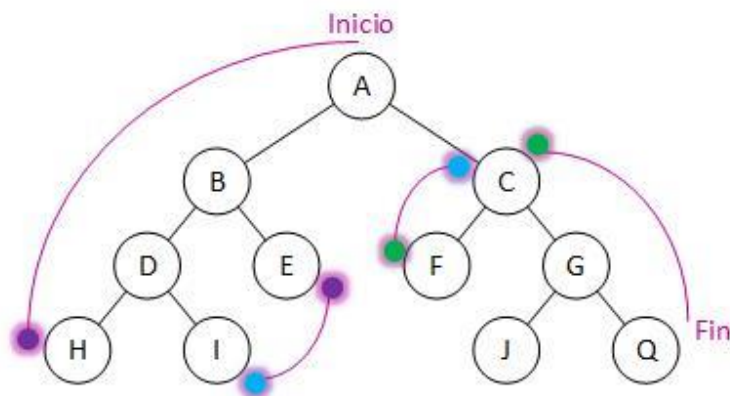
Vamos a utilizar este árbol, para representar a nivel grafico los recorridos



Pre-Orden – RID (Raíz, Izquierda, Derecha)

Se procesa primero la raíz, luego se recorre el subárbol izquierdo, y después el subárbol derecho. Sigue el siguiente orden:

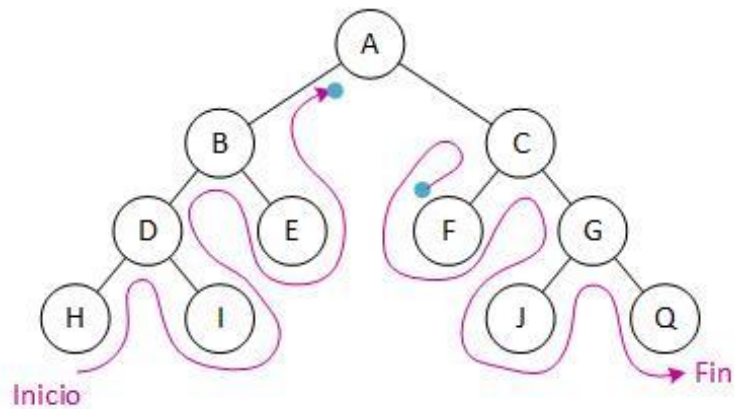
1. (R) - Empezar por Raíz
2. (I) - Recorrer subárbol izquierdo
3. (D) - Recorrer subárbol derecho



Lectura: A, B, D, H, I, E, C, R, G, J, Q

In Orden – IRD (Izquierda, Raíz, Derecha)

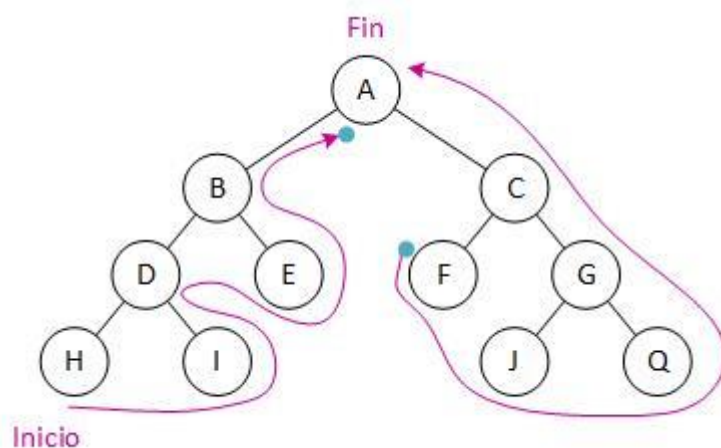
1. (I) - Recorrer sub árbol izquierdo
2. (R) - Raíz
3. (D) - Recorrer sub árbol derecho



Lectura: H, I, D, E, B, F, J, Q, G, C, A

Pos Orden – IRD (Izquierda, Derecha y Raíz)

1. (I) - Recorrer sub árbol izquierdo
2. (D) - Recorrer sub árbol derecho
3. (R) - Raíz



Lectura: H, D, I, B, E, A, F, C, J, G, Q

Estructura

```
typedef struct sNodo
{
    void *info; // Puntero a la información del nodo (guardada con malloc)
    size_t tamInfo; // Tamaño en bytes de la información
    struct sNodo *izq; // Puntero al hijo izquierdo
    struct sNodo *der; // Puntero al hijo derecho
} tNodoArbol;

typedef tNodoArbol* tArbol;
```

Crear Árbol

```
void crearArbol(tArbol *arbol)
{
    *arbol = NULL;
}
```

Recorrer a Código

Se utiliza la recursividad para recorrer árboles. Antes de entender como recorrer, se debe saber que, por cada nodo, se ejecuta la siguiente acción primitiva:

```
void accion(void *info, size_t tamInfo, size_t nivel, void *param);
```

Los distintitos recorridos que existen, utilizan casi el mismo código, solo cambia alguna línea y es la que se utiliza para indicar la raíz, izquierda o derecha.

- **Pre-Orden – RID (Raíz, Izquierda, Derecha)**

```
void mapPre(tArbol *arbol, void accion(void*,size_t,size_t,void*), void
*param, size_t nivel)
{
    // Si el árbol está vacío (llegamos a una rama que no existe), salimos
    sin hacer nada.
    if(!*arbol)
        return;
    /* R */accion((*arbol)->info,(*arbol)->tamInfo,nivel,param); // Actual
    /* I */mapPre(&(*arbol)->izq,accion,param,nivel+1); // Hijo izquierdo
    /* D */mapPre(&(*arbol)->izq,accion,param,nivel+1); // Hijo derecho
}
```

Este es el único recorrido que nos permite crear un árbol, a partir de un resultado de recorrido de árbol.

- **In Orden – IRD (Izquierda, Raíz, Derecha)**

```
void mapPre(tArbol *arbol, void accion(void*,size_t,size_t,void*), void
*param, size_t nivel)
{
    // Si el árbol está vacío (llegamos a una rama que no existe), salimos
    sin hacer nada.
    if(!*arbol)
        return;

    /* I */mapPre(&(*arbol)->izq,accion,param,nivel+1); // Hijo izquierdo
    /* R */accion((*arbol)->info,(*arbol)->tamInfo,nivel,param); // Actual
    /* D */mapPre(&(*arbol)->izq,accion,param,nivel+1); // Hijo derecho
}
```

- **Pos Orden – IRD (Izquierda, Derecha y Raíz)**

```
void mapPre(tArbol *arbol, void accion(void*,size_t,size_t,void*), void
*param, size_t nivel)
{
    // Si el árbol está vacío (llegamos a una rama que no existe), salimos
    sin hacer nada.
    if(!*arbol)
        return;

    /* I */mapPre(&(*arbol)->izq,accion,param,nivel+1); // Hijo izquierdo
    /* D */mapPre(&(*arbol)->izq,accion,param,nivel+1); // Hijo derecho
    /* R */accion((*arbol)->info,(*arbol)->tamInfo,nivel,param); // Actual
}
```

Contar

Cantidad de Nodos

```
size_t cantNodosArbol(tArbol *arbol)
{
    // Caso base: si el árbol está vacío, no hay nodos
    if(!*arbol)
        return 0;
    // Caso recursivo:
    // - Cuenta los nodos del subárbol izquierdo
    // - Cuenta los nodos del subárbol izquierdo
    // - suma 1 por el nodo actual
    return cantNodosArbol(&(*arbol)->izq)+cantNodosArbol(&(*arbol)->der)+1;
}
```

Si el puntero árbol es NULL, significa que no hay más nodos en esa rama, así que retorna 0. Si existe un nodo, suma:

- Cantidad de nodos del subárbol izquierdo
- Cantidad de nodos del subárbol derecho
- '+1' por el nodo actual

Altura del Árbol

```
#define MAX(X,Y) ( (X) > (Y) ? (X) : (Y) )

size_t alturaArbol(tArbol *arbol)
{
    // Caso base: si el árbol o rama está vacío, la altura es 0
    if(!*arbol)
        return 0;

    size_t altIzq = alturaArbol(&(*arbol)->izq); // Altura del subárbol izqui
    size_t altDer = alturaArbol(&(*arbol)->der); // Altura del subárbol derec

    // La altura del árbol es la mayor entre ambas subalturas + 1
    return MAX(altIzq, altDer) + 1;
}
```

Si el árbol o subárbol está vacío, retornamos 0. Caso contrario, para cualquier otro nodo:

- Llamada recursiva sobre su subárbol izquierdo y derecho
- Calcula cuál de las 2 ramas es más profunda
- A ese valor le suma 1 para utilizar el contador

Cantidad de Nodos Hijos y No Hijos

Cuenta la cantidad de hojas en un árbol binario. Ósea, los nodos que no tienen hijos tanto por derecha como por izquierda. Es una función muy similar a la anterior, pero con un condicional intermedio.

```
size_t contarHojas(tArbol* arbol)
{
    // Caso base: árbol vacío ⇒ no hay hojas
    if (!*arbol)
        return 0;

    // Si el nodo actual no tiene hijos, es una hoja
    if ((*arbol)->izq == NULL && (*arbol)->der == NULL)
        return 1;

    // Caso general: sumar hojas del subárbol izquierdo y derecho
    return contarHojas(&(*arbol)->izq) + contarHojas(&(*arbol)->der);
}
```

El código para contar la cantidad de nodos padres o no hijos, es más de lo mismo. Si el nodo actual tiene al menos un hijo, se suma 1 y se sigue contando por los hijos.

```
size_t contarNoHojas(tArbol* arbol)
{
    // Caso base: árbol vacío ⇒ no hay hojas
    if (!*arbol)
        return 0;

    // Si el nodo actual tiene al menos un hijo, es no-hoja
    if ((*arbol)->izq || (*arbol)->der)
        return 1+contarNoHojas(&(*arbol)->izq)+contarNoHojas(&(*arbol)->der);

    // Si no tiene hijos, no sumamos, solo seguimos recursión
    return 0;
}
```

Cantidad de Nodos hasta N Nivel

Cuenta la cantidad desde el nivel base (0) hasta el nivel que le indicamos por parámetros (incluyendo)

```
size_t cantNodosHastaNivel(tArbol* arbol, size_t nivel)
{
    // Si el árbol está vacío
    if (!*arbol)
        return 0;

    // Contar el nodo actual
    if(!nivel)
        return 1;

    // Contar el nodo actual + nodos de subárboles con nivel reducido
    return 1 +
        cantNodosHastaNivel(&(*arbol)->izq, nivel - 1) +
        cantNodosHastaNivel(&(*arbol)->der, nivel - 1);
}
```

La clave de esto, es que va reduciendo el nivel por 1 valor a la vez en cada retorno, y finaliza cuando este termina en 0.

Cantidad de Nodos en el Nivel N

Para esto, se necesitan 2 funciones:

- Una que llamara y utilizara el usuario donde entrega el arbol y nivelObjetivo. Se encarga de llamar la segunda función
- Otra, donde vamos decrementando la variable del nivelActual donde estamos posicionados, y mantenemos constante el nivelObjetivo.

Primera función:

```
size_t cantNodosEnNivel(tArbol* arbol, size_t nivel)
{
    // Se llama a la función recursiva auxiliar con nivelActual = 0 (raíz)
    return _cantNodosEnNivel(arbol, 0, nivel);
}
```

La segunda función. Verifica si estamos en el nivel deseado, si es así retorna 1. Caso contrario, se retorna de forma recursiva para el sub arbol izquierdo y derecho.

```
size_t _cantNodosEnNivel(tArbol* arbol, size_t nivelActual, size_t nivelObjetivo)
{
    // Si el árbol está vacío, no hay nodos para contar
    if (!*arbol)
        return 0;

    // Si estamos en el nivel objetivo, este nodo cuenta como uno
    if (nivelActual == nivelObjetivo)
        return 1;

    // sumar los nodos en los subárboles izquierdo y derecho
    // aumentando el nivel actual en cada llamada
    return _cantNodosEnNivel(&(*arbol)->izq, nivelActual + 1, nivelObjetivo)+
        _cantNodosEnNivel(&(*arbol)->der, nivelActual + 1, nivelObjetivo);
}
```

Cantidad de Nodos A Partir del Nivel N

Cuenta la cantidad de nodos que hay desde un cierto nivel hacia abajo (inclusive).

Insertar

El insertado en árbol se hace de forma predeterminada, de manera ordenada, es el único método de insertar. Por lo que, buscamos la posición adecuada con recursividad, e insertamos el nodo ahí. No obstante, la mayoría de código mostrado, ya es conocido por el usuario. Tenemos 2 formas:

```
int insArb(tArbol *arbol, void *info, size_t tamInfo, int cmp(void*, void*))
{
    tNodo *nue;
    int resCmp;

    // Recorremos iterativamente hasta la posición adecuada
    if(*arbol)
    {
        // Si es menor, va al subárbol izquierdo
        if((resCmp = cmp(info, (*arbol)->info)) < 0)
        {
            arbol = &(*arbol)->izq;
        }
        // Si es mayor, va al subárbol derecho
        else if(resCmp > 0)
        {
            arbol = &(*arbol)->der;
        }
        // Si es igual, no se inserta (no se permiten duplicados)
        else
        {
            return DUPLICADO;
        }
    }

    // Una vez llegada a la posición, creamos e insertamos el nodo
    if(!(nue = malloc(sizeof(tNodo))))
    {
        return ARBOL_LLENO;
    }

    if(!(nue->info = malloc(tamInfo)))
    {
        free(nue);
        return ARBOL_LLENO;
    }

    // Guardamos la información en el nuevo nodo
    nue->tamInfo = tamInfo;
    memcpy(nue->info, info, tamInfo);

    // Inicializamos los hijos como NULL
    nue->izq = NULL;
    nue->der = NULL;

    // Enlazamos el nuevo nodo al árbol
    *arbol = nue;

    return EXITO;
}
```

```

int insArb(tArbol *arbol, void *info, size_t tamInfo, int cmp(void*, void*))
{
    tNodo *nue;
    int resCmp;

    // Recorremos recursivamente hasta la posición adecuada
    if(*arbol)
    {
        // Si es menor, va al subárbol izquierdo
        if((resCmp = cmp(info, (*arbol)->info)) < 0)
        {
            return insArb(&(*arbol)->izq, info, tamInfo, cmp);
        }
        // Si es mayor, va al subárbol derecho
        else if(resCmp > 0)
        {
            return insArb(&(*arbol)->der, info, tamInfo, cmp);
        }
        // Si es igual, no se inserta (no se permiten duplicados)
        else
        {
            return DUPLICADO;
        }
    }

    // Una vez llegada a la posición, creamos e insertamos el nodo
    if(!(nue = malloc(sizeof(tNodo))))
    {
        return ERROR_MALLOC;
    }

    if(!(nue->info = malloc(tamInfo)))
    {
        free(nue);
        return ERROR_MALLOC;
    }

    // Guardamos la info en el nuevo nodo
    nue->tamInfo = tamInfo;
    memcpy(nue->info, info, tamInfo);

    // Inicializamos los hijos como NULL
    nue->izq = NULL;
    nue->der = NULL;

    // Enlazamos el nuevo nodo al árbol
    *arbol = nue;

    return EXITO;
}

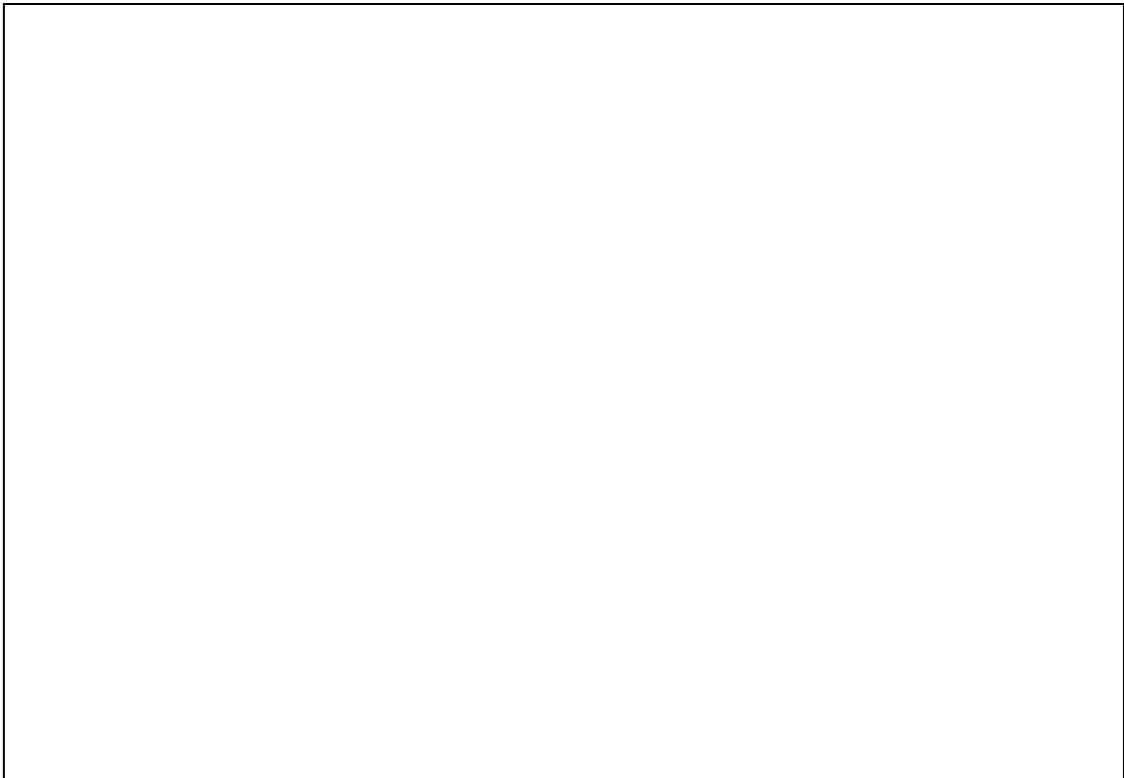
```

Lo único que cambia entre ambos códigos, es el desplazamiento.

Búsqueda en Árbol

Como ya se dijo anteriormente, la característica principal del árbol, es su eficiente para realizar búsquedas. Por lo que vamos a hacer una ahora.

Esta función, retorna la dirección de memoria del nodo de que la coincidió la búsqueda.



Índices
